

目 次

1. 研究活動と生成 AI

1.1 生成AIの現在	1
1.1.1 大規模言語モデル（LLM）の基本的な仕組み	1
1.1.2 Transformer アーキテクチャ—LLM を支える技術	2
1.1.3 ChatGPT の登場と生成 AI の進化	3
1.2 主要な AI ツールとその特徴	4
1.2.1 対 話 型 AI	4
1.2.2 検 索 特 化 型 AI	5
1.2.3 論 文 分 析 AI	6
1.2.4 コード支援AI	6
1.2.5 デ ザ イ ン AI	7
1.2.6 無料/有料の選び方ガイド	7
1.3 研究活動における AI 活用の可能性	8
1.3.1 研究活動の各場面での AI 活用	8
1.3.2 AI が得意なこと / 苦手なこと	10
1.3.3 「偶有的複雑さ」と「本質的複雑さ」	11
1.4 AI 利用の倫理と注意点	11
1.4.1 ハルシネーション（幻覚）の問題	12
1.4.2 著作権と剽窃	13
1.4.3 機密情報の漏洩リスク	13
1.4.4 各大学・学会の AI 利用ガイドライン	14

1.4.5 研究倫理における「AIの使用を明記する」原則	15
1.5 本書の構成と使い方	16
1.5.1 全10章の概観	16
1.5.2 15回授業での使い方	16
1.5.3 自習での使い方	17
演習問題	17
引用・参考文献	19

2. プロンプトエンジニアリング

2.1 なぜプロンプトが重要か	20
2.1.1 プロンプト次第で結果は大きく変わる	20
2.1.2 プロンプトエンジニアリングとは	21
2.2 ROCFモデル：良いプロンプトの4要素	22
2.2.1 Role（役割）：AIに専門家の役割を与える	22
2.2.2 Objective（目的）：何をしてほしいか明確に	23
2.2.3 Context（文脈）：背景情報を提供する	24
2.2.4 Format（形式）：出力形式を指定する	24
2.2.5 ROCFモデルの実践——4要素を組み合わせる	25
2.3 基本的なプロンプト技法	26
2.3.1 Zero-shot Prompting（例なしで指示）	26
2.3.2 Few-shot Prompting（例を添えて指示）	27
2.3.3 Chain-of-Thought（ステップバイステップの思考を促す）	28
2.3.4 各技法の比較と使い分け	29
2.4 高度なプロンプト技法	30
2.4.1 ペルソナ設定	30
2.4.2 段階的精緻化（Iterative Refinement）	31

2.4.3	メタプロンプティング	32
2.4.4	制約条件の活用	33
2.5	実践的なプロンプト設計技法	35
2.5.1	XML タグによるプロンプトの構造化	35
2.5.2	長文コンテキストの効果的な配置	36
2.5.3	引用ベースの回答で正確性を高める	37
2.5.4	指示の明確化：「やるな」より「代わりにこうして」	37
2.5.5	文脈と理由を添える	38
2.6	ハルシネーションの理解と対策	39
2.6.1	ハルシネーションとは何か	39
2.6.2	研究活動で特に危険なハルシネーション	39
2.6.3	5つの対策法	41
2.7	研究活動におけるプロンプト設計の実践	43
2.7.1	文献調査用プロンプトの設計	43
2.7.2	論文執筆用プロンプトの設計	45
2.7.3	コード生成用プロンプトの設計	46
2.7.4	プロンプトテンプレートの管理	47
	演習問題	48
	引用・参考文献	50

3. AIを使った文献調査

3.1	研究調査の課題	51
3.1.1	情報爆発の時代	51
3.1.2	キーワードの壁	52
3.1.3	時間的制約	52
3.1.4	思考の停滞	52

3.1.5 従来の調査プロセスと AI 活用の比較	53
3.2 文献調査 AI ツールの使い分け	54
3.2.1 対話型 AI — プレスト・壁打ちの相棒	54
3.2.2 検索特化型 AI — 出典付き回答でサーベイの初動	54
3.2.3 論文分析 AI — 深掘りのための専門ツール	55
3.2.4 目的別フローチャート	55
3.3 4ステップの文献調査ワークフロー	56
3.3.1 Step 1: キーワード拡張 — AI で関連キーワードを発見する	56
3.3.2 Step 2: 全体像を掴むサーベイ — Perplexity 等で俯瞰する	58
3.3.3 Step 3: 注目論文の深掘り — アブストラクトから構造的要約へ	59
3.3.4 Step 4: 研究の新規性を言語化する	61
3.4 情報の批判的検証	63
3.4.1 AI が出力した論文情報のファクトチェック手法	63
3.4.2 存在しない論文への対処（ハルシネーション問題）	64
3.4.3 バイアスの認識	65
3.5 文献管理とまとめ	66
3.5.1 調査結果の整理法	66
3.5.2 AI で文献レビューの草稿を作成する	67
3.5.3 文献管理ツールとの連携	67
演習問題	68
引用・参考文献	71

4. 英語論文の執筆と推敲

4.1 なぜ英語論文が必要か	72
4.1.1 国際会議・ジャーナルの重要性	72
4.1.2 研究成果の国際発信と評価	73

4.2 英語論文の基本構造	73
4.2.1 IMRaD 構造	73
4.2.2 各セクションの役割と書くべき内容	74
4.2.3 パラグラフの構成法	75
4.3 AI を使った英訳ワークフロー	76
4.4 推敲テクニック	79
4.4.1 段階的推敲法	79
4.4.2 比較推敲法	80
4.4.3 逆翻訳チェック	80
4.4.4 ネイティブチェック風レビュー	81
4.5 Abstract 作成の技法	82
4.5.1 Abstract の構造	82
4.5.2 150-200 語にまとめるプロンプト設計	82
4.5.3 良い Abstract と悪い Abstract の比較	82
4.6 よくあるミスと対策	84
4.6.1 すっきり最小主義：冗長な日本語を削ぎ落とす	84
4.6.2 まず具体化主義：曖昧な表現を具体的にする	86
4.6.3 とことん動詞主義：名詞句を動詞に戻す	87
4.6.4 能動態と受動態の使い分け	88
4.6.5 冠詞 (a/the) の使い分け	91
4.6.6 時制の選択	93
4.6.7 自動詞と他動詞の注意	95
4.6.8 直訳による不自然な表現	96
4.6.9 専門用語の確認方法	97
4.7 学術英語表現集	100
4.7.1 Introduction の定型表現	100
4.7.2 Methods の定型表現	100
4.7.3 Results の定型表現	100

4.7.4 Discussion / Conclusion の定型表現	100
引用・参考文献	103

5. 研究発表スライドの作成

5.1 研究発表スライドの基本	104
5.1.1 学会発表の基本構成	104
5.1.2 時間配分の目安（10 分発表の場合）	105
5.1.3 良いスライドの原則	106
5.2 AI による構成案の生成	106
5.2.1 研究テーマの構造化	106
5.2.2 AI に構成案を提案させるプロンプト設計	107
5.2.3 生成された構成案の批判的検討	108
5.3 各スライドの内容生成	109
5.3.1 研究背景の文章生成	109
5.3.2 研究手法の説明文生成	110
5.3.3 研究結果のまとめ生成	111
5.3.4 専門用語と平易な表現のバランス調整	111
5.3.5 AI の出力を自分の言葉に修正する重要性	111
5.4 デザイン AI によるスライド作成	112
5.4.1 Gamma — テキストからスライドを自動生成	112
5.4.2 Canva AI — テンプレートベースのデザイン支援	113
5.4.3 デザイン AI の使い分け	114
5.5 図表・概念図の生成	115
5.5.1 画像生成 AI による概念図	115
5.5.2 テキストベース図表ツール — Mermaid.js の活用	116
5.5.3 生成画像の著作権と正確性の検証	118

5.6 相互レビューと改善	118
5.6.1 ピアレビューのチェックポイント	118
5.6.2 AIに「聴衆として」フィードバックさせる	119
引用・参考文献	124

6. AIによるコード生成とテスト

6.1 AIとソフトウェア開発の現在	125
6.1.1 コード生成 AIの普及状況	125
6.1.2 開発者の AI 利用統計	125
6.1.3 AIが変える開発ワークフロー	126
6.2 AIによるコード生成の基本	127
6.2.1 基本フロー	127
6.2.2 効果的なプロンプトの4要素	127
6.2.3 Bad prompt vs Good prompt の比較	128
6.3 主要ツールの活用法	130
6.3.1 Gemini Pro / ChatGPT / Claude でのコード生成	130
6.3.2 GitHub Copilot の活用	131
6.3.3 対話的コード開発のテクニック	132
6.4 実践テクニック	133
6.4.1 段階的開発: 関数単位で生成・検証	133
6.4.2 テスト駆動: テストケースを先に書かせる	133
6.4.3 リファクタリング依頼	134
6.4.4 デバッグ支援: エラーメッセージの活用	134
6.5 テストコードの自動生成	135
6.5.1 pytest によるテストケース生成	135
6.5.2 テストカバレッジの考え方	138

6.5.3 AI 生成テストの検証ポイント	139
6.6 AI コードレビューとリファクタリング	139
6.6.1 AI にコードレビューを依頼するプロンプト	139
6.6.2 コードレビューチェックリスト	140
6.6.3 リファクタリングの種類と適用場面	141
6.6.4 ドキュメント自動生成	141
6.7 AI 生成コードの品質と注意点	142
6.7.1 セキュリティリスク	142
6.7.2 ライセンス問題	144
6.7.3 「信頼するが検証する」の原則	144
6.7.4 研究コードの再現性確保	145
引用・参考文献	149

7. データ分析と可視化

7.1 研究におけるデータ分析	150
7.1.1 なぜデータ分析が重要か	150
7.1.2 AI がデータ分析をどう変えるか	151
7.2 データ分析ワークフロー	151
7.2.1 Step 1: データの理解と前処理	151
7.2.2 Step 2: 探索的データ分析 (EDA)	154
7.2.3 Step 3: 統計的検定	155
7.3 論文品質のグラフ作成	157
7.3.1 matplotlib / seaborn の基本	157
7.3.2 効果的なグラフの選び方	158
7.3.3 論文に必要なグラフの要素	158
7.3.4 白黒印刷対応のグラフ設計	159

7.3.5 AIでグラフ作成コードを生成するプロンプト例	159
7.4 インタラクティブ可視化	160
7.4.1 Plotly の 紹 介	160
7.4.2 データ探索での活用	160
7.5 AIを使ったデータ分析の落とし穴	161
7.5.1 統計的な誤解	161
7.5.2 過学習への注意	161
7.5.3 AIが出した統計結果を鵜呑みにしない	162
7.6 実例：AIを用いたアンケート調査データの統計分析	162
7.6.1 研 究 の 概 要	162
7.6.2 AIとの協働の実例：コミット履歴から	163
7.6.3 AIが間違えた具体例	164
7.6.4 この事例から学べること	165
引用・参考文献	167

8. GitHub による研究成果管理

8.1 なぜ大学院生が GitHub を使うべきか	168
8.1.1 研究コードのバージョン管理	168
8.1.2 共同研究での活用	169
8.1.3 就職活動でのポートフォリオ効果	169
8.1.4 オープンサイエンスへの貢献	169
8.2 Git の 基 礎	170
8.2.1 バージョン管理とは何か	170
8.2.2 リポジトリ、コミット、ブランチの基本概念	170
8.3 GitHub の基本操作	171
8.3.1 GitHub アカウントの作成	171

8.3.2	GitHub Desktop のセットアップ	171
8.3.3	リポジトリの作成とクローン	172
8.3.4	コミット→プッシュの基本フロー	172
8.3.5	VS Code との連携	172
8.4	AI で Web ページを作成する	173
8.4.1	AI で HTML/CSS コードを生成する	173
8.4.2	VS Code での編集とプレビュー	173
8.4.3	AI でカスタマイズする	174
8.5	GitHub Pages で公開する	174
8.5.1	GitHub Pages とは	174
8.5.2	GitHub Pages の設定手順	174
8.5.3	更 新 フ ロ ー	174
8.6	研究ポートフォリオの構築	175
8.6.1	ポートフォリオに含めるもの	175
8.6.2	README.md の書き方	175
8.6.3	フォルダ構成のベストプラクティス	175
8.7	共同研究での GitHub 活用	176
8.7.1	Issue による課題管理	176
8.7.2	Pull Request によるコードレビュー	176
8.7.3	研究コードの共有とライセンス	177
	引用・参考文献	179

9. 就活文書の作成

9.1	大学院生の就職活動と AI	180
9.2	ガクチカとは何か	182
9.3	AI を使ったガクチカ作成ワークフロー	183

9.3.1 Step 1: 素材の棚卸し — AI に壁打ちしてもらう	183
9.3.2 Step 2: STAR 法での構造化	184
9.3.3 Step 3: AI で文章化 (400 字程度)	184
9.3.4 Step 4: AI で推敲・添削	185
9.4 研究を非専門家に伝える技法	186
9.5 ES 全体での AI 活用	187
9.6 面接準備での AI 活用	188
引用・参考文献	192

10. 研究の新規性分析と総合演習

10.1 なぜ新規性の言語化が重要か	193
10.2 新規性分析の 3 つの観点	194
10.2.1 観点 1: 方法論的新規性 — 新しい手法・アプローチの提案	194
10.2.2 観点 2: 実践的新規性 — 実世界への適用・検証	195
10.2.3 観点 3: 時間的観点 — 最新の研究動向の中での位置づけ	195
10.2.4 実例: 自動運転安全保証研究の新規性分析	196
10.3 AI を使った新規性分析ワークフロー	196
10.3.1 Step 1: 研究の位置づけマッピング	197
10.3.2 Step 2: 既存研究との差分分析	198
10.3.3 Step 3: 新規性の言語化	199
10.4 総合演習: 研究ポートフォリオの完成	199
10.5 AI 活用の振り返りと今後	201
引用・参考文献	205

A. 主要 AI ツールリファレンス

A.1	対話型 AI（汎用）	206
A.2	検索特化型 AI	207
A.3	論文分析・執筆支援 AI	207
A.4	コード支援 AI	208
A.5	デザイン・プレゼンテーション AI	208

B. プロンプトテンプレート集

B.1	文献調査用プロンプト	210
B.2	英語論文用プロンプト	212
B.3	コード生成用プロンプト	215
B.4	データ分析用プロンプト	217
B.5	スライド作成用プロンプト	218
B.6	ガクチカ・ES 用プロンプト	220

C. 学術英語表現集

C.1	Introduction で使える表現	223
C.2	Methods で使える表現	225
C.3	Results で使える表現	226
C.4	Discussion/Conclusion で使える表現	227
C.5	接続表現・論理展開表現	228

D. 15回授業計画

D.1	授業計画表	230
D.2	授業運営のヒント	232

D.3 授業の変形パターン	233
---------------------	-----

ま え が き

2022 年 11 月、OpenAI が ChatGPT を公開したとき、世界は大きく変わった。わずか 2 か月で 1 億人のユーザーを獲得したこの対話型 AI は、研究者の日常にも急速に浸透し始めた。文献調査、論文執筆、プログラミング、データ分析——あらゆる研究活動において、生成 AI は無視できない存在となっている。

しかし、その急速な普及に対して、大学院教育における体系的な指導は追いついていない。多くの大学院生は、SNS やネット記事の断片的な情報を頼りに、手探りで AI ツールを使っている。ある学生は便利さに感動して過度に依存し、別の学生は「使ってはいけないもの」と誤解して一切触れようとしない。どちらも、研究者として望ましい姿勢とは言えない。本書は、この状況を踏まえて執筆したものである。大学院生が生成 AI を批判的に、かつ効果的に活用するための体系的なガイドとして、本書を執筆した。

筆者は 2025 年度より、日本大学大学院における「情報論」の授業で、大学院生を対象に AI 活用の指導を行ってきた。授業を通じて痛感したのは、(1) 研究活動における AI 活用に焦点を当てた体系的な教材の不在、(2) プロンプトエンジニアリングの理論と実践のギャップ、(3) 何を AI に任せてよいのかという倫理的な判断力の必要性、の 3 点である。2025 年度、2026 年度の 2 年間にわたる授業実践を通じて蓄積した知見と、受講生からのフィードバックが、本書の土台となっている。

本書を貫く基本哲学は、「批判的 AI 活用」(Critical AI Usage) である。これは、AI を盲目的に信頼するのも、頭ごなしに拒否するのもない。AI の出力を常に批判的に検証し、自らの専門知識と判断力をもって取捨選択する姿勢のことである。具体的には、(1) AI は助手であり、研究者の代替ではない、(2) AI の出力は常に検証する、(3) AI の使用を透明にする、という 3 つの原則

に基づいている。

本書は、半期 15 回の授業に対応した構成となっている。第 1 章から第 10 章までの各章が 1〜2 回の授業に対応し、講義と演習を組み合わせることを想定している。自習で読み進める場合は、まず第 1 章と第 2 章を通読し、基本的な考え方と技法を身につけることを勧める。その後は、自分の研究活動で最も必要な章から読み進めてよい。

本書は、特定の AI ツールに依存しない設計としている。本文中では Chat-GPT、Claude、Gemini など多様なツールを例として取り上げるが、個々のツールの操作方法よりも、どのツールにも共通する活用の原則と思考法を重視している。AI 分野の進化は速く、書籍の出版後にも新たなツールや技法が登場することが予想されるため、GitHub リポジトリにて補足資料や更新情報を提供する予定である。

本書の執筆にあたり、多くの方々にお世話になった。日本大学大学院理工学研究科の「情報論」受講生の皆さんには、授業を通じて多くのフィードバックをいただいた。学生の率直な質問や試行錯誤の経験が、本書の内容を大きく豊かにしてくれた。心より感謝する。コロナ社編集部には、企画段階から丁寧なご助言をいただいた。Web サポートとの連携という出版形態を快くお認めいただいたことに感謝する。最後に、執筆期間中、常に支えてくれた家族に感謝したい。

2027 年

松野裕

1

研究活動と生成 AI

【この章で学ぶこと】

- 生成 AI の基本的な仕組みと主要ツールの特徴を理解する
- 研究活動における生成 AI の活用可能性と限界を認識する
- AI 利用における倫理的課題とリスクへの対処法を身につける

1.1 生成 AI の現在

1.1.1 大規模言語モデル（LLM）の基本的な仕組み

生成 AI（Generative AI）の中核を担うのが、**大規模言語モデル**（Large Language Model: LLM）である。まずは、その基本的な仕組みを直感的に理解しよう。

LLM は、一言で言えば「次に来る単語を予測する」モデルである。膨大なテキストデータ（Web ページ、書籍、論文など）を読み込み、言語のパターンを学習する。たとえば、「吾輩は」の次に「猫」が来る確率が高いことを、大量の日本語テキストから学習しているのである。

この「次の単語を予測する」という単純な原理から、なぜ高度な文章生成や質問応答が可能になるのか。それは、モデルの規模（パラメータ数）が一定の閾値を超えると、**創発的な能力**（emergent abilities）が現れるためだと考えら

れている。

LLM の学習は、大きく 3 つの段階で行われる。

LLM の学習 3 段階

第 1 段階：事前学習 (Pre-training)

インターネット上の大量のテキストデータを用いて、言語の基本的なパターンを学習する。この段階では、次の単語を予測する課題を繰り返し解くことで、文法、知識、推論能力などを獲得する。この学習には数千台の GPU を数か月間稼働させる必要があり、莫大な計算コストがかかる。

第 2 段階：ファインチューニング (Fine-tuning)

事前学習済みのモデルに対して、人間が作成した高品質な対話データを用いて追加学習を行う。これにより、モデルは「質問に対して適切に回答する」「指示に従って文章を生成する」といった能力を身につける。

第 3 段階：RLHF (Reinforcement Learning from Human Feedback)

人間のフィードバックに基づく強化学習である。モデルが生成した複数の回答を人間が評価し、より良い回答を優先的に学習させる。この段階により、モデルの出力は人間にとってより有用で、安全なものになる。

キーポイント: LLM は「知識のデータベース」ではなく、「言語パターンの統計的モデル」である。この理解が、後のハルシネーション（幻覚）の問題を正しく把握するための鍵となる。

1.1.2 Transformer アーキテクチャー—LLM を支える技術

現在の LLM のほぼすべてが、**Transformer** と呼ばれるニューラルネットワークアーキテクチャに基づいている。2017 年に Google の研究チームが発表した論文 “Attention Is All You Need”^{?)} で提案されたこのアーキテクチャは、

AI 研究に大きな影響を与えた。

Transformer の核心は「**注意機構** (Attention Mechanism)」である。これは、入力テキストの各単語が、他のすべての単語とどの程度関連しているかを計算する仕組みである。

たとえば、「教授が学生に論文の書き方を教えた。彼は丁寧に説明した。」という文において、「彼」が「教授」を指していることを理解するためには、文中の単語間の関係性を把握する必要がある。注意機構は、この関係性の把握を可能にする。

従来の手法 (RNN や LSTM) では、文の先頭から末尾まで順番に処理する必要があったため、長い文の処理が困難であった。Transformer は、文中のすべての単語間の関係を一度に (並列に) 計算できるため、長い文脈を効率的に扱えるようになった。

1.1.3 ChatGPT の登場と生成 AI の進化

生成 AI の歴史を振り返ると、2022 年 11 月がターニングポイントであった。OpenAI が ChatGPT を公開し、わずか 2 か月で 1 億ユーザーを獲得したのである。

その後の進化は目覚ましい。主要なマイルストーンを表表 1.1 に整理する。

表 1.1 生成 AI の主要なマイルストーン

時期	出来事	意義
2022 年 11 月	ChatGPT (GPT-3.5) 公開	対話型 AI の一般普及
2023 年 3 月	GPT-4 公開	マルチモーダル対応、推論能力の大幅な向上
2023 年 7 月	Claude 2 公開	長文脈処理、安全性重視の設計
2023 年 12 月	Gemini 公開	Google によるマルチモーダルモデル
2024 年	オープンソース LLM の台頭	Llama 3、Mistral など
2024~2025 年	AI エージェントの発展	自律的なタスク実行能力
2025 年	推論特化モデルの登場	o1、o3、DeepSeek-R1 など

研究活動に与えるインパクトという観点では、2024 年以降の「AI エージェント」と「推論特化モデル」の発展が特に注目すべきである。単なる質問応答を超えて、複数のステップにまたがる複雑なタスク（たとえば、文献検索→要約→比較分析→レポート作成）を一貫して実行できるようになりつつある。

キーポイント: 生成 AI の能力は急速に進化している。1 年前にできなかったことが、今日では可能になっている場合がある。常に最新の状況を把握する姿勢が重要である。

1.2 主要な AI ツールとその特徴

研究活動に活用できる AI ツールは多岐にわたる。ここでは、用途別に主要なツールを整理し、それぞれの強みと弱みを解説する。

1.2.1 対話型 AI

対話型 AI は、研究活動全般で最も頻繁に使用するツールである。

表 1.2 主要な対話型 AI ツールの比較

ツール	開発元	主な強み	主な弱み
ChatGPT (GPT-4o)	OpenAI	汎用性が高い、プラグイン豊富	料金プランが複雑
Claude	Anthropic	長文脈処理に優秀、日本語品質が高い	利用回数の制限がやや厳しい
Gemini Pro	Google	Google Workspace との連携	学術文献の取り扱いにやや課題

これらのツールは日々進化しており、上記の比較は執筆時点（2026 年）のものである。重要なのは、**単一のツールに固定せず、タスクに応じて使い分ける**ことである。

プロンプト例

【プロンプト例：ツールの比較を依頼する】

以下の研究タスクについて、最適な AI ツールの選択基準を教えてください。

タスク: 過去 5 年間の深層学習における画像認識手法の動向を調査し、サーベイ論文の骨子を作成したい。

考慮すべき点:

- 最新の論文情報へのアクセス
- 日本語での出力品質
- 長い文脈の維持

1.2.2 検索特化型 AI

従来のキーワード検索とは異なり、自然言語で質問を入力し、関連する情報を要約付きで取得できるツールである。

(a) **Perplexity** 自然言語で質問すると、Web から情報を収集し、出典付きで回答を生成する。研究の初期段階で、あるトピックの概要を素早く把握したい場合に有効である。

プロンプト例

【Perplexity への質問例】

“What are the current state-of-the-art methods for few-shot learning in natural language processing? Focus on papers published after 2023.”

(b) **Semantic Scholar** AI2 (Allen Institute for AI) が運用する学術論文検索エンジンである。2 億本以上の論文をインデックスしており、AI を用いた関連論文の推薦、引用関係の分析、TLDR (論文の超要約) 機能を提供する。

ヒント: Perplexity は「広く浅く」概要を把握するのに向いており、Semantic Scholar は「狭く深く」特定の学術トピックを調査するのに向いている。両者を組み合わせることで、効率的な文献調査が可能になる。

1.2.3 論文分析 AI

(a) **NotebookLM (Google)** PDF や論文をアップロードすると、その内容に基づいた対話が可能になる。複数の論文を同時に読み込ませ、横断的な質問をすることもできる。

プロンプト例

【NotebookLM の活用例】

1. 関連する論文を 5～10 本アップロードする
2. 「これらの論文に共通する研究課題は何ですか？」と質問する
3. 「論文 A と論文 B のアプローチの違いを表にまとめてください」と依頼する
4. 「これらの研究に残されている課題を列挙してください」と質問する

(b) **Elicit** 研究質問を入力すると、関連する論文を検索し、各論文の要約や結果を構造化して表示する。系統的な文献レビューの効率化に有効である。

(c) **Connected Papers** 1 本の論文を起点として、引用関係に基づく論文のネットワーク図を視覚的に表示する。研究分野の全体像を把握したり、重要な先行研究を見落としていないか確認したりするのに有効である。

1.2.4 コード支援 AI

(a) **GitHub Copilot** コードエディタ (VS Code 等) に統合され、コードの補完や生成を行う。コメントや関数名から意図を推測し、適切なコードを提案してくれる。

(b) **Cursor** AI 機能を統合した次世代のコードエディタである。コード全体の文脈を理解した上でのコード生成・修正が可能であり、「このファイル全体をリファクタリングして」といった高レベルの指示にも対応できる。

Listing 1.1 GitHub Copilot の活用例

```
1 # 以下のコメントを書くだけで、が適切なコードを生成するCopilot
2
3 # を使って、ファイルからデータを読み込み、pandasCSV
4 # 欠損値を平均値で補完し、
5 # 年齢別の統計量を計算する関数
6
7 def analyze_demographics(csv_path):
8     # がここから自動補完を行うCopilot
9     ...
```

1.2.5 デザイン AI

(a) **Gamma** テキストの入力から、スライドやドキュメントを自動生成する。研究発表のスライドのドラフト作成に活用できる。

(b) **Canva AI** グラフィックデザインツール Canva に統合された AI 機能。ポスター発表の作成や図表のデザインに活用できる。

1.2.6 無料/有料の選び方ガイド

大学院生にとって、コストは重要な考慮事項である。以下に、無料で利用できる範囲と、有料版を検討すべき場面を整理する。

無料で十分な場面:

- 初期の文献調査、ブレインストーミング
- 短い文章の校正・推敲
- 簡単なコード生成
- 基本的な質問応答

有料版の検討を勧める場面:

- 長文の論文執筆支援（コンテキスト長の制限）
- 大量のコード生成（回数制限の緩和）
- 高頻度の利用（日常的に研究に使う場合）
- 最新モデルへのアクセス

ヒント：多くの大学では、学生向けに GitHub Copilot の無料アカウントを提供している。また、Google Workspace を導入している大学では、Gemini の一部機能が利用可能な場合がある。所属大学の IT 部門に確認することを勧める。

1.3 研究活動における AI 活用の可能性

1.3.1 研究活動の各場面での AI 活用

研究活動は、大きく以下の場面に分けられる。それぞれの場面で、AI がどのように活用できるかを整理する。

（a）**文献調査** 研究の出発点である文献調査は、AI の活用効果が最も高い場面の一つである。

- 研究テーマに関連するキーワードの洗い出し
- 関連論文の検索と初期スクリーニング
- 論文の要約作成
- 複数論文の比較分析
- サーベイの構造化（年代別、手法別、応用分野別など）

プロンプト例

【プロンプト例：文献調査の出発点】

私は「強化学習を用いたロボットの歩行制御」について修士研究を始めます。以下について教えてください。

1. この分野の主要なサーベイ論文（2020 年以降）を 5 本挙げてください
2. 主要な研究グループ（大学・研究機関）を教えてください
3. この分野で現在最も注目されている課題は何ですか
4. 関連する国際会議のトップ 3 を教えてください

注意: 実在する論文のみを挙げてください。確信が持てない場合は、その旨を明記してください。

注意: AI が提示する論文タイトルや著者名は、ハルシネーション（架空の情報）である可能性がある。必ず Google Scholar や Semantic Scholar で実在を確認すること。

(b) 論文執筆

- アウトラインの作成と構成の検討
- 各セクションのドラフト作成
- 英語論文の校正・推敲
- 要旨（Abstract）の改善
- 関連研究セクションの整理

(c) 発表準備

- スライドの構成案作成
- 説明の流れの検討
- 想定質問の洗い出し
- 発表原稿の推敲

(d) データ分析

- 分析手法の選択に関するアドバイス
- 分析コードの生成
- 結果の解釈の支援
- 可視化のコード生成

(e) コード開発

- プロトタイプの迅速な開発
- デバッグの支援
- リファクタリング
- テストコードの生成

1.3.2 AIが得意なこと / 苦手なこと

AIを効果的に活用するためには、その得意・不得意を正しく理解することが欠かせない。

表 1.3 AI が得意なこと

カテゴリ	具体例
情報の整理・構造化	論文の要約、比較表の作成、アウトライン生成
言語変換	翻訳、パラフレーズ、文体の変換
パターンに基づく生成	定型的なコード、テンプレートに沿った文書
ブレインストーミング	アイデア出し、別の視点の提示
反復的な作業	大量のテキスト処理、フォーマット変換

表 1.4 AI が苦手なこと

カテゴリ	具体例
事実の正確性	最新の数値データ、固有名詞、引用情報
独創的な発想	真に新しい研究アイデアの創出
専門的な判断	実験条件の最適化、研究の方向性の決定
文脈の深い理解	研究分野固有の暗黙知、政治的配慮
数学的推論	複雑な証明、新しい定理の導出

1.3.3 「偶有的複雑さ」と「本質的複雑さ」

研究活動における AI 活用を考える上で、ソフトウェア工学者 Frederick P. Brooks の概念が有用である。Brooks は 1986 年の論文 “No Silver Bullet”⁷⁾ において、ソフトウェア開発の困難さを 2 種類に分類した。

本質的複雑さと偶有的複雑さ

本質的複雑さ (Essential Complexity)

問題そのものに内在する複雑さ。研究においては、「良い研究課題を設定する」「独自の仮説を立てる」「結果の意義を解釈する」といった知的活動がこれにあたる。AI で自動化することは困難であり、研究者自身の能力が問われる。

偶有的複雑さ (Accidental Complexity)

問題を解くための道具や方法に起因する複雑さ。研究においては、「論文のフォーマットを整える」「グラフを作成する」「定型的なコードを書く」「英語の文法をチェックする」といった作業がこれにあたる。AI で大幅に効率化できる領域である。

キーポイント: AI 活用の目標は、「偶有的複雑さ」にかかる時間と労力を削減し、「本質的複雑さ」に向き合うための時間を確保することである。AI に「偶有的複雑さ」を任せることで、研究者は本来注力すべき知的作業に集中できる。

1.4 AI 利用の倫理と注意点

生成 AI の活用には、大きな可能性がある一方で、看過できないリスクも存在する。研究者として、これらのリスクを正しく理解し、適切に対処する姿勢が求められる。

1.4.1 ハルシネーション（幻覚）の問題

ハルシネーション（Hallucination）とは、AI がもっともらしいが事実に反する情報を生成する現象である。研究活動で最も注意すべきリスクの一つである^{?)}。

ハルシネーションの典型例:

- **架空の論文引用:** 実在しない論文のタイトル、著者名、雑誌名、出版年をもっともらしく生成する。DOI まで付与されることもあるが、確認すると存在しない。
- **誤った数値データ:** 統計データや実験結果を尋ねた際に、もっともらしいが不正確な数値を回答する。
- **存在しない概念の説明:** 架空の用語や理論をあたかも確立されたものであるかのように説明する。

プロンプト例

【ハルシネーションの実例】

質問: “Self-attention 機構を提案した最初の論文を教えてください”

ハルシネーションを含む回答例（実際に AI が生成した例）:

“Self-attention 機構は、Cheng et al. (2016) の論文 ‘Long Short-Term Memory-Networks for Machine Reading’ で初めて提案されました。”

→ この論文は実在するが、self-attention の最初の提案としては不正確である。Vaswani et al. (2017)^{?)} の “Attention Is All You Need” が、現在の self-attention 機構の原型である。

注意: ハルシネーションは「AI のバグ」ではなく、LLM の動作原理に由来する本質的な特性である。完全に排除することは現時点では困難であり、ユーザー側の検証が必須である。

1.4.2 著作権と剽窃

AI が生成したテキストの著作権は、法的にも未確定の部分が多い。研究活動では、以下の点に注意が必要である。

AI の出力をそのまま使うことのリスク:

- AI の出力には、学習データに含まれるテキストの断片が含まれる可能性がある。そのまま論文に転用すると、意図せず他者の文章を剽窃してしまうリスクがある。
- AI が生成した文章を「自分の文章」として提出することは、多くの教育機関・学術雑誌で不正行為とみなされる。

適切な対処法:

1. AI の出力はあくまで「ドラフト」として扱い、自分の言葉で書き直す
2. AI 生成テキストを使用した場合は、その旨を明記する
3. 剽窃チェックツール（Turnitin 等）で確認する

1.4.3 機密情報の漏洩リスク

対話型 AI にテキストを入力すると、その内容がサービス提供者のサーバーに送信される。以下の情報は、AI に入力しないよう注意が必要である。

- 未発表の研究データ
- 特許出願前の技術情報
- 個人情報（被験者データなど）
- 企業との共同研究に関する機密情報
- パスワードや API キー

ヒント: 多くの AI サービスでは、入力データをモデルの学習に使用しないオプション（オプトアウト）が提供されている。機密性の高い情報を扱う場合は、必ずこの設定を確認すること。企業向けの API 版やエンタープライズ版では、データの学習利用がデフォルトで無効になっていることが多い。

1.4.4 各大学・学会の AI 利用ガイドライン

多くの大学や学会が、AI 利用に関するガイドラインを策定している。研究者は、自身が所属する機関や、論文を投稿する学会のガイドラインを必ず確認すべきである。

日本国内の動向:

- 文部科学省: 大学・高等教育における AI 利用に関する暫定的な考え方を公表（2023 年 7 月）¹⁾
- 日本学術会議: AI と研究倫理に関する提言を公表
- 各大学: 独自のガイドラインを策定（東京大学、京都大学など先行事例あり）

国際的な動向:

- Nature: AI 生成テキストの使用ポリシーを策定。AI を共著者として認めない立場。使用した場合は Methods 等で明記^{?)}。
- Science: 同様に AI の共著者としての記載を認めない。
- IEEE, ACM: 各学会で AI 利用に関するポリシーを策定。

プロンプト例

【論文における AI 使用の明記例】

In this study, we used ChatGPT (GPT-4, OpenAI) to assist with English language editing of the manuscript. All AI-generated suggestions were reviewed and revised by the authors. The AI tool was not used for data analysis, interpretation, or the generation of scientific conclusions.

1.4.5 研究倫理における「AI の使用を明記する」原則

研究活動で AI を使用した場合、何を AI に行わせたかを具体的に明記することが基本原則である。

明記すべき内容:

1. 使用した AI ツールの名称とバージョン
2. AI を使用した工程（文献調査、執筆支援、コード生成など）
3. AI の出力をどのように検証・修正したか

明記の場所:

- 論文の場合: Methods、Acknowledgements など（投稿先の規定に従う）
- 修士論文の場合: 指導教員と相談の上、適切な箇所に記載
- 授業のレポートの場合: 担当教員の指示に従う

キーポイント: AI の使用を隠すことは、研究倫理違反につながる。逆に、AI の使用を適切に開示した上で活用することは、現代の研究者にとって求められるスキルである。透明性を保つことが信頼の基盤である。

1.5 本書の構成と使い方

1.5.1 全10章の概観

本書は全10章で構成されている。各章の概要と、章間の関係を表表1.5に示す。

表 1.5 本書の全体構成

章	タイトル	内容の概要
1	研究活動と生成 AI	基礎知識、ツール概要、倫理（本章）
2	プロンプトエンジニアリング	効果的なプロンプトの設計原則と技法
3	AI を使った文献調査	文献検索、論文読解、サーベイ作成
4	英語論文の執筆と推敲	IMRaD 構造、英訳、段階的推敲
5	研究発表スライドの作成	構成案生成、デザイン AI、図表作成
6	AI によるコード生成とテスト	コード生成、テスト、レビュー、デバッグ
7	データ分析と可視化	統計分析、論文品質のグラフ作成
8	GitHub による研究成果管理	バージョン管理、Web ページ、ポートフォリオ
9	就活文書の作成	ガクチカ、ES、STAR 法、面接準備
10	研究の新規性分析と総合演習	新規性の言語化、ポートフォリオ統合

1.5.2 15 回授業での使い方

本書を大学院の授業で使用する場合の推奨スケジュールは、まえがきの表を参照されたい。各回は90分の授業を想定しており、前半を講義、後半を演習に充てることを推奨する。

授業で使用する場合の補足:

- 各章末の演習問題は、授業内演習または宿題として活用可能
- 第8週の間接発表では、各自の研究テーマに対する AI 活用計画を発表させることを推奨
- 第15週の総合演習では、実際の研究活動に AI を活用した成果を報告させる

1.5.3 自習での使い方

自習で本書を活用する場合は、以下の順序を推奨する。

1. **まず第 1 章と第 2 章を通読する（必須）**：AI の基本知識とプロンプトの書き方は、他のすべての章の基盤となる。
2. **自分の研究フェーズに応じた章を読む**：文献調査の段階なら第 3 章、論文執筆の段階なら第 6 章など。
3. **演習問題に取り組む**：読むだけでなく、実際に手を動かすことが重要。各章の演習問題は、知識の定着を助けるよう設計されている。
4. **GitHub の補足資料を確認する**：本書の出版後に更新された情報や、追加のプロンプトテンプレートが掲載されている。

演 習 問 題

1. 問題 1.1（基礎知識の確認）

大規模言語モデル（LLM）の学習における 3 つの段階（事前学習、ファインチューニング、RLHF）について、それぞれの目的と特徴を 200 字程度で説明せよ。

2. 問題 1.2（ツールの比較）

ChatGPT、Claude、Gemini Pro の 3 つのツールについて、以下の研究タスクそれぞれに最も適したツールを選び、その理由を述べよ。

- (a) 英語論文の Abstract を日本語で要約する
- (b) 50 ページ以上の論文をアップロードして質問する
- (c) Google スプレッドシートのデータを分析するコードを生成する

3. 問題 1.3（本質的複雑さと偶有的複雑さ）

あなた自身の研究活動（または想定する研究活動）において、「本質的複雑さ」に分類される作業と「偶有的複雑さ」に分類される作業をそれぞれ 5 つ挙げ、AI の活用可能性について論じよ。

4. 問題 1.4 (倫理的判断)

以下の各場面で、AI の利用は適切かどうかを判断し、その理由を述べよ。
また、利用する場合にどのような注意が必要かを述べよ。

- (a) 修士論文の先行研究セクションのドラフトを AI に書かせ、自分で修正した
- (b) 学会発表のスライドの構成案を AI に考えさせた
- (c) 実験データの統計分析のコードを AI に書かせた
- (d) 研究費申請書の研究計画を AI に書かせ、そのまま提出した
- (e) AI が生成した文献リストを検証せずに論文に掲載した

5. 問題 1.5 (実践演習)

任意の対話型 AI ツールを使用して、以下を実行せよ。

- (a) あなたの研究テーマ（または興味のあるテーマ）について、AI に概要を説明させる
- (b) その回答に含まれる事実関係を 3 つ以上選び、それぞれについて信頼できる情報源（論文、公式ドキュメントなど）で正確性を検証する
- (c) 検証結果をレポートにまとめ、AI の回答の正確性を評価する

引用・参考文献

- 1) 文部科学省. 大学・高専における生成 AI の教学面の取扱いについて（暫定的な考え方）. https://www.mext.go.jp/b_menu/houdou/2023/mext_01260.html, 2023.

2

プロンプトエンジニアリング

【この章で学ぶこと】

- 効果的なプロンプトの設計原則（ROCF モデル）を理解する
- Zero-shot、Few-shot、Chain-of-Thought などの技法を使い分けられる
- ハルシネーションを検出し対処する方法を身につける

2.1 なぜプロンプトが重要か

2.1.1 プロンプト次第で結果は大きく変わる

生成 AI から良い出力を得るためには、良い入力（プロンプト）が欠かせない。同じ目的であっても、プロンプトの書き方によって、得られる回答の質は大きく異なる。

以下の例を見てほしい。ある修士学生が、自分の研究テーマに関する先行研究を調べたい場面を想定する。

プロンプト例

【悪いプロンプトの例】

深層学習の論文を教えて

この質問に対して、AIは「深層学習」という広大な分野の中から、何を回答すべきか判断できない。結果として、一般的すぎる回答や、質問者の意図とズレた回答が返ってくることが多い。

プロンプト例

【良いプロンプトの例】

私は修士1年で、「Few-shot 学習を用いた製造業における
外観検査の自動化」をテーマに研究を始めます。

以下の条件で、関連する先行研究を5本紹介してください。

条件:

- 2021 年以降に発表された査読付き論文
- Few-shot learning または Meta-learning の手法を用いている
- 製造業の品質管理に関連する応用研究
- 各論文について、タイトル、著者、掲載誌、主要な貢献を
100 字程度で説明してください

注意: 実在する論文のみを挙げてください。確信が持てない場合は「確認が必要」と明記してください。

このプロンプトでは、背景（修士1年で研究を始める段階）、具体的な研究テーマ、求める情報の条件、出力形式が明確に指定されている。AIはこれらの情報を手がかりに、より適切な回答を生成できる。

キーポイント: プロンプトは「AIへの仕様書」である。ソフトウェア開発において曖昧な仕様書から良い製品が生まれないのと同様に、曖昧なプロンプトから良い回答は生まれない。

2.1.2 プロンプトエンジニアリングとは

プロンプトエンジニアリング（Prompt Engineering）とは、AIから望ましい出力を得るために、入力（プロンプト）を設計・最適化する技術のことである。

る?)。

プロンプトエンジニアリングは、単なる「質問の仕方」にとどまらない。それは以下の要素を含む、体系的な設計プロセスである。

1. **目的の明確化**: AI に何をさせたいのかを明確にする
2. **文脈の設計**: AI が適切な回答を生成するために必要な背景情報を選定する
3. **制約の指定**: 出力の形式、長さ、スタイルなどの条件を設定する
4. **反復的改善**: 結果を評価し、プロンプトを修正するサイクルを回す

研究者にとってプロンプトエンジニアリングは、今後ますます重要なスキルとなる。それは、AI の能力を引き出すためにも、AI の限界を見極めるためにも重要である。

2.2 ROCF モデル：良いプロンプトの4要素

本書では、効果的なプロンプトを設計するためのフレームワークとして、**ROCF** モデルを提案する。ROCF は以下の4要素の頭文字である。

ROCF モデルの4要素

- **Role**（役割） — AI に専門家の役割を与える
- **Objective**（目的） — 何をしてほしいか明確に
- **Context**（文脈） — 背景情報を提供する
- **Format**（形式） — 出力形式を指定する

2.2.1 Role（役割）：AI に専門家の役割を与える

AI に特定の専門家の役割を付与することで、その分野に適した語彙、視点、判断基準で回答を生成させることができる。

プロンプト例

【Role 要素の例】

例 1: “あなたは計算機科学分野の経験豊富な査読者です。”

例 2: “あなたは統計学の教授で、大学院生の研究指導を行っています。”

例 3: “あなたはテクニカルライターで、学術論文の英語校正を専門としています。”

Role の効果は、単に回答のトーンを変えるだけではない。たとえば、「査読者として」と指定すると、AI は論文の弱点や改善点を指摘する方向で回答を生成する傾向がある。「指導教員として」と指定すると、学生の理解を助けるような丁寧な説明を生成しやすくなる。

ヒント: Role は具体的であるほど効果的である。「専門家として」よりも「機械学習分野で 20 年の研究経験を持つ教授として、修士学生の論文を指導する立場で」と指定した方が、より適切な回答を得やすい。

2.2.2 Objective (目的)：何をしてほしいか明確に

Objective は、AI に実行してほしいタスクを明確に指定する要素である。曖昧な依頼は曖昧な結果を生む。

プロンプト例

【Objective 要素の比較】

曖昧な目的: “この論文について教えてください”

明確な目的: “この論文の研究手法と実験結果を、以下の 3 つの観点から分析してください:

(1) 新規性、(2) 再現性、(3) 既存手法との比較”

Objective を書く際のポイント:

1. **動詞を明確にする:** 「教えてください」ではなく「要約してください」「比

較してください」「評価してください」など

2. **範囲を限定する**：「すべて」ではなく「上位3つ」「主要な点」「特にXに関して」など
3. **成功基準を示す**：何をもって良い回答とするかを伝える

2.2.3 Context（文脈）：背景情報を提供する

Context は、AI が適切な回答を生成するために必要な背景情報である。研究活動では特に重要な要素となる。

プロンプト例

【Context 要素の例】

“以下の背景を踏まえて回答してください：

- 私は情報工学専攻の修士1年で、深層学習を用いた自然言語処理が専門です
- 対象とする言語は日本語です
- 使用可能な計算資源は GPU 1 台（NVIDIA A100）です
- 投稿先は ACL 2027 を予定しています
- 研究期間は1年半です”

適切な Context を提供することで、AI はより現実的で実行可能な提案を行えるようになる。たとえば、「GPU 1 台」という制約を伝えれば、計算コストが膨大なモデルの提案を避け、効率的なアプローチを推薦してくれる。

2.2.4 Format（形式）：出力形式を指定する

Format は、AI の出力がどのような形式であるべきかを指定する要素である。

プロンプト例

【Format 要素の例】

例 1: “箇条書きで 5 項目にまとめてください”

例 2: “表形式で比較してください (列: 手法名、精度、計算コスト、利点、欠点) ”

例 3: “ $\text{\LaTeX}$ 形式で数式を含めて記述してください”

例 4: “以下の JSON 形式で出力してください”

2.2.5 ROCF モデルの実践——4 要素を組み合わせる

4 つの要素を組み合わせた実践的なプロンプトの例を示す。

プロンプト例

【ROCF モデルの完全な適用例】

[Role] あなたは自然言語処理分野の研究に精通した大学教授です。修士学生の研究指導を日常的に行っています。

[Objective] 以下の研究テーマについて、修士論文の構成案（章立て）を提案してください。各章の概要と、そこで扱うべき内容を 200 字程度で説明してください。

[Context]

- 研究テーマ: 「事前学習済み言語モデルを用いた日本語法律文書の自動要約」
- 修士 2 年の学生で、修士論文の提出は 6 か月後です
- すでにベースラインの実験は完了し、提案手法の実装中です
- 関連研究として、BERT を用いた法律文書分類の研究はすでにサーベイ済みです

[Format] 以下の形式で出力してください:

第 X 章 [章タイトル] / 概要: [200 字程度の説明] / 含めるべき内容: [項目リスト]

キーポイント: ROCF モデルの 4 要素すべてを毎回使う必要はない。タスクの性質に応じて、必要な要素を選択すればよい。ただし、Objective と Context は常に含めることを推奨する。

2.3 基本的なプロンプト技法

2.3.1 Zero-shot Prompting (例なしで指示)

Zero-shot prompting は、具体例を一切提示せず、指示文のみでタスクを実行させる技法である。最も基本的なプロンプト技法であり、明確なタスクに

は十分に機能する。

プロンプト例

【Zero-shot prompting の例】

以下の論文タイトルを、学術的な日本語に翻訳してください。

“Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”

適している場面:

- タスクが明確で、AI が十分な学習データを持っている場合
- 翻訳、要約、校正などの定型的なタスク
- 簡単な質問応答

注意点:

- タスクの定義が曖昧な場合、AI が意図と異なる解釈をする可能性がある
- 出力形式を厳密に制御したい場合は、追加の指定が必要

2.3.2 Few-shot Prompting (例を添えて指示)

Few-shot prompting は、望ましい入出力の例を数個提示した上で、タスクを実行させる技法である²⁾。AI に「こういう形式で、こういう品質の回答を期待している」というお手本を示すことで、出力の品質と一貫性が大幅に向上する。

プロンプト例

【Few-shot prompting の例：論文の分類】

以下の論文タイトルを、研究分野に分類してください。

例 1:

タイトル: “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”

分野: 自然言語処理 > 事前学習モデル

例 2:

タイトル: “ResNet: Deep Residual Learning for Image Recognition”

分野: コンピュータビジョン > 画像認識

例 3:

タイトル: “Playing Atari with Deep Reinforcement Learning”

分野: 強化学習 > ゲーム AI

では、以下の論文を分類してください:

タイトル: “Generative Adversarial Nets”

分野:

適している場面:

- 出力の形式や品質を厳密に制御したい場合
- AI があまり得意でないタスク（特殊な分類基準など）
- 独自の評価基準や記述スタイルを適用したい場合

Few-shot の例数の目安:

- 2〜5 個の例があれば、多くのタスクで十分
- 例が多すぎると、コンテキスト長を浪費する
- 例は多様性を持たせる（似た例ばかりにしない）

2.3.3 Chain-of-Thought（ステップバイステップの思考を促す）

Chain-of-Thought (CoT) prompting は、AI に思考過程を段階的に出

力させる技法である^{?)}。複雑な推論が必要なタスクで、回答の正確性が著しく改善される。

プロンプト例

【Chain-of-Thought prompting の例：研究手法の選択】

以下の研究課題に対して、最適な実験手法を選択してください。

ステップバイステップで考え、各ステップの根拠を示してください。

研究課題：「SNS 投稿のテキストから、投稿者の感情状態を 4 カテゴリー（喜び、怒り、悲しみ、恐怖）に分類するモデルを開発したい。ただし、ラベル付きデータは各カテゴリ 100 件しかない。」

ステップ 1: まず、この課題の本質的な要件を整理してください

ステップ 2: 考えられる手法を 3 つ以上列挙してください

ステップ 3: 各手法のメリット・デメリットを分析してください

ステップ 4: データ量の制約を考慮して、最適な手法を推薦してください

ステップ 5: 推薦した手法の実装計画を簡潔に示してください

Chain-of-Thought の効果が特に高い場面:

- 数学的な推論が必要な場合
- 複数の要因を考慮した判断が必要な場合
- 複雑な比較・分析が必要な場合

キーポイント: 「ステップバイステップで考えてください」という一文を追加するだけでも、回答の質が向上することが多い。これは、AI が中間的な推論過程を生成することで、最終的な結論の正確性が高まるためである。

2.3.4 各技法の比較と使い分け

実際の研究活動では、これらの技法を組み合わせて使うことが多い。たとえば、Few-shot の例に Chain-of-Thought の思考過程を含めることで、「こうい

表 2.1 プロンプト技法の比較

技法	適用場面	例の必要数	効果
Zero-shot	明確な定型タスク	0	速い、簡潔
Few-shot	形式の制御が必要	2～5	出力品質の安定化
Chain-of-Thought	複雑な推論	0～3	推論精度の向上

う思考プロセスで、こういう形式の回答を期待している」ということを AI に伝えることができる (Few-shot CoT)。

2.4 高度なプロンプト技法

基本的な技法を習得したら、次はより高度な技法に挑戦しよう。これらの技法は、研究活動の中でも特に複雑なタスクで効果的である。

2.4.1 ペルソナ設定

ペルソナ設定は、ROCF モデルの Role 要素をさらに発展させた技法である。AI に特定の視点から回答させることで、多角的な分析や批判的なフィードバックを得ることができる。

プロンプト例

【ペルソナ設定の実践例：論文の多角的レビュー】

— ペルソナ 1: 厳格な査読者 —

あなたは、トップカンファレンス (NeurIPS, ICML) の査読を 10 年以上務めてきた厳格な研究者です。以下の論文概要に対して、主要な弱点と改善すべき点を 5 つ指摘してください。

特に、技術的な新規性と実験の妥当性について厳しく評価してください。

[論文概要をここに貼る]

プロンプト例

— ペルソナ 2: 好意的な指導教員 —

あなたは、修士学生の研究指導に情熱を持つ教授です。

以下の論文概要に対して、良い点を3つ、改善の余地がある点を3つ指摘し、各改善点に対して具体的なアドバイスを提供してください。学生のモチベーションを維持しながら、建設的なフィードバックを心がけてください。

[論文概要をここに貼る]

プロンプト例

— ペルソナ 3: 隣接分野の研究者 —

あなたは、情報検索（IR）分野の研究者です。

自然言語処理（NLP）の専門家ではありませんが、テキスト処理に関連する知識を持っています。以下の論文概要を読んで、IR 分野の研究者として疑問に思う点や、IR 分野との接点を指摘してください。

[論文概要をここに貼る]

同じ論文概要に対して、異なるペルソナからのフィードバックを得ることで、自分では気づかなかった観点や問題点を発見できることがある。

2.4.2 段階的精緻化 (Iterative Refinement)

段階的精緻化は、AI との対話を複数ターンに分け、出力を少しずつ改善していく技法である。一度で完璧な出力を求めるのではなく、反復的に品質を高めていく。

プロンプト例

【段階的精緻化の例：研究要旨の作成】

— ターン 1: ドラフト生成 —

以下の情報をもとに、国際会議に投稿する論文の Abstract（250 語以内、英語）のドラフトを作成してください。

研究テーマ: [テーマ] / 手法: [手法の概要] / 結果: [主要な結果] / 貢献: [主要な貢献]

— ターン 2: 構造の改善 —

ありがとうございます。以下の点を修正してください: (1) 最初の文で、研究の動機をより明確にしてください (2) 手法の説明をもう少し具体的にしてください (3) 結果の数値を追加してください: 精度 92.3%, ベースライン比+4.7 ポイント

— ターン 3: 言語の洗練 —

構造は良くなりました。次に、以下の観点で言語面を改善してください: (1) 受動態を能動態に変えられる箇所を修正 (2) 冗長な表現を簡潔にする (3) 学術的な語彙として不適切な表現があれば修正

段階的精緻化のメリット:

- 各ステップで焦点を絞れるため、改善の方向が明確になる
- AI の出力を確認しながら修正指示を出せるため、最終的な品質が高くなる
- 人間の意図を AI に徐々に正確に伝えることができる

2.4.3 メタプロンプティング

メタプロンプティングは、AI に「プロンプト自体を考えさせる」技法である^{?)}。自分のタスクに最適なプロンプトがわからない場合に有効である。

プロンプト例

【メタプロンプティングの例】

私は以下のタスクを実行したいのですが、最適なプロンプトの書き方がわかりません。

タスク: 修士論文の「関連研究」セクションを書くために、10本の論文を体系的に整理し、比較分析したい。

このタスクに対して、最も効果的なプロンプトを設計してください。以下の点を含めてください:

1. プロンプトの全文
2. なぜその書き方が効果的なのかの説明
3. プロンプトをさらに改善するためのヒント

メタプロンプティングは、プロンプトエンジニアリングの学習そのものにも有効である。AIが設計するプロンプトを分析することで、効果的なプロンプトの特徴を学ぶことができる。

2.4.4 制約条件の活用

AIの出力を精密に制御するためには、適切な制約条件を設定することが有効である。

プロンプト例

【制約条件の活用例】

以下の制約条件に従って、研究計画書のドラフトを作成してください。

制約条件:

1. 文字数: 2000 字以内
2. 構成: 背景、目的、手法、期待される成果の 4 セクション
3. 文体: 「である」調、学術的な表現
4. 含めるべきキーワード: 深層学習、転移学習、ドメイン適応
5. 避けるべき表現: 「画期的」「世界初」などの誇張表現
6. 参考にすべきスタイル: 科研費申請書の研究計画調書
7. 対象読者: 情報工学分野の審査委員 (AI 専門とは限らない)

効果的な制約条件のカテゴリを表表 2.2 に示す。

表 2.2 制約条件のカテゴリと例

カテゴリ	例
長さの制約	「300 字以内」「5 項目で」
形式の制約	「表形式で」「 LATEX で」「箇条書きで」
内容の制約	「技術的な詳細は省く」「数式を含める」
否定の制約	「専門用語を使わない」「主観的な評価は含めない」
品質の制約	「学術論文レベルの正確さで」「初学者にもわかるように」
スタイルの制約	「である調」「フォーマルな英語」

ヒント: 制約条件は多すぎると逆効果になることがある。AI が全ての制約を同時に満たそうとして、かえって不自然な出力になる場合がある。重要度の高い制約を 3~5 個に絞ることを推奨する。

2.5 実践的なプロンプト設計技法

前節までに紹介した技法に加え、AI サービス提供企業が公開しているベストプラクティスから、研究活動に役立つ実践的な技法を紹介する。本節では、特に Anthropic のプロンプトエンジニアリングガイドに基づく手法を取り上げる。

2.5.1 XML タグによるプロンプトの構造化

複雑なプロンプトを作成する際、**XML タグ**を用いてプロンプトの各部分を明示的に区切ることで、AI が指示・文脈・入力データを曖昧さなく解釈できるようになる。たとえば、指示部分を<instructions>タグで、分析対象の文書を<document>タグで囲むといった方法である。

この技法は、指示文、背景情報、具体的なデータ、出力形式の指定など、複数の異なる種類の情報を 1 つのプロンプトに含める場合に特に有効である。タグによって各要素の境界が明確になるため、AI が指示の一部をデータと誤認したり、データの一部を指示として解釈するといった問題を防ぐことができる。

研究活動では、論文を AI に分析させる場面でこの技法が威力を発揮する。論文テキストを<document>タグで囲み、分析の依頼を別のタグで明示することで、AI は「何が分析対象で、何が指示なのか」を正確に把握できる。

プロンプト例

【XML タグを用いた構造化プロンプトの例】

```
<instructions>
```

以下の論文のアブストラクトを読み、背景・手法・結果・限界の4点で要約してください。

```
</instructions>
```

```
<document>
```

[論文のアブストラクトをここに貼り付け]

```
</document>
```

```
<output_format>
```

箇条書きで、各項目 2-3 文程度にまとめてください。

```
</output_format>
```

2.5.2 長文コンテキストの効果的な配置

長い文書（論文全文やデータセット）を AI に与えて分析や質問応答を行う場合、**文書をプロンプトの先頭に配置し、質問や指示を末尾に配置することで**、回答品質が向上することが Anthropic の検証により報告されている。改善幅は最大 30%に達する場合もあるとされる。

この配置が効果的な理由は、LLM がプロンプト末尾の情報に対してより強い注意を向ける傾向があるためである。長い文書の後に質問を配置することで、AI は「いま何を求められているのか」を明確に認識した状態で文書内容を参照できる。

研究活動では、論文の全文を AI に読み込ませて質問する場面が多い。そのような場合、「論文テキスト→質問」の順にプロンプトを構成すると、回答の的確さが向上する。

ヒント: 長い論文やデータを AI に分析させるとき、文書を先に、質問を後に配置すると回答品質が向上する。「資料→指示」の順を心がけよう。

2.5.3 引用ベースの回答で正確性を高める

AI に文書の分析を依頼する際、まずソース文書から関連部分を引用させ、その後に分析を述べさせるという手順を踏ませることで、ハルシネーションを大幅に抑制できる。この手法は**グラウンディング** (grounding) と呼ばれ、AI の回答を原文に「根拠づける」効果がある。

引用を先に行わせることで、AI は回答の根拠を原文に求めるようになり、「もっともらしいが原文にない情報」を生成するリスクが低減する。これは文献レビューや論文査読のように、原文の正確な参照が求められる研究場面で特に有用である。

プロンプト例

【引用ベースの回答を促すプロンプトの例】

以下の論文から、研究手法に関する記述を引用してください。

引用部分を<quotes>タグで囲み、その後に<analysis>タグ内で分析を行ってください。

[論文テキスト]

2.5.4 指示の明確化：「やるな」より「代わりにこうして」

AI に出力を制御する指示を与える際、**否定形の指示**（「～しないでください」）よりも、**肯定形の指示**（「代わりに～してください」）の方が効果的である。否定形の指示は「何をすべきでないか」を伝えるだけで、AI に明確な行動の方向を示さない。一方、肯定形の指示は AI が取るべき行動を具体的に示すため、出力がより安定する。

悪い例と良い例の比較:

- **悪い例:** 「箇条書きにしないでください」
→ AI は箇条書き以外の何をすべきか判断に迷う
- **良い例:** 「段落形式で、各段落に 1 つのポイントを含めて書いてください」
→ AI は具体的な出力形式を把握できる

もう一つの例を挙げる。

- **悪い例:** 「マークダウンを使わないでください」
- **良い例:** 「滑らかな段落構成の文章で書いてください」

2.5.5 文脈と理由を添える

AI への指示では、何をしてほしいかだけでなく、なぜそうしてほしいかを説明することで、AI がその意図を汎化的に理解し、より適切に対応できるようになる。理由が明確であれば、指示で明示的にカバーしていない状況でも、AI は意図に沿った判断を下しやすくなる。

悪い例と良い例の比較:

- **悪い例:** 「省略記号 (...) を使わないでください」
→ AI は指示には従えるが、なぜかわからないため類似の状況で判断を誤る可能性がある
- **良い例:** 「この文章はテキスト読み上げソフトで使用するため、省略記号 (...) は使わないでください。読み上げソフトが省略記号を正しく発音できないためです。」
→ AI は理由を理解し、省略記号以外にも読み上げに問題がありそうな表現を自主的に避けられる

研究活動では、たとえば「数式は $\text{\LaTeX}$ 形式で書いてください。この文章を Overleaf にそのまま貼り付けて使うためです。」のように理由を添えることで、数式以外の部分でも $\text{\LaTeX}$ に適した記法を AI が選択するようになる。

本節の内容は Anthropic のプロンプトエンジニアリングガイド (Anthropic 公式サイト「Prompting best practices」) を参考にしている。各 AI ツールの最新のベストプラクティスは、公式ドキュメントを確認されたい。

2.6 ハルシネーションの理解と対策

2.6.1 ハルシネーションとは何か

ハルシネーション (Hallucination) とは、AI がもっともらしいが事実に反する情報を生成する現象である。第 1 章でも触れたが、プロンプトエンジニアリングの観点から、ここでより詳しく解説する。

ハルシネーションが生じる技術的な背景を簡潔に説明する。LLM は「次のトークン (単語) を予測する」モデルである。この予測は、学習データから得られた統計的パターンに基づいている。つまり、LLM は「事実を記憶している」のではなく、「もっともらしい文を生成している」のである。

この「もっともらしさ」と「正確さ」のギャップがハルシネーションの本質である。

2.6.2 研究活動で特に危険なハルシネーション

研究活動では、以下のタイプのハルシネーションが特に危険である。

(a) **タイプ 1: 架空の論文引用** 最も頻繁に報告されるハルシネーションの一つである。AI は、実在しない論文のタイトル、著者名、雑誌名、出版年、さらには DOI 番号までもっともらしく生成することがある。

プロンプト例

【架空の論文引用の例】

質問: “ドメイン適応における最新の手法について、主要な論文を3本教えてください”

AI の回答 (ハルシネーションを含む):

1. Zhang, W., & Liu, H. (2024). “Progressive Domain Adaptation with Contrastive Learning for Visual Recognition.” IEEE TPAMI, 46(3), 1234–1248.

→ この論文は実在しない可能性がある。著者名、タイトル、巻号ページがもっともらしく生成されているが、DOI で確認すると存在しない。

(b) タイプ2: 誤った数値データ

プロンプト例

【誤った数値データの例】

質問: “ImageNet における最新の画像認識精度の SOTA は?”

AI の回答: “2025 年時点で、ImageNet における

Top-1 精度の SOTA は 94.7% であり、

これは XYZ モデルによって達成されました。”

→ 数値やモデル名が不正確である可能性がある。

特に SOTA は頻繁に更新されるため、AI の学習データの
カットオフ以降の情報は誤っている可能性が高い。

(c) タイプ3: もっともらしい虚偽の説明

プロンプト例

【もっともらしい虚偽の説明の例】

質問: “Batch Normalization が有効な理由を教えてください”

AI の回答: “Batch Normalization は、内部共変量シフト

(Internal Covariate Shift) を軽減することで

学習を安定化させます。”

→ 長年信じられてきた説明だが、

Santurkar et al. (2018)^{?)} の

研究により、

この説明が主要因ではないことが示されている。

AI は「広く引用されている説明」を生成しがちであり、

最新の理解を反映していない可能性がある。

2.6.3 5つの対策法

ハルシネーションに対処するための5つの具体的な対策法を紹介する。

ハルシネーション対策の5つの方法

対策 1: 出典を要求する

プロンプトの中で、明示的に出典の提示を要求する。また、出典の信頼性を評価する基準も指定する。

対策 2: 複数ツールでクロスチェック

重要な情報は、複数の AI ツールや検索エンジンで確認する。

対策 3: 原典にあたって確認

AI が提示した情報の元となる原典（論文、公式ドキュメント、データソース）を直接確認する。これは最も確実な対策であり、研究者の基本動作でもある。

対策 4: 「わからない」と言わせる

AI に「わからない場合はわからないと回答してよい」と明示的に許可することで、ハルシネーションを抑制できることがある。

対策 5: 段階的に検証する

AI の回答を一度に全て信頼するのではなく、小さな単位に分割して段階的に検証する。

プロンプト例

【出典を要求するプロンプト例】

以下の質問に回答する際、各主張に対して出典を明記してください。出典は以下の形式で記載してください:

- 著者名、論文タイトル、掲載誌/会議名、年

また、以下の点に注意してください:

- 確実に実在する論文のみを引用してください
- 記憶が曖昧な場合は「要確認」と明記してください
- 出典を特定できない主張は、「一般的な見解」と注記してください

プロンプト例

【「わからない」を許可するプロンプト例】

以下の質問に回答してください。ただし、以下のルールに従ってください:

- 確信がある情報のみを回答してください
- 不確実な情報には [要確認] タグを付けてください
- わからないことは「この点についてはわかりません」と正直に回答してください
- 推測で回答する場合は、「推測ですが」と明記してください

決して架空の情報を生成しないでください。

キーポイント: ハルシネーション対策の本質は、「AI の出力を無条件に信頼しない」という姿勢である。これは「AI を使わない」ということではなく、「AI の出力を仮説として扱い、検証してから採用する」ということである。この姿勢は、科学的な研究態度そのものとも言える。

2.7 研究活動におけるプロンプト設計の実践

ここまで学んだ技法を、研究活動の具体的な場面に適用する方法を示す。

2.7.1 文献調査用プロンプトの設計

文献調査は、研究の初期段階で最も AI の支援が有効な場面である。

プロンプト例

【文献調査用プロンプト：サーベイの構造化】

[Role] あなたは情報工学分野の研究に精通した図書館司書です。学術文献の体系的な整理と分類を専門としています。

[Objective] 以下の研究テーマに関連する研究の分類体系を提案してください。

[Context] 研究テーマ: 「大規模言語モデルのファインチューニング手法」 / 目的: 修士論文の関連研究セクションの構成を決めるため / 対象期間: 2020 年～2025 年

[Format] 大分類 > 中分類 > 小分類。

各分類に対して: 定義 (50 字程度)、代表的な手法名を 2～3 個、この分類が重要な理由

注意: 実在する手法名のみを使用してください。

ベストプラクティス:

- 最初は広く探索し、徐々に焦点を絞る
- AI が提示した論文は必ず Google Scholar や Semantic Scholar で確認する
- サーベイの構造は AI に提案させ、個々の論文の評価は自分で行う

アンチパターン:

- AI が生成した文献リストをそのまま使う (ハルシネーションのリスク)
- AI の要約だけで論文を読んだ気になる (原典の精読は不可欠)
- 1 回のプロンプトで完璧なサーベイを期待する (段階的に進める)

2.7.2 論文執筆用プロンプトの設計

プロンプト例

【論文執筆用プロンプト：Abstract の推敲】

[Role] あなたは英語ネイティブの学術論文校正者です。

計算機科学分野の論文を年間 100 本以上校正しています。

[Objective] 以下の英語 Abstract を推敲してください。

3 つのバージョンを提示し、それぞれの特徴を説明してください。

[Context] 投稿先: ACL 2027 (自然言語処理のトップ

会議) / 文字数制限: 250 語以内 / 論文の主要な貢献:

新しいファインチューニング手法の提案とその有効性の実証

[Format] 各バージョンについて:

- (1) 修正後の Abstract 全文、
- (2) 主な変更点の一覧 (変更前→変更後)、
- (3) そのバージョンの長所と短所

ベストプラクティス:

- 最初に構成（アウトライン）を固め、その後で各セクションを執筆する
- AI に生成させた文章は必ず自分で読み直し、加筆修正する
- 専門用語の使い方が正しいか確認する

アンチパターン:

- 論文全体を AI に一括で書かせる（品質が低下し、一貫性も失われる）
- AI の文体をそのまま使う（自分の文体で書き直すべき）
- 投稿先のスタイルガイドを無視する

2.7.3 コード生成用プロンプトの設計

プロンプト例

【コード生成用プロンプト：データ分析スクリプト】

[Role] あなたは Python のデータ分析に精通したシニアエンジニアです。

[Objective] 以下の仕様に基づいて、データ分析用の Python スクリプトを作成してください。

[Context] データ: CSV ファイル (列: id, age, gender, score1, score2, score3, group) / 行数: 約 500 行 /

目的: グループ間のスコア比較 /

環境: Python 3.11, pandas, scipy, matplotlib

[仕様]

1. CSV ファイルを読み込む
2. 基本統計量（平均、標準偏差、中央値）をグループ別に算出
3. グループ間の差の検定 (t 検定または Mann-Whitney U 検定をデータの正規性に基づいて自動選択)
4. 結果を可視化（箱ひげ図、棒グラフ）
5. 結果を L^AT_EX 形式の表として出力

ベストプラクティス:

- 仕様を明確に記述し、あいまいさを排除する
- 生成されたコードは必ず自分で動作確認する
- テストケースも AI に生成させ、網羅的にテストする
- コードの各行が何をしているか理解してから使用する

アンチパターン:

- 理解していないコードをそのまま使う（バグの温床）
- エラーが出た際に、エラーメッセージを読まずに AI に丸投げする
- セキュリティに関わるコード（認証、暗号化など）を AI に任せる

2.7.4 プロンプトテンプレートの管理

研究活動で頻繁に使うプロンプトは、テンプレートとして保存・管理することを推奨する。

Listing 2.1 推奨フォルダ構成

```
1 prompts/
2   literature_review/
3     survey_structure.md      # サーベイの構造化
4     paper_summary.md        # 論文の要約
5     comparison_table.md     # 論文の比較表作成
6   writing/
7     abstract_review.md      # 推敲Abstract
8     related_work.md        # 関連研究セクション
9     english_editing.md     # 英語校正
10  coding/
11    data_analysis.md        # データ分析
12    debug.md                # デバッグ支援
13    test_generation.md      # テスト生成
14  presentation/
15    slide_outline.md        # スライド構成
16    qa_preparation.md       # 質疑応答準備
```

テンプレートの各ファイルには、以下の情報を含めるとよい:

1. テンプレートの目的と適用場面
2. プロンプトの全文（変数部分は【ここに入力】で示す）
3. 使用時の注意点
4. 過去の使用実績と改善履歴

ヒント: プロンプトテンプレートを Git で管理すると、改善の履歴を追跡できる。チーム（研究室）で共有すれば、メンバー全員の AI 活用スキルの底上げにもつながる。

演 習 問 題

1. 問題 2.1 (ROCF モデルの適用)

以下のタスクに対して、ROCF モデルの 4 要素 (Role, Objective, Context, Format) を明示的に含むプロンプトを設計せよ。

タスク: あなたの研究テーマ (または興味のあるテーマ) に関連する先行研究を 5 本リストアップし、各論文の手法と結果を比較する表を作成したい。

2. 問題 2.2 (プロンプト技法の使い分け)

以下の 3 つのタスクに対して、それぞれ最適なプロンプト技法 (Zero-shot, Few-shot, Chain-of-Thought) を選択し、その技法を用いたプロンプトを作成せよ。なぜその技法を選んだかも説明せよ。

- (a) 英語の論文タイトルを日本語に翻訳する
- (b) 研究論文の Abstract を「新規性」「有用性」「信頼性」の 3 観点で評価する
- (c) 2 つの研究手法の違いを分析し、どちらが特定の問題設定に適しているか判断する

3. 問題 2.3 (ハルシネーション検出の実践)

任意の対話型 AI を使い、以下を実行せよ。

- (a) 自分の研究分野に関する質問をし、AI に論文を 5 本推薦させる
- (b) 推薦された 5 本の論文すべてについて、Google Scholar または Semantic Scholar で実在を確認する
- (c) 実在しない論文があった場合、その割合を記録する
- (d) ハルシネーションを減らすためにプロンプトを改善し、再度実行する
- (e) 改善前後の結果を比較し、レポートにまとめる

4. 問題 2.4 (段階的精緻化の実践)

任意の対話型 AI を使い、以下の手順で段階的精緻化を実践せよ。

- (a) 自分の研究テーマの Abstract (200~300 字、日本語) のドラフトを AI に生成させる
- (b) 生成されたドラフトの問題点を 3 つ以上指摘し、AI に修正を依頼する
- (c) 修正後のバージョンをさらに改善するため、別の観点から修正を依頼する
- (d) 最終バージョンと最初のドラフトを比較し、どのように改善されたかを分析する

5. 問題 2.5 (メタプロンプティング)

AI に「効果的なプロンプトを設計してもらおう」というメタプロンプティングを実践せよ。

- (a) 以下のタスクを AI に伝える: 「修士論文の中間発表用スライド (10 枚程度) の構成案を作成したい」
- (b) AI に、このタスクに最適なプロンプトを設計させる
- (c) AI が設計したプロンプトを実際に使用し、結果を評価する
- (d) AI が設計したプロンプトの良い点と改善すべき点を分析する

6. 問題 2.6 (総合演習: プロンプトの比較実験)

同じタスクに対して、3 種類のプロンプト (シンプルなプロンプト、ROCF モデルを適用したプロンプト、Few-shot + Chain-of-Thought を組み合わせたプロンプト) を作成し、それぞれの出力を比較せよ。

タスク: 「あなたの研究分野における、今後 5 年間の主要な研究トレンドを予測する」

評価の観点: 回答の具体性、回答の正確性 (検証可能な情報の割合)、回答の有用性 (研究活動に実際に役立つか)、ハルシネーションの有無

引用・参考文献

3

AIを使った文献調査

【この章で学ぶこと】

- AI ツールを活用した効率的な文献調査ワークフローを習得する — 4ステップの体系的な調査手法を実践できるようになる
- 情報の正確性を批判的に検証する方法を身につける — AI が生成する情報の限界を理解し、ファクトチェックを行える
- 研究の新規性を言語化する技法を理解する — 先行研究との差分を明確に説明できるようになる

3.1 研究調査の課題

3.1.1 情報爆発の時代

現代の研究者が直面する最大の課題は、学術情報の爆発的な増加である。プレプリントサーバー arXiv の投稿数は、2010 年には年間約 7 万件であったが、2023 年には年間約 19 万件に達し、わずか 13 年で約 2.7 倍に増加した。PubMed に登録される生物医学系論文も、年間約 100 万件に達している。

この情報爆発は、特に大学院生にとって深刻な問題である。自分の研究テーマに関連する論文をすべて読むことは物理的に不可能であり、重要な先行研究を見落とすリスクが常につきまとう。

3.1.2 キーワードの壁

文献調査で見落としが発生する大きな原因の一つは、「キーワードの壁」である。同じ概念でも分野によって異なる用語が使われることがある。例えば、以下のような例がある。

- 「転移学習 (Transfer Learning)」と「ドメイン適応 (Domain Adaptation)」
- 「特徴量エンジニアリング (Feature Engineering)」と「表現学習 (Representation Learning)」
- 「異常検知 (Anomaly Detection)」と「外れ値検出 (Outlier Detection)」
と「新奇性検出 (Novelty Detection)」

自分が知らない用語で記述されている重要な研究を見逃す可能性は、特に分野横断的な研究において顕著である。従来の検索エンジンはキーワード一致に依存するため、この問題を解決できなかった。

3.1.3 時間的制約

大学院生の日常は、授業、研究、TA 業務、学会準備など多忙を極める。その中で英語論文を精読する時間を確保することは容易ではない。1 本の論文を深く理解するのに数時間かかることも珍しくなく、関連論文を網羅的に調査するには膨大な時間が必要である。

特に英語が母語でない日本語話者にとって、英語論文の読解には追加の負荷がかかる。専門用語の理解、複雑な文構造の把握、文脈に依存した表現の解釈など、言語的な障壁が調査効率を低下させる。

3.1.4 思考の停滞

文献調査を進める中で、多くの大学院生が経験するのが「思考の停滞」である。先行研究を読めば読むほど、「すでに研究されている」「自分の研究に新規性がない」と感じてしまうことがある。

この問題の根本は、自分の研究の新規性をうまく言語化できないことにある。研究の新規性は、先行研究との差分を明確にすることで初めて可視化される。

しかし、その差分を的確に表現する技術は、経験を積まなければ身につかないものであった。AIはこの言語化のプロセスを支援する。

3.1.5 従来の調査プロセスと AI 活用の比較

AIを活用した文献調査は、従来の調査プロセスを根本的に変えるものではなく、各段階を加速・拡張するものである。表表 3.1 に、従来の調査プロセスと AI を活用した調査プロセスの比較を示す。

表 3.1 従来の文献調査と AI を活用した文献調査の比較

項目	従来の調査プロセス	AI を活用した調査プロセス
キーワード発見 全体像の把握	自分の知識と経験に依存 論文を 1 本ずつ読む	AI との対話でアイデアを拡張 AI で分野の全体像を短時間で 把握
論文の深掘り	原文を精読	AI で構造的に要約し、重要部 分を精読
思考の整理 プロセス	自分の頭の中で整理 直線的（検索→読む→ま とめ）	AI に壁打ちして思考を言語化 反復的・探索的サイクル

注意：機密情報の取り扱い

AI ツールに未発表の研究データ、特許出願前のアイデア、個人情報などの機密情報を入力してはならない。入力されたデータが AI の学習に使用される可能性があるためである。特に、共同研究先との秘密保持契約（NDA）が存在する場合は、契約内容を確認した上で AI ツールの利用範囲を判断すること。

3.2 文献調査 AI ツールの使い分け

3.2.1 対話型 AI — プレスト・壁打ちの相棒

主なツール: Gemini Pro、ChatGPT (GPT-4o)、Claude

対話型 AI は、研究のアイデア出しや方向性の検討に最適である。人間の指導教員や研究仲間と壁打ちするように、AI と対話することで思考を整理できる。

長所:

- 自然言語で曖昧な質問ができる
- 対話を通じて段階的に思考を深められる
- 異なる視点からの提案を得られる
- 複雑な概念の説明を求められる

短所:

- 出典が明示されないことが多い
- 存在しない論文を生成する場合がある（ハルシネーション）
- 学習データの時期によって最新情報が反映されない
- 回答の正確性を自分で検証する必要がある

3.2.2 検索特化型 AI — 出典付き回答でサーベイの初動

主なツール: Perplexity

検索特化型 AI は、ウェブ検索と AI の回答生成を組み合わせたツールである。回答に出典リンクが付与されるため、情報の検証が容易である。

長所:

- 回答に出典 URL が付く
- リアルタイムのウェブ情報を反映できる
- サーベイの初期段階で全体像を掴むのに適している
- 英語と日本語の両方で検索可能

短所:

- 深い分析や批判的検討は苦手
- 出典の質にばらつきがある（学術論文以外の情報も混在）
- 対話の文脈保持能力は対話型 AI に劣る

3.2.3 論文分析 AI — 深掘りのための専門ツール

主なツール: NotebookLM、Elicit、Connected Papers、Semantic Scholar
論文分析に特化した AI ツールは、学術文献の深い理解を支援する。

表 3.2 論文分析 AI ツールの比較

ツール名	特徴	最適な用途
NotebookLM	アップロードした文書をもとに質問応答	特定論文の深い理解
Elicit	学術論文の検索・要約に特化	体系的な文献レビュー
Connected Papers	引用関係の可視化	研究分野のマッピング
Semantic Scholar	AI 支援の論文検索エンジン	関連論文の網羅的発見

3.2.4 目的別フローチャート

文献調査の目的に応じて、最適なツールの組み合わせは異なる。以下に目的別の推奨フローを示す。

目的別の推奨フロー

パターン A: 新しい研究テーマの探索

1. 対話型 AI (Claude/ChatGPT) でブレインストーミング
2. Perplexity で関連分野の概要を把握
3. Connected Papers で研究分野のマッピング

パターン B: 既存テーマの先行研究調査

1. Perplexity で主要な先行研究を特定
2. Semantic Scholar または Elicit で網羅的な検索
3. NotebookLM で重要論文を深掘り
4. 対話型 AI で新規性の言語化

パターン C: 特定論文の理解

1. NotebookLM に論文をアップロードして質問
2. 対話型 AI に要約・解説を依頼
3. Connected Papers で関連研究を確認

3.3 4 ステップの文献調査ワークフロー

3.3.1 Step 1: キーワード拡張 — AI で関連キーワードを発見する

文献調査の第一歩は、検索に使うキーワードの拡張である。自分が知っているキーワードだけでは、関連する研究を見落とす可能性がある。AI を使って関連キーワードを網羅的に発見しよう。

プロンプト例

【基本プロンプト例：キーワード拡張】

私は「[研究テーマ]」について文献調査を行いたいと考えています。この研究テーマに関連する検索キーワードを、以下の観点から網羅的にリストアップしてください。

1. 同義語・類義語（英語・日本語両方）
2. 上位概念・下位概念
3. 関連する手法名・アルゴリズム名
4. 関連する応用分野
5. この分野で頻出するキーワード

それぞれのキーワードについて、簡単な説明も付けてください。

プロンプト例

【具体例 — 「グラフニューラルネットワークによる薬物相互作用予測」の場合】

私は「グラフニューラルネットワークによる薬物相互作用予測」について文献調査を行いたいと考えています。

この研究テーマに関連する検索キーワードを、以下の観点から網羅的にリストアップしてください。

1. 同義語・類義語（英語・日本語両方）
2. 上位概念・下位概念
3. 関連する手法名・アルゴリズム名
4. 関連する応用分野
5. この分野で頻出するキーワード

それぞれのキーワードについて、簡単な説明も付けてください。

出力の評価と活用法:

AI が生成したキーワードリストは、そのまま使うのではなく、以下の手順で

評価・活用する。

1. **知っているキーワードの確認** — リストに自分が既知のキーワードが含まれているか確認する。含まれていない場合、プロンプトの改善が必要
2. **未知のキーワードの学習** — 知らなかったキーワードについて、追加で調べる
3. **キーワードの優先順位付け** — 自分の研究テーマとの関連度が高い順に並べ替える
4. **検索クエリの構築** — 優先度の高いキーワードを組み合わせ、Google Scholar や Semantic Scholar の検索クエリを作成する

ヒント: キーワード拡張は一度で完了するものではない。調査を進める中で新たなキーワードが見つかったら、このステップに戻って検索範囲を広げよう。反復的な調査が網羅性を高める。

3.3.2 Step 2: 全体像を掴むサーベイ — Perplexity 等で俯瞰する

キーワードが揃ったら、次は研究分野の全体像を掴むサーベイを行う。ここでは Perplexity のような検索特化型 AI が効果的である。

(a) **英語プロンプトの重要性** 学術情報の大部分は英語で公開されている。そのため、サーベイのプロンプトは英語で入力することを強く推奨する。日本語で入力すると、日本語のウェブページが優先的に検索され、最新の学術論文が見落とされる可能性がある。

プロンプト例

【英語でのサーベイプロンプト例】

What are the current state-of-the-art approaches for drug-drug interaction prediction using graph neural networks?

Please provide:

1. A brief overview of the field (2–3 paragraphs)
2. Key methods and their characteristics (with citations)
3. Major datasets used in this field
4. Open challenges and future directions
5. Key research groups and their contributions

出典リンクの確認方法:

Perplexity が提示する出典は、必ず以下の手順で確認する。

1. 出典リンクをクリックして、実際のページが存在するか確認する
2. 論文の場合、DOI または arXiv ID が正しいか確認する
3. Google Scholar で論文タイトルを検索し、被引用数や掲載ジャーナルを確認する
4. 情報が一次情報源に基づいているか確認する（ブログ記事やニュース記事ではなく、論文原文を参照する）

ヒント: Perplexity の「Academic」モードを使うと、学術論文を優先的に検索してくれる。サーベイの初期段階では、このモードを活用しよう。

3.3.3 Step 3: 注目論文の深掘り — アブストラクトから構造的要約へ

全体像を掴んだら、特に重要と思われる論文を深く読み込む。AI を活用して、効率的に論文の要点を抽出しよう。

プロンプト例

【4 点要約テンプレート】

以下の論文のアブストラクト（および可能であれば本文）を読み、次の 4 点について日本語で構造的に要約してください。

【論文タイトル】：[タイトルを入力]

【アブストラクト】：[アブストラクトを貼り付け]

1. **背景 (Background)**：この研究が取り組む問題は何か？なぜ重要か？
2. **手法 (Method)**：どのようなアプローチ・手法を提案しているか？技術的な新規性は何か？
3. **結果 (Results)**：主要な実験結果は何か？既存手法と比較してどの程度の改善が得られたか？
4. **限界 (Limitations)**：この研究の限界や未解決の課題は何か？今後の研究の方向性として何が示唆されているか？

また、この論文の新規性を一文で表現してください。

(a) **複数論文の比較要約** 重要な論文が複数見つかった場合、AI を使って比較表を作成できる。

プロンプト例

【複数論文の比較要約プロンプト】

以下の3つの論文について、比較表を作成してください。

論文 1: [タイトル] — [アブストラクト]

論文 2: [タイトル] — [アブストラクト]

論文 3: [タイトル] — [アブストラクト]

比較の軸:

- 解決しようとしている問題
- 提案手法の特徴
- 使用データセット
- 主要な結果（定量的な指標を含む）
- 限界と課題
- 自分の研究テーマ「[テーマ]」との関連性

ヒント: NotebookLM を使えば、論文 PDF を直接アップロードして質問できる。アブストラクトだけでなく、実験設定の詳細や図表の解釈についても質問できるため、深い理解が得られる。

3.3.4 Step 4: 研究の新規性を言語化する

文献調査の最終目標は、自分の研究の新規性を明確に言語化することである。このステップが最も重要であり、同時に最も難しいステップである。

プロンプト例

【新規性言語化のためのプロンプト】

私の研究テーマは「[テーマ]」です。以下に、関連する先行研究とその限界をまとめました。

先行研究 1: [タイトル] / 手法: [手法の概要] / 限界: [限界]

先行研究 2: [タイトル] / 手法: [手法の概要] / 限界: [限界]

先行研究 3: [タイトル] / 手法: [手法の概要] / 限界: [限界]

私が提案する手法は「[自分の手法の概要]」です。

以下の点について、整理を手伝ってください。

1. 先行研究に共通する未解決の課題は何か？
2. 私の提案手法は、それらの課題をどのように解決するか？
3. 私の研究の新規性を、学术论文の Introduction に書くような形式で 3-4 文にまとめてください。
4. この新規性の主張に対して、査読者が指摘しそうな弱点は何か？

(a) 差分マトリクスの作成

プロンプト例

【差分マトリクス作成プロンプト】

以下の先行研究と私の研究について、
差分マトリクスを
作成してください。
比較軸として以下を含めてください:

- 対象とする問題設定
- 使用するデータの種類
- モデルアーキテクチャ
- 学習方法
- 評価指標
- 計算コスト
- 実用性

各セルには、具体的な違いを簡潔に記述してください。
自分の研究の優位点には ★ をつけてください。

ヒント: 新規性の言語化は、AI の出力をそのまま使うのではなく、あくまで「たたき台」として活用すること。最終的な新規性の主張は、指導教員や研究仲間と議論して磨き上げることが重要である。

3.4 情報の批判的検証

3.4.1 AI が出力した論文情報のファクトチェック手法

AI は非常に有用なツールであるが、その出力を無批判に信頼することは危険である。特に文献調査では、以下のような問題が発生する可能性がある。

ファクトチェックのチェックリスト

- ☐ 論文のタイトルと著者名が正確か（Google Scholar で検索）
- ☐ 掲載ジャーナル/学会名が正しいか
- ☐ 発表年が正確か
- ☐ 引用されている数値や結果が論文原文と一致するか
- ☐ 論文の主張が AI の要約と整合しているか

具体的なファクトチェック手順:

1. **Google Scholar での検索:** AI が挙げた論文のタイトルをそのまま Google Scholar に入力し、実在する論文かどうかを確認する
2. **DOI の確認:** DOI が提示されている場合、doi.org で解決できるか確認する
3. **著者のホームページ確認:** 主要著者の研究室ページや個人ページで、該当論文が掲載されているか確認する
4. **数値の照合:** 重要な実験結果の数値は、必ず論文原文で確認する

3.4.2 存在しない論文への対処（ハルシネーション問題）

AI が存在しない論文を「作り上げる」ことは、ハルシネーションの典型例である。もっともらしいタイトル、著者名、ジャーナル名が生成されるが、実際には存在しない論文であるケースが少なくない。

ハルシネーションを疑うべきサイン:

- タイトルが非常に一般的で、どの分野にも当てはまりそうな表現
- 有名な研究者の名前が著者に含まれているが、その研究者の専門分野と一致しない
- 掲載ジャーナルが非常に有名だが、分野が異なる
- 発表年が最新すぎる、または古すぎる
- DOI や URL が提示されない

プロンプト例

【ハルシネーション対処のプロンプト例】

先ほど挙げていただいた論文について確認したいのですが、
以下の論文は Google Scholar で見つかりませんでした。

「[論文タイトル]」

この論文は実在しますか？もし存在しない場合、
同様の内容を扱う実在する論文を教えてください。その際、
Google Scholar で確認できるように、正確なタイトルと
著者名を記載してください。

ヒント: Perplexity は出典リンク付きで回答を生成するため、ハルシネーションのリスクが比較的低い。論文の実在性が重要な場合は、Perplexity で確認するか、直接 Google Scholar や Semantic Scholar で検索しよう。

3.4.3 バイアスの認識

AI の回答には、学習データに起因するバイアスが含まれる可能性がある。文献調査で注意すべきバイアスには以下がある。

(a) **英語圏バイアス** AI は英語のデータで主に学習されているため、英語圏の研究が過度に強調される傾向がある。日本語や他の言語で発表された重要な研究が見落とされる可能性がある。

(b) **引用数バイアス** 被引用数が多い「有名な」論文が優先的に紹介される傾向があり、最新の研究や小規模な研究グループの重要な成果が見落とされる可能性がある。

(c) **分野バイアス** AI の学習データに含まれる論文の分野分布に偏りがあるため、特定の分野（例：機械学習、自然言語処理）の研究が過度に強調される可能性がある。

(d) **出版バイアス** 肯定的な結果を報告する論文が優先される傾向があ

り、否定的な結果や再現性の問題を報告する論文が見落とされる可能性がある。

これらのバイアスを認識した上で、AI の出力を批判的に評価することが重要である。

3.5 文献管理とまとめ

3.5.1 調査結果の整理法

文献調査で得られた情報は、体系的に整理することで初めて研究に活用できる。AI を使って、調査結果を表形式で整理しよう。

プロンプト例

【文献比較表の作成プロンプト】

以下の論文情報をもとに、比較表を作成してください。

[論文情報をリストアップ]

表の列には以下を含めてください:

- 論文（著者, 年）
- 研究目的
- 提案手法
- データセット
- 主要結果
- 限界・課題
- 自分の研究との関連

整理のポイントとして、以下の項目を意識すること。

- **時系列での整理:** 研究の発展の流れを把握するため、発表年順に並べる
- **手法別の整理:** 異なるアプローチを分類し、各手法の特徴を比較する
- **問題設定別の整理:** 同じ問題に取り組む研究をグループ化する

3.5.2 AIで文献レビューの草稿を作成する

文献調査の結果を論文の「関連研究」セクションにまとめる作業も、AIの支援を受けることができる。

プロンプト例

【文献レビュー草稿作成プロンプト】

以下の先行研究の情報をもとに、論文の“Related Work”

セクションの日本語草稿を作成してください。

[先行研究の要約リスト]

以下の要件を満たしてください:

1. 研究を意味のあるカテゴリに分類して整理する
2. 各カテゴリ内で、研究の発展を時系列で説明する
3. 各研究の貢献と限界を簡潔に述べる
4. 最後に、これらの先行研究と本研究の関係を明確にする
5. 学術論文にふさわしい客観的な文体を使用する

ヒント: AIが生成した文献レビューの草稿は、あくまで出発点である。必ず自分で読み直し、以下を確認すること。(1) 論文の内容が正確に要約されているか、(2) 研究間の関係が正しく記述されているか、(3) 自分の研究の位置づけが明確か。

3.5.3 文献管理ツールとの連携

AIで発見した論文は、文献管理ツールに登録して一元管理しよう。主要な文献管理ツールとその特徴は表表 3.3 の通りである。

表 3.3 主要な文献管理ツールの比較

ツール名	特徴	料金
Zotero	オープンソース、ブラウザ拡張が便利	無料 (300MB まで)
Mendeley	PDF 管理が優秀、Elsevier との連携	無料 (2GB まで)
Paperpile	Google Docs との連携が強力	有料 (学生割引あり)
EndNote	多くの大学で機関ライセンスあり	有料

AI と文献管理ツールの連携ワークフロー

1. AI で文献調査を行い、重要な論文を特定する
2. Google Scholar や Semantic Scholar で論文を確認し、文献管理ツールに登録する
3. 文献管理ツールでタグ付け・メモの追加を行う
4. 論文執筆時に、文献管理ツールから引用を挿入する

ヒント: Zotero のブラウザ Extension を使えば、Google Scholar や arXiv のページからワンクリックで論文に登録できる。AI で見つけた論文を効率的に管理するためにも、文献管理ツールの導入を強く推奨する。

演 習 問 題

1. 演習 1: キーワード拡張の実践

自分の研究テーマ（または興味のある研究トピック）について、AI を使ってキーワード拡張を行いなさい。以下の手順に従うこと。

- (a) 対話型 AI に研究テーマを説明し、関連キーワードを少なくとも 20 個生成させる
- (b) 生成されたキーワードを「知っていたもの」と「知らなかったもの」に分類する
- (c) 知らなかったキーワードのうち、最も重要だと思うもの 3 つについて、Google Scholar で検索して関連論文を見つける

- (d) この演習を通じて発見した新しい視点や気づきをまとめる

2. 演習 2: ファクトチェックの実践

以下の手順で、AI のハルシネーション検出を体験しなさい。

- (a) 対話型 AI に「[自分の研究分野]における重要な論文を 10 本挙げてください」と質問する
- (b) 挙げられた 10 本の論文すべてについて、Google Scholar で実在を確認する
- (c) 存在しない論文があった場合、その数と割合を記録する
- (d) ハルシネーションが発生した論文について、どのような特徴があったかを分析する
- (e) 結果をもとに、AI の出力を検証する際の注意点をまとめる

3. 演習 3: 4 点要約の実践

自分の研究テーマに関連する英語論文を 1 本選び、以下を行いなさい。

- (a) まず自分でアブストラクトを読み、4 点（背景・手法・結果・限界）を日本語でまとめる
- (b) 次に、本章のテンプレートを使って AI に 4 点要約を依頼する
- (c) 自分の要約と AI の要約を比較し、(a) AI が正確に要約できた点、(b) AI が見落とした点、(c) AI が誤って解釈した点、をそれぞれ挙げる
- (d) この比較を通じて、AI 要約を活用する際のベストプラクティスをまとめる

4. 演習 4: 新規性の言語化

以下の手順で、自分の研究の新規性を言語化しなさい。

- (a) 自分の研究テーマに最も近い先行研究を 3 本特定する
- (b) 各先行研究について、手法と限界を整理する
- (c) 本章のプロンプトテンプレートを使って、AI に新規性の言語化を依頼する
- (d) AI の出力を批判的に検討し、修正・改善する

- (e) 最終的な新規性の記述を、指導教員やゼミのメンバーに共有してフィードバックを得る

5. 演習 5: 文献レビューの草稿作成

自分の研究テーマについて、以下の手順で文献レビューの草稿を作成しなさい。

- (a) これまでの演習で収集した文献情報をもとに、AI に文献レビューの草稿を作成させる
- (b) 草稿を読み、以下の観点で評価する: (a) 分類は適切か、(b) 時系列の流れは正確か、(c) 各研究の記述は正確か
- (c) 問題があった箇所を修正し、自分の研究の位置づけを加筆する
- (d) 完成した草稿を文献管理ツール (Zotero など) の引用情報と照合し、参考文献リストを作成する

本章のまとめ: AI は文献調査を効率化する強力なツールであるが、万能ではない。AI の出力を鵜呑みにせず、常にファクトチェックを行い、批判的思考を維持することが必要である。文献調査の目的は、先行研究を知ることだけでなく、自分の研究の新規性と価値を明確にすることにある。AI を「思考の補助輪」として活用し、自分自身の研究力を高めてほしい。

引用・参考文献

4

英語論文の執筆と推敲

【この章で学ぶこと】

- 英語論文の基本構造（IMRaD）を理解する — 各セクションの役割と書くべき内容を把握し、構造的な論文を書ける
- AIを活用した英訳・執筆・推敲ワークフローを習得する — 日本語原稿からAIの支援を受けて高品質な英語論文を仕上げるプロセスを実践できる
- 学術的な英語表現を適切に使い分けられる — 分野に応じた定型表現を活用し、査読者に好印象を与える論文を執筆できる

4.1 なぜ英語論文が必要か

4.1.1 国際会議・ジャーナルの重要性

研究の価値は、その成果が広く共有され、他の研究者に活用されて初めて発揮される。日本語で発表された研究は、国内の研究者には届くが、世界中の研究者にはほとんど届かない。トップカンファレンス（NeurIPS、ICML、ACL、CVPR、AAAI等）やトップジャーナル（Nature、Science、IEEE Transactions等）はすべて英語で発表されており、国際的な研究コミュニティの共通言語は英語である。

大学院生にとって英語論文の執筆は、以下の理由で極めて重要である。

- **研究成果の国際的な可視性:** 英語論文は世界中の研究者が読むことができ、被引用数の向上につながる
- **キャリア形成:** 国際的な実績は、アカデミアでも産業界でも高く評価される
- **研究コミュニティへの参加:** 英語論文を発表することで、国際的な研究ネットワークに参加できる
- **フィードバックの質:** 国際会議の査読者は世界トップレベルの研究者であり、質の高いフィードバックが得られる

4.1.2 研究成果の国際発信と評価

英語論文の執筆は、単に日本語を英語に翻訳する作業ではない。国際的な読者に向けて、研究の背景、意義、手法、結果を論理的かつ説得力のある形で伝える作業である。

しかし、英語を母語としない研究者にとって、学術英語の壁は非常に高い。従来は英文校正サービスやネイティブの共同研究者に頼ることが一般的であったが、AIの登場により、英語論文執筆のハードルは大きく下がった。AIは万能ではないが、適切に活用すれば、執筆の効率と品質を大幅に向上させられる。

4.2 英語論文の基本構造

4.2.1 IMRaD 構造

国際的な学術論文の標準的な構造はIMRaDと呼ばれ、以下の4つのセクションから構成される。

これに加えて、以下のセクションが一般的に含まれる。

- **Abstract (要旨):** 論文全体の要約 (150–300 語程度)
- **Related Work (関連研究):** 先行研究の整理と本研究の位置づけ
- **Conclusion (結論):** 研究の要約と将来展望

表 4.1 IMRaD 構造の概要

セクション	英語名	主な内容	時制
はじめに	Introduction	研究の背景、目的、貢献	現在形・過去形
手法	Methods	提案手法の詳細	過去形
結果	Results	実験結果の提示	過去形
考察	Discussion	結果の解釈、限界、将来展望	現在形・過去形

- **References (参考文献)** : 引用した文献のリスト

4.2.2 各セクションの役割と書くべき内容

Introduction (はじめに) の構造

Introduction は「逆三角形」の構造で書く。まず広い背景から始めて、徐々に焦点を絞り、最後に自分の研究の貢献を明確に述べる。

1. 研究分野の一般的な背景と重要性 (1-2 段落)
2. 既存研究の課題や未解決の問題 (1-2 段落)
3. 本研究のアプローチと貢献 (1 段落)
4. 論文の構成の概要 (1 段落、省略可)

Methods (手法) :

提案手法を、読者が再現できるレベルで詳細に記述する。

1. 問題設定の形式的な定義
2. 提案手法の全体像
3. 各構成要素の詳細
4. 実装の詳細 (ハイパーパラメータ、使用ライブラリ等)

Results (結果) :

実験結果を客観的に提示する。解釈は Discussion で行う。

1. 実験設定 (データセット、評価指標、ベースライン)
2. 主要な結果 (表やグラフ)
3. 結果の記述 (数値の比較、傾向の指摘)

Discussion (考察) :

結果の意味を解釈し、研究の意義と限界を議論する。

1. 主要な発見の解釈
2. 先行研究との比較
3. 研究の限界
4. 将来の研究方向

4.2.3 パラグラフの構成法

英語の学術論文では、パラグラフ（段落）が意味の基本単位である。各パラグラフは以下の構造に従う。

1. **トピックセンテンス:** パラグラフの主張を 1 文で述べる（段落の冒頭に置く）
2. **サポート:** トピックセンテンスを支える根拠、例、データを述べる（2-5 文）
3. **結論/遷移:** パラグラフの内容をまとめ、次のパラグラフへつなげる（1 文）

プロンプト例

以下の日本語の段落を、トピックセンテンス・サポート・結論の構造に整理してください。必要に応じて、論理の飛躍を指摘してください。

[日本語の段落を貼り付け]

Tip: 1 パラグラフ 1 メッセージ

英語論文では「1 パラグラフ 1 メッセージ」の原則を守ることが重要である。複数のメッセージが混在するパラグラフは、読者の理解を妨げる。日本語の文章は 1 つの段落に複数の話題を含むことがあるが、英語論文ではそれぞれを独立したパラグラフに分けるのが原則である。

4.3 AIを使った英訳ワークフロー

Step 1: 日本語原稿の準備と構造化

AIに英訳を依頼する前に、日本語原稿の品質を高めることが重要である。曖昧な日本語からは曖昧な英語しか生まれない。

日本語原稿の準備チェックリスト:

- 各段落にトピックセンテンスがあるか
- 論理の飛躍がないか
- 主語と述語が明確か（日本語は主語省略が多いので注意）
- 専門用語が一貫して使われているか
- 図表への参照が正しいか

構造化のためのプロンプト:

プロンプト例

以下の日本語原稿を、英語論文に翻訳する前の準備として、構造的に整理してください。

具体的には: 1. 各段落のトピックセンテンスを明示してください 2. 論理の飛躍や不明確な箇所を指摘してください 3. 主語が省略されている文を特定し、主語を補ってください 4. 段落の分割・統合が必要な箇所を提案してください

[日本語原稿を貼り付け]

Step 2: セクション単位で AI に英訳を依頼

日本語原稿が整理できたら、セクション単位で AI に英訳を依頼する。論文全体を一度に訳すのではなく、セクション単位で行うことで、品質の管理がしやすくなる。

効果的な英訳プロンプト:

プロンプト例

以下の日本語原稿を、学術英語に翻訳してください。

【条件】 - 分野: [例: コンピュータサイエンス / 機械学習] - 対象読者: 国際会議の査読者（専門家） - 文体: 学術論文（フォーマル） - セクション: [例: Introduction / Methods / Results] - 時制: [例: 過去形を基本とする（Methods の場合）]

【注意事項】 - 以下の専門用語は指定の英語表現を使用してください: [用語 1]: [英語表現 1] [用語 2]: [英語表現 2] - 受動態を過度に使用しないでください - 1 文が長くなりすぎないようにしてください（目安: 25 語以内）

【日本語原稿】 [原稿を貼り付け]

分野指定の重要性

学術英語の表現は分野によって大きく異なる。例えば、同じ「提案する」でも分野によって適切な表現が異なる。

- コンピュータサイエンス: “We propose...” / “We present...”
- 医学: “We hypothesize...” / “We examined...”
- 物理学: “We derive...” / “We demonstrate...”

Tip: 参考論文の活用

翻訳の際は、自分の分野のトップ論文を 2-3 本手元に置いておき、表現や文体の参考にするとよい。AI が生成した英語が、その分野の慣習に合っているかを確認するための基準になる。

Step 3: 学術的表現への洗練

初回の翻訳結果をさらに学術的に洗練する。

プロンプト例

以下の英語論文の草稿を、より学術的で洗練された表現に改善してください。

【改善の方向性】1. 曖昧な表現をより具体的・精確にする 2. 冗長な表現を簡潔にする 3. 接続語を適切に使い、論理の流れを明確にする 4. 学術論文で一般的な表現に置き換える（例: "very good" → "substantially improved"） 5. ヘッジング表現を適切に使用する（例: "This proves..." → "These results suggest..."）

改善箇所には【変更理由】を付けてください。

[英語草稿を貼り付け]

Step 4: 文法・スタイルチェック

最終段階として、文法とスタイルの統一性を確認する。

プロンプト例

以下の英語論文について、文法・スタイルの最終チェックを行ってください。

【チェック項目】1. 文法エラー（主語-動詞の一致、冠詞、前置詞など） 2. 時制の一貫性（セクション内で統一されているか） 3. 用語の一貫性（同じ概念に同じ用語が使われているか） 4. 数値の表記（有効数字、単位の表記） 5. 参照の形式（Figure 1、Table 2 などの表記統一） 6. スペルの統一（British/American English の混在がないか）

エラーが見つかった場合、修正案と理由を示してください。

[英語論文を貼り付け]

4.4 推敲テクニック

4.4.1 段階的推敲法

英語論文の推敲は、一度に全てをチェックしようとする、多くの問題を見落とす。以下の3段階に分けて、段階的に推敲を行うとよい。

第1段階: 文法チェック

プロンプト例

以下の英語論文について、文法エラーのみに焦点を当ててチェックしてください。表現の改善提案は不要です。純粋な文法エラー（主語-動詞の一致、冠詞、前置詞、時制の誤りなど）のみを指摘し、修正案を示してください。

[英語論文を貼り付け]

第2段階: 表現チェック

プロンプト例

以下の英語論文について、表現の改善に焦点を当ててチェックしてください。文法は正しいが、以下の観点で改善できる箇所を指摘してください。

1. より学術的な表現への置き換え 2. 冗長な表現の簡潔化 3. 曖昧な表現の明確化 4. 不自然な表現（日本語直訳調）の修正

[英語論文を貼り付け]

第3段階: 論理チェック

プロンプト例

以下の英語論文について、論理構造に焦点を当ててチェックしてください。

1. 段落間の論理的つながりは適切か
2. 主張に対する根拠は十分か
3. 飛躍や矛盾はないか
4. 読者が混乱する可能性のある箇所はどこか

[英語論文を貼り付け]

4.4.2 比較推敲法

複数の AI ツールで翻訳を行い、最良の表現を選ぶ方法である。

プロンプト例

以下の日本語を 3 つの異なるスタイルで英訳してください。

スタイル A: 簡潔で直接的な表現スタイル B: 詳細で丁寧な表現スタイル

C: [分野名] の国際会議で一般的な表現

それぞれの訳について、長所と短所を説明してください。

[日本語原稿を貼り付け]

さらに、Claude、ChatGPT、Gemini など異なる AI で同じ文章を翻訳し、表現を比較することも効果的である。各 AI には得意な表現スタイルがあり、それぞれの良い部分を組み合わせることで、より高品質な英語論文を作成できる。

4.4.3 逆翻訳チェック

英訳が原文の意味を正確に反映しているかを確認する強力な方法が、逆翻訳チェックである。

プロンプト例

以下の英語を日本語に翻訳してください。できるだけ直訳に近い形で翻訳し、英語の構造がわかるようにしてください。

[英訳された論文の一部を貼り付け]

逆翻訳された日本語を元の日本語原稿と比較し、意味のズレがないかを確認

する。ズレが見つかった場合、英訳を修正する。

逆翻訳で発見できる問題:

- 原文にあった情報が英訳で欠落している
- 英訳が原文とは異なる意味に解釈される
- 曖昧な英語表現が原文の意図と異なる解釈を生んでいる
- 専門用語の翻訳が不正確である

4.4.4 ネイティブチェック風レビュー

AIに査読者やネイティブスピーカーの役割を担わせることで、論文の品質を事前に確認できる。

プロンプト例

あなたは英語を母語とする [分野名] の研究者です。以下の英語論文を読み、非ネイティブ話者が書いたと思われる不自然な表現を指摘してください。指摘の際は以下の形式をお願いします: - 原文: [不自然な表現] - 修正案: [自然な表現] - 理由: [なぜ不自然か、どう改善されるか]

特に以下の点に注意してください: 1. 冠詞の使い方 2. 前置詞の選択 3. コロケーション (単語の自然な組み合わせ) 4. 文のリズムと読みやすさ

[英語論文を貼り付け]

Tip: 推敲のタイミング

推敲は時間をおいて行うのが効果的である。書いた直後ではなく、翌日に見直すと、問題点が見つかりやすくなる。AIを使った推敲も同様に、初回の英訳から時間をおいてから行うとよい。新鮮な目で見ることで、より多くの改善点を発見できる。

4.5 Abstract 作成の技法

4.5.1 Abstract の構造

Abstract は論文全体の縮図であり、読者が論文を読むかどうかを決める最も重要な部分である。以下の 5 要素を含めるのが標準的である。

1. 背景 (Background) : 研究分野の文脈と解決すべき問題 (1-2 文)
2. 目的 (Objective) : 本研究で何を達成するか (1 文)
3. 手法 (Method) : 提案するアプローチの概要 (2-3 文)
4. 結果 (Results) : 主要な実験結果の数値 (1-2 文)
5. 意義 (Significance) : 研究の意義と影響 (1 文)

4.5.2 150-200 語にまとめるプロンプト設計

プロンプト例

以下の情報をもとに、学術論文の Abstract を英語で作成してください。

【研究情報】 - 研究分野: [分野] - 解決する問題: [問題の記述] - 提案手法: [手法の概要] - 主要な結果: [数値を含む結果] - 研究の意義: [意義の記述]

【条件】 - 語数: 150-200 語 - 構造: 背景→目的→手法→結果→意義の順 - 時制: 背景は現在形、手法と結果は過去形、意義は現在形 - 具体的な数値を必ず含める - 略語を初出で定義する

4.5.3 良い Abstract と悪い Abstract の比較

悪い例 (問題点を含む) :

In this paper, we propose a new method for text classification. Our method uses deep learning. We conducted experiments and obtained good results. Our method outperformed baseline methods. This work contributes to the field of natural language

processing.

問題点:

- 具体性がない (“new method”、“good results”)
- 数値が一切含まれていない
- 手法の特徴が不明
- 読者にとって有益な情報がほとんどない

良い例:

Text classification in low-resource languages remains challenging due to the scarcity of labeled training data. We propose CrossLingual-BERT, a cross-lingual transfer learning framework that leverages pre-trained multilingual representations to classify texts in languages with limited annotated corpora. Our approach fine-tunes a multilingual BERT model on high-resource source languages and adapts it to target languages using only 100 labeled examples per class. Experiments on five typologically diverse languages demonstrate that CrossLingual-BERT achieves an average F1 score of 87.3%, outperforming monolingual baselines by 12.5 percentage points and the previous state-of-the-art cross-lingual method by 4.2 percentage points. These results suggest that effective cross-lingual transfer is possible even with minimal target-language supervision, opening new possibilities for NLP applications in under-resourced languages.

優れている点:

- 背景が具体的で問題が明確
- 手法の核心的な特徴が述べられている
- 具体的な数値が含まれている

- 研究の意義が最後に述べられている

Tip: Abstract の執筆タイミング

Abstract は論文の最後に書くのが効率的である。全セクションを書き終えてから Abstract を作成すれば、研究の全体像を正確に反映できる。AI に Abstract の草稿を作成させる際も、論文の完成後に全セクションの内容を入力として与えるとよい。

4.6 よくあるミスと対策

本節では、日本語話者が英語論文を書く際に陥りやすいミスとその対策を、授業で扱った具体例とともに体系的に解説する。まず各トピックの概念的な基礎を理解し、その後に早見表を参照するという流れで学習を進めてほしい。

松野先生の教え： 結局、日本語（母国語）の能力が重要である。必要最小限の日本語にしてから英語にする。これが技術英語の基本である。

4.6.1 すっきり最小主義：冗長な日本語を削ぎ落とす

英語に翻訳する前に、まず日本語を簡潔にすることが最も重要なステップである。冗長な日本語からは冗長な英語しか生まれない。

（１）**重複表現を削る** 日本語は同じ意味の言葉を重ねる傾向がある。英訳前にこれを削ぎ落とす。

例：重複表現の除去

原文：「リチャージャブルバッテリーの安定した充電のためには、電気が流れる端子部が清浄であることが重要です」

問題：「リチャージャブル（再充電可能な）」と「充電」が重複。「電気が流れる」と「端子」も重複。

整理後：「安定した充電のためには端子を清潔に保つこと」

英訳： Clean the terminals for stable charging.

（２） 空っぽ表現を捨てる 日本語には意味のない「飾り」表現が多い。これを除去してから英訳する。

例：空っぽ表現の除去

原文：「オフィスでの集中力の妨げの原因となる不快な騒音を排除することで、生産性の向上が期待できるようになる」

整理：「妨げの原因となる」→「妨げる」、「期待できるようになる」→不要

整理後：「騒音を排除すれば生産性が向上する」

英訳： Eliminating noise improves productivity.

確信度を調整したい場合は、助動詞を使う。

- 弱い主張（5%～10%程度）： Eliminating noise *can* improve productivity.
- 中程度の主張： Eliminating noise *may* improve productivity.
- 強い主張： Eliminating noise *will* improve productivity.

（３） 無駄な丁寧表現を除去する 日本語の丁寧表現は英語に直訳すると不自然になる。

例：丁寧表現の除去

原文：「本日、無事に設計の最終確認を終了することができました」

問題：「本日」「無事に」「終了することができました」は英語では不要

整理後：「設計の最終確認を完了した」

英訳：We completed the final design review.

プロンプト例

以下の日本語から、重複表現・空っぽ表現・無駄な丁寧表現を取り除き、簡潔な日本語に整理してください。

その後、整理した日本語を英訳してください。

[日本語原文を貼り付け]

4.6.2 まず具体化主義：曖昧な表現を具体的にする

日本語の曖昧な表現は、英語にすると意味が通らなくなる。英訳前に具体的な表現に置き換える。

例：曖昧表現の具体化

原文：「オイル量のチェックはできるだけ頻繁に行うことが望ましい」

問題：「できるだけ頻繁に」は英語にしにくい (as frequently as possible は不自然)

具体化：「オイル量は1日1回点検すること」

英訳：Check the oil level once a day.

数値の曖昧さにも注意が必要である。

例：数値の具体化

原文：「生産は数千万だ」

問題：「数千万」は英語でどう訳すか不明確

具体化：「年間生産台数は 3,600 万台である」

英訳：Annual production is 36 million units.

略語も曖昧表現の一種である。初出時には必ずスペルアウトする。

例：略語のスペルアウト

推奨：an antilock braking system (ABS)

許容：an ABS (antilock braking system)

不可：いきなり ABS と書く（読者が意味を知らない可能性がある）

4.6.3 とことん動詞主義：名詞句を動詞に戻す

日本語では「～の実施」「～の確認」「～を行う」のように動詞を名詞化する傾向がある。これをそのまま英訳すると、名詞句が連鎖した読みにくい英文になる。動詞に戻してから英訳する。

例：名詞句 → 動詞

原文：「付録 1 はユニットの分解手順の説明である」

名詞的英訳：Appendix 1 is a description of the disassembly procedure of the unit.

問題：前置詞 of が 3 回連続し、読みにくい

動詞的英訳：Appendix 1 describes how to disassemble the unit.

例：「行う」を削る

原文：「両企業はライセンス契約の修正を行った」

名詞的英訳： The two companies made an amendment to the license agreement.

動詞的英訳： The two companies amended the license agreement.

動詞を使うメリットは2つある。

1. **前置詞が減る：** 名詞句には前置詞 (of, to, for 等) が必要だが、動詞にすれば不要になることが多い。
2. **文が短くなる：** 「～の実施を行った」(名詞+動詞) が「～を実施した」(動詞のみ) になる。

4つの原則のまとめ： 英訳の前に日本語を整える4つのステップ：(1) すっきり最小主義で冗長さを削り、(2) 具体化主義で曖昧さをなくし、(3) 動詞主義で名詞句を動詞に戻し、(4) 能動態主義で受動態を能動態に変える。この4ステップを踏むだけで、AIによる英訳の品質は格段に上がる。

4.6.4 能動態と受動態の使い分け

(1) 「**まっすぐ能動態主義**」の原則 日本語は受動態を多用する言語であるが、英語—特に技術英語や学術英語—では能動態が好まれる。「すっきり最小主義」(余計な情報を削ぎ落とす)と「まっすぐ能動態主義」(まず能動態で書くことを第一選択とする)を基本原則として覚えておくとよい。

日本語の技術文書では、「～されている」「～が行われた」といった受動態が自然であり、多用される。しかし、これをそのまま英語にすると冗長で読みにくい文章になる。英語では「誰が何をしたか」を明確にする能動態がストレートで好まれる。

例 1: 日本語的な受動態から英語的な能動態へ

- **日本語:** 「作業油の流量は自動で調整されている」
- **受動態英訳:** “The flow of the fluid is automatically regulated.”
- **能動態英訳:** “Pressure-sensitive valves automatically regulate the flow of the fluid.” (主語が明確でストレート)

例 2: 学術論文での能動態と受動態

- **受動態:** “The experiment was conducted by the authors.” (冗長)
- **能動態:** “The authors conducted the experiment.” (よりストレート)
- **受動態:** “It was observed that the accuracy was improved.” (回りくどい)
- **能動態:** “We observed that the accuracy improved.” (明確)

(2) 受動態を使うべき 4 つの場面 「まっすぐ能動態主義」とはいえ、受動態が適切な場面もある。以下の 4 つのケースでは、受動態を積極的に使うべきである。

1. **主語の統一 (Subject Consistency) :** パラグラフ内で主語を統一するために受動態を使う。文ごとに主語がころころ変わると読みにくくなる。
2. **主語の短縮 (Shorter Subject) :** 能動態にすると主語が非常に長くなる場合、受動態を使って主語を短くする。
3. **責任のあいまい化 (Ambiguating Agency) :** 行為者を明示したくない場合、または行為者が重要でない場合に受動態を使う。
4. **話題の流れ (Topic Flow) :** 既知情報を文頭に、新情報を文末に配置するために受動態を使う。英語では「旧情報→新情報」の流れが自然である。

受動態が適切な場面の例

- 主語の統一: “The samples were collected from three sites. They were then filtered and analyzed.” (“The samples” を主語に統一)
- 手法の説明: “The samples were analyzed using mass spectrometry.” (誰がやったかより何をしたかが重要)
- 責任のあいまい化: “Mistakes were made in the initial calibration.” (誰の責任かを明示しない)

注意: 「うっかり受け身」は通じない

日本語では「うっかり壊してしまった」のように、意図せずにそうなったことを受身で表現する(迷惑の受身)。しかし英語には同じ機能の受動態がないため、日本語の感覚でそのまま受動態を使うと意味が通じなくなる。英語では意図の有無にかかわらず、行為者を主語にして能動態で書くのが原則である。

プロンプト例

以下の英文の受動態を能動態に書き換えてください。ただし、受動態の方が適切な場合はその理由(主語の統一・主語の短縮・責任のあいまい化・話題の流れのいずれか)とともに残してください。

[英文を貼り付け]

以下に、受動態から能動態への代表的な改善パターンを早見表としてまとめる。

表 4.2 受動態から能動態への改善例

受動態(冗長)	能動態(簡潔)
The model was trained by us using...	We trained the model using...
It was observed that the accuracy was improved by...	We observed that the accuracy improved by...
The data was collected from...	We collected data from...

Tip: “We” の使用

“We” を主語にすることに抵抗がある方もいるが、現代の学術英語では“We”の使用は一般的に受け入れられている。ただし、過度に“We”を繰り返さないよう、受動態と能動態を適切に混ぜることが理想的である。

4.6.5 冠詞 (a/the) の使い分け

(1) **なぜ冠詞が難しいのか** 冠詞 (a, an, the, 無冠詞) は、日本語にない概念であるため、日本語話者にとって最も間違いやすい文法事項である。冠詞の選択は、「読者がその名詞をどの程度特定できるか」という情報の共有状態に基づいている。

(2) **the の 3 つの用法** 定冠詞 the には、大きく分けて 3 つの用法がある。

1. **唯一照応**: 世界に一つしかないもの、文脈上唯一に定まるもの。

例: “the sun” (太陽は一つ)、“the Internet” (インターネットは一つ)、“the proposed method” (提案手法は一つ)

2. **前方照応**: 前の文で既に導入されたものを指す。a/an で初出した名詞を、2 回目以降 the で受ける。

例: “Click the button to open **a** new window. Drag the images onto **the** new window.” (a で導入→the で再参照)

3. **後方照応**: 名詞の直後に修飾語句 (関係節、前置詞句など) があり、それによって特定されるもの。

例: “Locate **the** four screws that are behind the lid.” (“that are behind the lid” によって特定される)

学術論文での冠詞の具体例

- 初出: “We propose **a** method for text classification.”
- 再出: “**The** method consists of three modules.” (前方照応)
- 特定: “**The** method proposed by Smith et al. (2023)” (後方照応)
- 唯一: “**The** proposed method outperforms all baselines.” (唯一照応)
- よくある間違い: “We propose method” → “We propose **a** method” (冠詞の欠落)

(3) **可算名詞と不可算名詞** 同じ単語でも、可算名詞と不可算名詞で意味が変わることがある。冠詞の有無が意味を決定するため、注意が必要である。

- **iron** (不可算) = 鉄 (物質) / **an iron** (可算) = アイロン (道具)
- **chicken** (不可算) = 鶏肉 (食材) / **a chicken** (可算) = ニワトリ (動物)
- **paper** (不可算) = 紙 (素材) / **a paper** (可算) = 論文
- **glass** (不可算) = ガラス (素材) / **a glass** (可算) = コップ

プロンプト例

以下の英文の冠詞 (a/an/the) の使い方を確認し、誤りがあれば修正してください。修正した場合は、唯一照応・前方照応・後方照応のどれに該当するか説明してください。

[英文を貼り付け]

以下に、冠詞の基本ルールを早見表としてまとめる。

基本ルール:

- **the**: 特定のもの、前出のもの、唯一のもの (唯一照応・前方照応・後方照応)
- **a/an**: 不特定のもの、初出のもの

- **無冠詞**: 複数形の一般論、不可算名詞の一般論

プロンプト例

以下の英語論文の冠詞の使用を確認してください。特に以下のパターンに注意してチェックしてください:

1. 初出の名詞に "the" を使っていないか
2. 前出の名詞に "a/an" を使っていないか
3. 数式や手法名に不要な冠詞がついていないか
4. "the proposed method" のように特定される名詞に "the" が付いているか

[英語論文を貼り付け]

4.6.6 時制の選択

(1) **論文のセクションと時制の関係** 英語論文では、セクションによって適切な時制が決まっている。これは単なる慣習ではなく、各セクションで述べる内容の性質に基づいている。

- **Methods (手法)**: 過去形を使う。実験は過去に行ったことだからである。

例: "We trained the model using 10,000 samples."

- **Results (結果)**: 過去形を使う。結果も過去に得たものだからである。

例: "The proposed method achieved an accuracy of 95.3%."

- **Introduction / Discussion**: 現在形を使う。一般的な事実や先行研究の知見は現在も成り立つものだからである。

例: "Deep learning has become a dominant approach in NLP."

(2) **単純現在形の本質: 再現性 100%** 単純現在形は、「いつでも・誰がやっても成り立つ事実」を述べるときに使う。つまり、再現性が 100%であることを主張する時制である。

- "Water boils at 100° C." (水は 100° C で沸騰する = いつでも成り立つ事実)
- "This algorithm converges in $O(n \log n)$ time." (このアルゴリズムは

$O(n \log n)$ で収束する = 数学的事実)

逆に言えば、単純現在形を使うということは「これは普遍的な事実である」と主張していることになる。自分の実験結果に単純現在形を使うと、「この結果はいつでも再現される」という強い主張になるので注意が必要である。

(3) **現在完了形の3つの用法** 現在完了形は学術論文で頻繁に使われるが、以下の3つの用法を区別して使う必要がある。

1. **完了 (already)** : 動作が完了したことを表す。

例: “We have completed the analysis.” (分析を完了した)

2. **経験 (times)** : 過去の経験を表す。

例: “Several studies have shown that this approach is effective.” (複数の研究が示してきた)

3. **継続 (for / since)** : 過去から現在まで続いていることを表す。

例: “This method has been used for over a decade.” (10年以上使われ続けている)

(4) **助動詞の使い分け** 助動詞は、話者の「確信度」を表す重要な手段である。学術論文では、主張の強さを適切にコントロールするために助動詞を使い分ける。

- **will** = 確信・コミットメント (確信度が高い)

例: “This approach will enable faster training.”

- **can** = 可能性・能力

例: “This method can handle large-scale datasets.”

- **may** = 不確実性・推測 (確信度が低い)

例: “This discrepancy may be due to overfitting.”

注意: “cannot” と “can not” の違い

“cannot” (1 語) は「～できない」という不可能の意味である。一方、“can not” (2 語) は “can not only A but also B” のように、“not” が後続の語句を否定する場合に使う。通常の「～できない」を表すときは必ず “cannot” と 1 語で書く。

以下に、セクション別の適切な時制を早見表としてまとめる。

表 4.3 セクション別の適切な時制

セクション	基本の時制	例
Introduction	現在形 (一般的事実) / 過去形 (先行研究の報告)	“Deep learning has achieved...” “Smith et al. (2023) proposed...”
Methods	過去形	“We trained the model using...”
Results	過去形	“The proposed method achieved...”
Discussion	現在形 (解釈) / 過去形 (結果への言及)	“These results suggest...” “Our model showed...”

プロンプト例

以下の英語論文の Methods セクションについて、時制が適切に使われているか確認してください。Methods では基本的に過去形を使用します。不適切な時制が使われている箇所を指摘し、修正してください。

[Methods セクションを貼り付け]

4.6.7 自動詞と他動詞の注意

英語では、同じ動詞でも自動詞として使うか他動詞として使うかで意味が大きく変わることがある。日本語では助詞(「を」「に」など)で関係を示すため、動詞の自他をあまり意識しないが、英語では目的語の有無が意味を決定する。

自動詞と他動詞で意味が変わる例

- **submit**
 - － 他動詞: “We submitted the paper.” (論文を提出した)
 - － 自動詞: “The army submitted.” (軍が降伏した・屈服した)
- **search**
 - － 他動詞: “I searched him.” (彼をボディチェックした = 身体検査した)
 - － 自動詞+前置詞: “I searched for him.” (彼を探した)
- **run**
 - － 自動詞: “The program runs.” (プログラムが動作する)
 - － 他動詞: “We run the program.” (プログラムを実行する)

特に“search”は要注意である。“search the database”は「データベースの中を探す」(正しい用法)だが、“search him”は「彼の身体を搜索する」という意味になり、「彼を探す」とは全く異なる。「～を探す」と言いたい場合は“search for”を使う。

学術論文で動詞を使う際は、辞書で自動詞・他動詞の区別を必ず確認すること。特に、日本語で「～を」と言える動詞が英語でも他動詞とは限らない点に注意が必要である。

4.6.8 直訳による不自然な表現

日本語を直訳すると、文法的には正しくても不自然な英語になることがある。

表 4.4 直訳による不自然な表現と自然な英語

日本語直訳 (不自然)	自然な英語
“We could confirm that...”	“We confirmed that...”
“It is considered that...”	“We consider...” / “This suggests...”
“From this result...”	“Based on these results...”
“We performed the experiment...”	“We conducted experiments...”
“The accuracy became high...”	“The accuracy improved...”
“As a merit of our method...”	“An advantage of our method is...”

—— プロンプト例 ——

以下の英語論文について、日本語の直訳に起因する不自然な表現を特定してください。各箇所について、より自然な英語表現を提案し、なぜその表現の方が自然かを説明してください。

[英語論文を貼り付け]

4.6.9 専門用語の確認方法

専門用語の英語表現が分からない場合や、正しいかどうか確認したい場合は、以下の方法が有効である。

1. **Google Scholar での用例検索:** “用語” を含む論文を検索し、トップ論文でどのように使われているか確認する
2. **AI に確認を依頼:**

—— プロンプト例 ——

「[日本語の専門用語]」の英語表現として、以下のどちらが [分野名] の学術論文で一般的ですか？

A: [候補 1] B: [候補 2]

それぞれの使用頻度や文脈の違いがあれば教えてください。また、他に一般的な表現があれば提示してください。

表 4.5 Introduction で使える定型表現

目的	表現例
分野の重要性を述べる	“X has attracted significant attention in recent years.” “X plays a crucial role in...” “X is of fundamental importance to...”
先行研究を紹介する	“Several studies have investigated...” “Previous work has shown that...” “Smith et al. (2023) demonstrated that...”
問題点を指摘する	“However, existing methods suffer from...” “Despite these advances, ... remains challenging.” “A major limitation of prior work is...”
本研究の目的を述べる	“In this paper, we propose...” “This work addresses the problem of...” “We present a novel approach to...”
貢献を列挙する	“Our main contributions are as follows:” “The key contributions of this work include:”

表 4.6 Methods で使える定型表現

目的	表現例
手法の全体像を述べる	“Our approach consists of three main components:” “The overall architecture is illustrated in Figure X.” “We formulate the problem as...”
手順を説明する	“First, we... Then, we... Finally, we...” “The input is processed through...” “We apply X to obtain Y.”
数式を導入する	“The objective function is defined as:” “We optimize the following loss function:” “where X denotes the...”
実装の詳細を述べる	“We implemented our method using...” “All experiments were conducted on...” “The hyperparameters were tuned via...”

表 4.7 Results で使える定型表現

目的	表現例
結果を提示する	“Table X summarizes the results of...” “As shown in Figure X, our method...” “The results are presented in Table X.”
比較する	“Our method outperforms the baseline by X%.” “Compared to the state-of-the-art, our approach achieves...” “The improvement is statistically significant ($p < 0.05$).”
傾向を述べる	“We observe a consistent improvement across...” “The performance increases as X grows.” “Notably, our method shows robust performance on...”

表 4.8 Discussion / Conclusion で使える定型表現

目的	表現例
結果を解釈する	“These results suggest that...” “One possible explanation is that...” “This finding is consistent with...”
限界を述べる	“A limitation of our study is...” “Our approach may not generalize to...” “One potential concern is...”
将来の方向を述べる	“Future work could explore...” “A promising direction is to...” “We plan to extend this work by...”
結論を述べる	“In summary, we have presented...” “This work demonstrates the effectiveness of...” “Our findings highlight the potential of...”

4.7 学術英語表現集

4.7.1 Introduction の定型表現

4.7.2 Methods の定型表現

4.7.3 Results の定型表現

4.7.4 Discussion / Conclusion の定型表現

Tip: 定型表現の活用法

これらの表現を暗記する必要はない。執筆時に参照表として使い、自分の分野の論文でよく見かける表現を徐々に身につけていくとよい。定型表現は「型」として活用し、その中に自分の研究の独自の内容を盛り込むことが重要である。

演習問題

1. 演習 1: IMRaD 構造の分析

自分の研究分野のトップ会議またはジャーナルから論文を 1 本選び、以下を行いなさい。

- (a) 各セクション (Introduction, Methods, Results, Discussion) の段落数と概算語数を調べる
- (b) Introduction の「逆三角形構造」(背景→問題→本研究の貢献) を具体的に特定する
- (c) 各段落のトピックセンテンスを抜き出し、それだけを読んで論文の流れが理解できるか確認する
- (d) この分析から得られた、英語論文の構造に関する気づきをまとめる

2. 演習 2: AI を使った英訳の実践

自分の研究に関する日本語の文章 (300–500 字) を用意し、以下を行い

なさい。

- (a) 本章の Step 1 のチェックリストを使って日本語原稿を確認・修正する
- (b) Step 2 のプロンプトテンプレートを使って、AI に英訳を依頼する
- (c) Step 3 のプロンプトを使って、英訳を学術的に洗練する
- (d) Step 2 の英訳と Step 3 の洗練後の英訳を比較し、どのような改善がなされたかを分析する
- (e) 自分の分野の論文と表現を比較し、さらなる改善点がないか検討する

3. 演習 3: 逆翻訳チェックの実践

演習 2 で作成した英語論文の一部について、以下を行いなさい。

- (a) 別の AI ツール（演習 2 で Claude を使った場合は ChatGPT 等）で日本語に逆翻訳する
- (b) 逆翻訳された日本語を元の日本語原稿と比較し、意味のズレを特定する
- (c) ズレが見つかった箇所について、英訳のどこに問題があるかを分析する
- (d) 英訳を修正し、再度逆翻訳して意味のズレが解消されたか確認する

4. 演習 4: Abstract 作成の実践

自分の研究（または研究計画）について、以下を行いなさい。

- (a) 本章の Abstract 構造（背景→目的→手法→結果→意義）に従い、各要素を日本語で箇条書きにする
- (b) 本章のプロンプトテンプレートを使って、AI に英語の Abstract を作成させる
- (c) 作成された Abstract が 150–200 語に収まっているか確認する
- (d) 本章の「良い Abstract の条件」（具体性、数値、構造）を満たしているか評価する
- (e) 必要に応じて修正し、最終版を完成させる

5. 演習 5: 総合推敲の実践

演習 2-4 で作成した英語テキストをまとめ、以下の段階的推敲を行いなさい。

- (a) 第1段階として、文法チェックのプロンプトを使って文法エラーを修正する
- (b) 第2段階として、表現チェックのプロンプトを使って学術的表現に洗練する
- (c) 第3段階として、論理チェックのプロンプトを使って論理構造を確認する
- (d) 最後に、ネイティブチェック風レビューのプロンプトを使って自然さを確認する
- (e) 各段階で見つかった問題の種類と数を記録し、自分の英語ライティングの傾向を分析する

本章のまとめ

英語論文の執筆は、日本語話者にとって大きな挑戦であるが、AIを活用することでそのハードルを大幅に下げることができる。重要なのは、AIを「翻訳機」としてではなく「執筆パートナー」として活用することである。AIの出力をそのまま使うのではなく、段階的な推敲プロセスを通じて品質を高め、最終的には自分の言葉として責任を持てる英語論文に仕上げるのが大切である。繰り返し実践することで、AIに頼らなくても書ける力が自然と身についていく。

引用・参考文献

5

研究発表スライドの作成

【この章で学ぶこと】

- AIを活用して論理的で説得力のある研究発表スライドを作成できる — 発表の構成案からスライド内容まで、AIを効果的に活用した作成プロセスを実践できる
- デザイン AI ツールを使った効率的なスライド作成法を習得する — Gamma や Canva AI などのツールを活用し、視覚的に優れたスライドを短時間で作成できる
- 図表・概念図の生成と批判的な検討ができる — AIによる図表生成の可能性と限界を理解し、適切に活用・検証できる

5.1 研究発表スライドの基本

5.1.1 学会発表の基本構成

研究発表のスライドは、論文と同様に論理的な構成が求められる。ただし、スライドは「読む」ものではなく「見せる」ものであるため、論文とは異なるアプローチが必要である。

標準的な発表構成:

表 5.1 研究発表スライドの標準的な構成

スライド	内容	目的
タイトル	研究タイトル、著者名、所属	研究の第一印象を決める
背景	研究分野の現状と課題	聴衆に問題の重要性を理解させる
目的	本研究で解決する問題	研究のゴールを明確にする
関連研究	先行研究と本研究の位置づけ	新規性の根拠を示す
提案手法	アプローチの概要と詳細	技術的な貢献を説明する
実験設定	データセット、評価指標、比較手法	実験の妥当性を示す
結果	主要な実験結果	提案手法の有効性を示す
考察	結果の解釈、分析	結果の意味を深く理解させる
まとめ	貢献の要約と将来展望	研究のインパクトを印象づける

5.1.2 時間配分の目安（10 分発表の場合）

10 分間の口頭発表は、学会で最も一般的な発表形式の一つである。以下に推奨される時間配分を示す。

表 5.2 10 分発表の推奨時間配分

セクション	時間	スライド枚数（目安）
タイトル・自己紹介	0.5 分	1 枚
背景・問題設定	2 分	2-3 枚
提案手法	3 分	3-4 枚
実験結果	2.5 分	2-3 枚
考察・まとめ	2 分	2 枚
合計	10 分	10-13 枚

1 枚のスライドに費やす時間は、平均して 45 秒から 1 分程度である。情報を詰め込みすぎると、聴衆は内容を理解する前に次のスライドに移ってしまう。

5.1.3 良いスライドの原則

1 スライド 1 メッセージの原則

各スライドが伝えるべきメッセージは1つだけである。複数のメッセージを1枚に詰め込むと、聴衆はどこに注目すべきかわからなくなる。スライドのタイトルに、そのスライドのメッセージを端的に表現するとよい。

悪い例のタイトル: 「実験結果」

良い例のタイトル: 「提案手法は全データセットでベースラインを上回った」

文字量の制限:

- タイトル: 1行 (最大15語程度)
- 本文: 1スライドあたり最大6行
- フォントサイズ: タイトル 28pt 以上、本文 20pt 以上
- 箇条書きは3-5項目まで

視覚的な原則:

- 図表を積極的に使う (テキストより図の方が記憶に残る)
- 色の使いすぎを避ける (メインカラー + アクセントカラーの2-3色)
- 余白を十分にとる (スライドの30%以上は空白にする)
- アニメーションは必要最小限にする

Tip: スライド作成の出発点

スライドを作成する前に、まず「各スライドで伝えたいメッセージ」を箇条書きで書き出すとよい。メッセージの一覧が論理的に流れていれば、スライド作成はスムーズに進む。この段階で AI を活用すると効率的である。

5.2 AI による構成案の生成

5.2.1 研究テーマの構造化

AI に構成案を依頼する前に、自分の研究を1-2文で簡潔に表現できるように

しておくことが重要である。これは「エレベーターピッチ」とも呼ばれ、短時間で研究の本質を伝える技術である。

プロンプト例

私の研究を以下の情報をもとに、1-2 文の簡潔な説明にまとめてください。

- 研究分野: [分野] - 解決する問題: [問題] - 提案するアプローチ: [手法の概要] - 主な結果: [成果] - 研究の意義: [意義]

対象聴衆: [例: 同じ分野の研究者 / 異分野の研究者 / 一般的な理系の聴衆]

実例：研究テーマの整理

整理前（冗長な記述）：

「私は、水耕栽培における光源の種類がレタスの成長速度と栄養価に与える影響について、LED と蛍光灯を比較する実験を行い、光合成効率と二次代謝産物の生成量の観点から分析しています。」

整理後（1-2 文に凝縮）：

「水耕栽培レタスにおける LED 光源と蛍光灯の比較研究：成長速度と栄養価への影響」

このように、研究の核心を 1-2 文に絞ることが AI への指示の出発点となる。冗長な記述では、AI も焦点の定まらない構成案を生成しがちである。

5.2.2 AI に構成案を提案させるプロンプト設計

研究テーマが整理できたら、AI に発表スライドの構成案を提案させる。

プロンプト例

以下の研究について、10 分間の学会口頭発表のスライド構成案を作成してください。

【研究概要】タイトル: [研究タイトル] 分野: [研究分野] 概要: [1-2 文の研究概要]

【詳細情報】背景: [研究の背景と動機] 提案手法: [手法の概要] 主要結果: [主な実験結果] 結論: [研究の結論]

【発表条件】発表時間: 10 分 (質疑 5 分) 対象聴衆: [聴衆の専門性レベル] 発表言語: [日本語/英語]

以下の形式で構成案を作成してください: - スライド番号- スライドタイトル (メッセージ型) - 含めるべき内容 (箇条書き 3-5 項目) - 想定所要時間- 視覚要素の提案 (図表、グラフなど)

5.2.3 生成された構成案の批判的検討

AI が生成した構成案は、必ず以下の 4 つの観点から批判的に検討する。

1. 論理一貫性の確認:

スライドのタイトルだけを順番に読んで、研究のストーリーが自然に流れているか確認する。

プロンプト例

以下のスライド構成案について、論理的な一貫性を確認してください。スライドタイトルの順序を読むだけで、研究のストーリーが伝わるかどうかを評価してください。

論理の飛躍、不足している要素、順序の改善案があれば指摘してください。
[スライドタイトルのリスト]

2. 網羅性の確認:

必要な情報がすべて含まれているか確認する。特に以下の要素は必須である。

- 研究の動機（なぜこの研究が重要か）
- 先行研究との差別化（何が新しいのか）
- 手法の核心的なアイデア（どのように解決するか）
- 定量的な結果（どの程度うまくいったか）
- 限界と将来展望（残された課題は何か）

3. 独自性の確認:

構成案が一般的すぎないか、自分の研究の独自性が反映されているか確認する。AI は汎用的な構成を提案する傾向があるため、自分の研究の強みが際立つように修正を加える。

4. 時間配分の確認:

各スライドの想定所要時間を合計し、発表時間内に収まるか確認する。実際に声に出してリハーサルすると、想定よりも時間がかかることが多いため、余裕を持った時間配分にするとよい。

Tip: 早い段階でのフィードバック

構成案の段階で指導教員やゼミのメンバーにフィードバックをもらうのが効果的である。スライドを作り込んでからの大幅な修正は時間のロスが大きいため、早い段階で方向性を固めるとよい。

5.3 各スライドの内容生成

5.3.1 研究背景の文章生成

研究背景のスライドは、聴衆の注意を引き、研究の重要性を理解してもらうための出発点である。

プロンプト例

以下の研究テーマの背景を、学会発表スライド用の簡潔な文章として生成してください。

【研究テーマ】：[テーマ] 【解決したい問題】：[問題] 【対象聴衆】：[聴衆の属性]

以下の条件を守ってください: 1. 箇条書き形式で、1項目あたり1行(15-20文字程度) 2. 専門用語は[聴衆の属性]が理解できるレベルにする 3. 問題の重要性が直感的に伝わるデータや事実を含める 4. 最後に「この問題をどう解決するか？」への橋渡しとなる文で締める 5. 合計 5-6 行以内

5.3.2 研究手法の説明文生成

提案手法の説明は、発表の核心部分である。複雑な手法を聴衆に理解してもらうためには、段階的な説明が重要である。

プロンプト例

以下の研究手法を、学会発表スライド用に段階的に説明する文章を生成してください。

【手法の概要】：[手法の詳細な説明]

以下の構成で、3枚のスライドに分けてください:

スライド 1: 手法の全体像 (Overview) - 手法の核心的なアイデアを1文で- 全体のパイプラインを箇条書き 3-4項目で

スライド 2: 技術的な詳細 (Key Technique) - 最も重要な技術的貢献に焦点- 数式は最小限 (1-2個まで) - 直感的な説明を優先

スライド 3: 実装のポイント- 実用的な工夫- 計算コストや効率性

各スライドは5行以内で、図表の提案も含めてください。

5.3.3 研究結果のまとめ生成

プロンプト例

以下の実験結果を、学会発表スライド用にまとめてください。

【実験結果】：[結果のデータを貼り付け]

以下の形式でお願いします: 1. 主要な発見を重要度順に3つ挙げる 2. 各発見を1行で端的に表現する 3. 数値は効果的に強調する(例: "精度 12.5% 向上") 4. 図表(グラフの種類)の提案 5. 聴衆が「なるほど」と思えるような解釈を1文添える

5.3.4 専門用語と平易な表現のバランス調整

学会発表では、聴衆の専門性に応じて表現のレベルを調整する必要がある。

プロンプト例

以下のスライド内容について、対象聴衆に合わせて表現を調整してください。

【現在の内容】：[スライドの内容]

【対象聴衆】：[例: 情報科学を専攻する修士学生]

調整の方針: 1. 聴衆が確実に知っている用語はそのまま使用 2. 聴衆が知らない可能性のある用語は、初出時に簡単な説明を付加 3. 略語は初出でフルスペルを示す 4. 過度に専門的な詳細は省略し、直感的な説明に置き換える

5.3.5 AIの出力を自分の言葉に修正する重要性

AIが生成したスライド内容は、あくまで出発点である。以下の理由から、必ず自分の言葉に修正すること。

なぜ自分の言葉に修正が必要か

1. **発表の自然さ:** AI が生成した文章をそのまま読み上げると、不自然な発表になる。自分が普段使う言葉遣いに修正することで、自信を持って発表できる。
2. **質疑への対応:** 自分の言葉でスライドを作成していれば、質疑応答で深い質問をされても自然に回答できる。AI の文章をそのまま使っていると、内容を十分に理解していないことが露呈するリスクがある。
3. **独自の視点の反映:** AI は汎用的な表現を生成するが、あなた自身の研究に対する理解や解釈は、自分の言葉でしか表現できない。

Tip: 違和感を大切にする

AI が生成した内容を読んで「これは自分の考えと違う」と感じた箇所は、積極的に修正するとよい。その違和感こそが、あなた自身の研究に対する理解の深さを反映している。

5.4 デザイン AI によるスライド作成

5.4.1 Gamma — テキストからスライドを自動生成

Gamma (gamma.app) は、テキスト入力から高品質なプレゼンテーションスライドを自動生成する AI ツールである。研究発表スライドの初稿を短時間で作成するのに適している。

基本的な使い方:

1. Gamma のウェブサイトアクセスし、アカウントを作成する
2. 「Create new」から「Presentation」を選択する
3. 研究の概要テキストを入力するか、アウトライン形式で構成を指定する
4. AI がスライドのデザインと内容を自動生成する
5. 生成されたスライドを編集・カスタマイズする

研究発表に使う際のプロンプト例:**プロンプト例**

Create a 12-slide academic research presentation:

Title: [研究タイトル]

Slide 1: Title slide Slide 2: Research Background - [背景の要点] Slide 3: Problem Statement - [問題の記述] Slide 4: Related Work - [先行研究の位置づけ] Slide 5-7: Proposed Method - [手法の説明] Slide 8: Experimental Setup - [実験設定] Slide 9-10: Results - [結果の要点] Slide 11: Discussion - [考察の要点] Slide 12: Conclusion and Future Work - [結論]

Style: Clean, minimal, academic Color scheme: Blue and white

Gamma の長所:

- テキストから短時間でスライドを生成できる
- デザインが洗練されている
- 図表のプレースホルダーが自動配置される
- エクスポート形式が豊富（PDF, PPTX 等）

Gamma の短所:

- 細かいレイアウトの調整には限界がある
- 数式の表示には対応していない場合がある
- 学術発表特有のフォーマット（ヘッダー/フッターなど）のカスタマイズが限定的
- 無料プランでは一部機能が制限される

5.4.2 Canva AI — テンプレートベースのデザイン支援

Canva (canva.com) は、テンプレートベースのデザインツールで、AI 機能 (Magic Design) によるスライド作成支援を提供している。

基本的な使い方:

1. Canva にアクセスし、「プレゼンテーション」を選択する
2. テンプレートを選択する（学術発表向けテンプレートが複数用意されている）
3. Magic Design にテーマを入力すると、テンプレートに合わせたスライドが提案される
4. 各スライドの内容を編集する
5. 図表やアイコンを Canva の素材ライブラリから追加する

学術発表に適したテンプレートの選び方:

- シンプルで余白が十分にあるもの
- 色数が少ないもの（2-3 色）
- フォントが読みやすいもの（サンセリフ体推奨）
- 装飾的な要素が少ないもの
- グラフや表のプレースホルダーがあるもの

Tip: 最終調整は PowerPoint で

デザイン AI で作成したスライドは、必ず PowerPoint や Keynote 形式でエクスポートし、最終的な微調整はこれらのツールで行うとよい。特に数式、特殊な図表、アニメーションについては、PowerPoint 等で仕上げた方が柔軟性が高い。

5.4.3 デザイン AI の使い分け

表 5.3 デザイン AI ツールの比較

項目	Gamma	Canva AI
強み	テキストからの自動生成	テンプレートの豊富さ
作成速度	非常に速い	速い
カスタマイズ性	中程度	高い
数式対応	限定的	限定的
共同編集	対応	対応
学術向けテンプレート	少なめ	多い
推奨用途	初稿の高速作成	デザインの洗練

5.5 図表・概念図の生成

5.5.1 画像生成 AI による概念図

研究発表では、手法の概念図やシステムの全体像を示す図が非常に重要である。画像生成 AI（DALL-E、Stable Diffusion 等）を活用して、概念図のアイデアを得ることができる。

活用できる場面:

- 研究のイメージを伝えるための概要図
- 応用分野のイラスト（医療、環境、ロボティクスなど）
- スライドの背景画像やアイコン

活用が難しい場面:

- 正確な技術的アーキテクチャ図（ネットワーク構造など）
- 数値データを含むグラフや表
- 数式や記号を含む図

プロンプト例:

プロンプト例

Create a clean, minimalist scientific illustration showing the concept of transfer learning in natural language processing. Show knowledge flowing from a large pre-trained model to a smaller task-specific model. Use a blue and white color scheme. Style: technical diagram, flat design, no text.

Tip: 2段階アプローチ

画像生成 AI は概念的なイラストの「アイデア出し」には有用であるが、最終的な研究発表用の図は、専用のツール（PowerPoint、Illustrator、draw.io 等）で自作することを推奨する。生成画像には不正確な要素が含まれている可能性があり、学術発表での使用には注意が必要である。

5.5.2 テキストベース図表ツール — Mermaid.js の活用

Mermaid.js は、テキストベースで図表を生成するツールである。AI に Mermaid コードを生成させることで、フローチャート、シーケンス図、状態遷移図などを効率的に作成できる。

フローチャートの生成プロンプト:

プロンプト例

以下の研究手法のパイプラインを、Mermaid.js のフローチャートとして表現してください。

【手法のパイプライン】: 1. 入力データの前処理（テキストのトークナイズ、正規化）2. 特徴抽出（BERT によるエンコーディング）3. 分類層（全結合層 + Softmax）4. 出力（カテゴリラベル）

各ステップ間のデータの流れが明確にわかるようにしてください。重要なステップは色を変えて強調してください。

AI が生成する Mermaid コードの例:

```

1 graph TD
2     A[入力テキスト] --> B[前処理]
3     B --> B1[B1 トークナイズ]
4     B --> B2[B2 正規化]
5     B1 --> C[特徴抽出]
6     B2 --> C
7     C --> C1[C1 [BERT エンコーダ]]
8     C1 --> D[分類層]

```

```
9      D --> D1全結合層 []
10     D1 --> D2[Softmax]
11     D2 --> Eカテゴリラベル出力 []
12
13     style C1 fill:#4A90D9,stroke:#333,color:#fff
14     style D2 fill:#4A90D9,stroke:#333,color:#fff
```

シーケンス図の生成プロンプト:

プロンプト例

以下のシステムの処理フローを、Mermaid.js のシーケンス図として表現してください。

【処理フロー】: 1. ユーザーがクエリを入力 2. フロントエンドが API サーバーにリクエストを送信 3. API サーバーがデータベースに問い合わせ 4. データベースが結果を返す 5. API サーバーが AI モデルに推論を依頼 6. AI モデルが結果を返す 7. API サーバーがフロントエンドにレスポンスを返す 8. フロントエンドがユーザーに結果を表示

研究手法の比較図:

プロンプト例

以下の 3 つの手法の比較を、Mermaid.js の図として表現してください。各手法の入力、処理ステップ、出力を並列に表示し、共通点と相違点が一目でわかるようにしてください。

手法 A: [概要] 手法 B: [概要] 手法 C (提案手法): [概要]

Mermaid コードの活用方法:

生成された Mermaid コードは、以下の方法でスライドに組み込むことができる。

1. **Mermaid Live Editor** (mermaid.live) で SVG または PNG 画像としてエクスポート
2. **VS Code 拡張機能** (Markdown Preview Mermaid Support) でプレ

ビュー

3. **GitHub の Markdown** は Mermaid を直接レンダリング可能
4. エクスポートした画像を PowerPoint や Keynote に貼り付け

5.5.3 生成画像の著作権と正確性の検証

AIで生成した図表をスライドに使用する際は、以下の点に注意が必要である。

著作権に関する注意:

- 画像生成 AI の利用規約を確認する（商用利用、学術利用の可否）
- DALL-E で生成した画像は OpenAI の利用規約に基づいて使用可能
- Stable Diffusion はモデルのライセンスによって異なる
- 学会の規定によっては、AI 生成画像の使用に制限がある場合がある

正確性の検証:

- AI が生成した概念図の要素が正確かどうかを確認する
- 生成画像に含まれるテキストは不正確なことが多いため、自分で修正する
- 技術的な図（ネットワーク構造、数式を含む図）は特に注意して検証する
- 不正確な図を使用すると、査読者や聴衆の信頼を損なう可能性がある

Tip: Overview 図の作成

学会発表では、手法の全体像を示す「Overview 図」が特に重要である。AI で概念図のラフ案を生成し、それを参考にして PowerPoint や draw.io で正確な図を自作するという 2 段階のアプローチが効果的である。最終的な図には、自分で正確性を保証できるものだけを使用すること。

5.6 相互レビューと改善

5.6.1 ピアレビューのチェックポイント

スライドが完成したら、発表前にピアレビュー（相互レビュー）を行うとよい。以下のチェックポイントを活用すること。

内容面のチェック:

- 研究の動機が明確に伝わるか
- 提案手法の核心的なアイデアが理解できるか
- 実験結果が説得力を持って提示されているか
- 結論が実験結果に基づいているか
- スライド間の論理的なつながりがスムーズか

デザイン面のチェック:

- テキストが読みやすいか（フォントサイズ、コントラスト）
- 図表が鮮明で、説明なしでも理解できるか
- 色の使い方が一貫しているか
- 余白が十分にあるか
- アニメーションが適切か（過度でないか）

発表面のチェック:

- 制限時間内に収まるか
- 各スライドのメッセージが明確か
- 専門用語が適切に説明されているか
- 質疑応答で予想される質問への準備ができているか

5.6.2 AIに「聴衆として」フィードバックさせる

人間のレビューに加えて、AIにも聴衆の視点からフィードバックを求めることができる。

基本プロンプト:

プロンプト例

あなたは[分野名]の学会に参加している研究者です。以下のスライド内容（テキスト版）を読み、発表に対するフィードバックを提供してください。

【スライド内容】スライド 1: [タイトルと内容] スライド 2: [タイトルと内容] ...

以下の観点からフィードバックをお願いします: 1. ストーリーの流れは自然か? 論理的な飛躍はないか? 2. 背景と動機は十分に説得力があるか? 3. 提案手法の説明で不明確な点はないか? 4. 実験結果の提示は効果的か? 5. 聴衆として、質疑応答で聞きたい質問は何か? (3つ挙げてください) 6. 全体的な改善提案

異なる聴衆レベルでのフィードバック:

プロンプト例

以下のスライド内容について、異なる立場の聴衆からのフィードバックをシミュレーションしてください。

聴衆 A: この分野の専門家（教授レベル） 聴衆 B: 同じ学科の別分野の大学院生 聴衆 C: 理系だが異なる学部の学部生

それぞれの聴衆が「理解できなかった」と感じる可能性が高いポイントを指摘してください。

[スライド内容]

質疑応答の準備:

プロンプト例

以下の研究発表スライドの内容をもとに、学会の質疑応答で想定される質問を10個生成してください。

質問は以下のカテゴリに分類してください: 1. 手法に関する技術的な質問 (3問) 2. 実験設定に関する質問 (2問) 3. 結果の解釈に関する質問 (2問) 4. 研究の限界や将来展望に関する質問 (2問) 5. 厳しい指摘・批判的な質問 (1問)

各質問に対する推奨回答の骨子も示してください。

[スライド内容]

Tip: AIと人間のフィードバックの組み合わせ

AIからのフィードバックと人間からのフィードバックは、異なる強みを持っている。AIは論理的な一貫性や網羅性のチェックに優れているが、発表の「温度感」や「説得力」は人間のフィードバックの方が的確である。両方を組み合わせることで、より完成度の高い発表になる。

演習問題

1. 演習 1: 1 スライド 1 メッセージの実践

自分の研究（または興味のある研究トピック）について、以下を行いなさい。

- (a) 10分間の口頭発表を想定し、各スライドで伝えたいメッセージを箇条書きで12個書き出す
- (b) 各メッセージが「1スライド1メッセージ」の原則を満たしているか確認する（複数のメッセージが混在していたら分割する）
- (c) メッセージの順序を読むだけで、研究のストーリーが自然に流れるか確認する

- (d) 必要に応じてメッセージの追加・削除・順序変更を行い、最終的な構成案を完成させる

2. 演習 2: AI による構成案の比較

本章のプロンプトテンプレートを使って、2つの異なる AI（例: Claude と ChatGPT）にスライド構成案を生成させ、以下を行いなさい。

- (a) 2つの構成案の共通点と相違点を整理する
- (b) 各構成案の長所と短所を分析する
- (c) 2つの構成案の良い部分を組み合わせて、最終的な構成案を作成する
- (d) 最終構成案が、本章で述べた4つの検討観点（論理一貫性、網羅性、独自性、時間配分）を満たしているか確認する

3. 演習 3: Mermaid.js による図表作成

自分の研究手法のパイプライン（または興味のある手法）について、以下を行いなさい。

- (a) AI に Mermaid.js のフローチャートコードを生成させる
- (b) Mermaid Live Editor (mermaid.live) でコードを貼り付け、図を確認する
- (c) 図が正確に手法を表現しているか検証し、必要に応じてコードを修正する
- (d) SVG または PNG 形式でエクスポートし、スライドに挿入する
- (e) テキストベース図表ツールの長所と短所をまとめる

4. 演習 4: デザイン AI の活用

演習 1 または 2 で作成した構成案をもとに、以下を行いなさい。

- (a) Gamma (gamma.app) を使ってスライドの初稿を自動生成する
- (b) 生成されたスライドのデザインと内容进行评估する（良い点、改善すべき点）
- (c) 内容を自分の言葉に修正する
- (d) デザインの改善（色、フォント、レイアウト、余白）を行う
- (e) PowerPoint または Keynote 形式でエクスポートし、最終的な微調

整を行う

5. 演習 5: AI を活用した質疑応答準備

完成したスライド（演習 4 の成果物、またはこれまでに作成したスライド）について、以下を行いなさい。

- (a) 本章のプロンプトを使って、AI に想定質問を 10 個生成させる
- (b) 各質問に対する回答を自分で準備する（AI の推奨回答を参考にしつつ、自分の言葉で）
- (c) 特に難しいと感じた質問 3 つについて、回答を声に出して練習する
- (d) ゼミや研究室メンバーの前でリハーサル発表を行い、実際にどのような質問が出るかを記録する
- (e) AI が予測した質問と実際の質問を比較し、AI による質疑準備の有効性を評価する

本章のまとめ

研究発表スライドの作成は、研究内容の理解と伝達力の両方が求められる総合的な作業である。AI は構成案の生成、内容の整理、図表の作成、フィードバックの提供など、多くの段階で役に立つ。しかし、最も重要なのは「自分の研究を自分の言葉で伝える」ことである。AI の出力は必ず自分で検証・修正し、発表のリハーサルを十分に行うこと。良い発表は、優れたスライドと十分な練習の両方から生まれる。

引用・参考文献

6

AI によるコード生成とテスト

【この章で学ぶこと】

- AI を活用したコード生成の基本フローとプロンプト設計を習得する — 自然言語からコードを生成するプロセスを理解し、効果的なプロンプトを書けるようになる
- テストコードの自動生成と AI コードレビューを実践できる — テスト駆動開発の考え方を学び、AI を活用した品質向上手法を身につける
- AI 生成コードの品質・セキュリティリスクを理解し対処できる — 生成コードを盲信せず、適切に検証・修正する力を養う

6.1 AI とソフトウェア開発の現在

6.1.1 コード生成 AI の普及状況

2024 年以降、AI によるコード生成は研究者・開発者の日常的なツールとなった。主要なツールとその特徴を以下に整理する。

6.1.2 開発者の AI 利用統計

GitHub の 2024 年開発者調査によると、回答者の 92%が AI コーディング

表 6.1 主要なコード生成 AI ツール

ツール	特徴	料金
GitHub Copilot	VS Code 統合、コード補完に特化	学生無料 (GitHub Education)
Cursor	AI 統合型エディタ、対話的開発	無料枠あり、Pro 月\$20
ChatGPT / GPT-4	汎用的、説明付きコード生成	無料枠あり、Plus 月\$20
Gemini Pro	Google 連携、長文コンテキスト	無料枠あり
Claude	長文コンテキスト、正確な推論	無料枠あり、Pro 月\$20

ツールを職場内外で利用しており、70%以上が「生産性が向上した」と回答している。特に以下の場面での活用が報告されている。

- 定型的なコードの作成（ボイラープレートコード）：87%
- 新しいプログラミング言語の学習：73%
- バグの特定と修正：68%
- テストコードの作成：62%

6.1.3 AI が変える開発ワークフロー

従来の開発ワークフローでは、プログラマーが仕様を読み、設計し、コードを行わずつ書いていた。AI の登場により、このプロセスは大きく変わりつつある。

従来のワークフロー：

仕様理解 → 設計 → コーディング → テスト → デバッグ → レビュー

AI 活用ワークフロー：

仕様記述（プロンプト） → AI 生成 → 人間がレビュー・修正
→ AI でテスト生成 → 検証

重要なのは、AI は「コードを書く」部分を加速するが、「何を作るか」「なぜ作るか」を考えるのは依然として人間の仕事であるということだ。大学院生にとって、研究のロジックを明確にする力がますます重要になっている。

6.2 AI によるコード生成の基本

6.2.1 基本フロー

AI によるコード生成は、以下の 3 ステップで行う。

Step 1: 自然言語で仕様を記述する（プロンプト作成）

Step 2: AI がコードを生成する

Step 3: 人間がレビュー・テスト・修正する

このうち、最も重要なのは **Step 1（プロンプト作成）** である。指示が曖昧だと、生成されるコードも意図に沿わないものになりやすい。明確で具体的な仕様を書くことが、良いコードを得る鍵である。

6.2.2 効果的なプロンプトの 4 要素

AI に高品質なコードを生成させるためには、以下の 4 つの要素をプロンプトに含めることが重要である。

表 6.2 効果的なプロンプトの 4 要素

要素	内容	例
1. 言語・フレームワーク指定	使用する言語やライブラリを明示	「Python 3.10、pandas を使用」
2. 入出力仕様	何を受け取り、何を返すか	「CSV ファイルパスを受け取り、 DataFrame を返す」
3. 制約条件	パフォーマンス、エラー処理等	「メモリ効率を重視、 欠損値は NaN で保持」
4. 例示	具体的な入出力例	「入力: 'data.csv' → 出力: 100 行× 5 列の DataFrame」

6.2.3 Bad prompt vs Good prompt の比較

Bad prompt (悪い例) :

プロンプト例

CSV を読み込む関数を書いて

この指示では、言語もライブラリも不明であり、エラー処理の有無も分らない。AI は何らかのコードを返すが、研究の要件に合うかは運次第である。

Good prompt (良い例) :

プロンプト例

以下の仕様で Python 関数を作成してください。

関数名 load_experiment_data

目的実験結果の CSV ファイルを読み込み、前処理済みの DataFrame を返す

使用ライブラリ- Python 3.10 - pandas 2.0 以上

入力- file_path: str — CSV ファイルのパス

出力- pd.DataFrame — 以下の前処理を適用済み 1. 列名を英語の小文字に統一 (スペースはアンダースコアに変換) 2. 数値列の欠損値は列の中央値で補完 3. 重複行を削除

エラー処理- ファイルが存在しない場合: FileNotFoundError を発生- CSV の解析に失敗した場合: ValueError を発生 (メッセージにファイル名を含む)

このように具体的な仕様を書くことで、AI はより正確で実用的なコードを生成できる。

AI が生成するコードの例:

```
1 import pandas as pd
2 import os
3
4
```

```
5 def load_experiment_data(file_path: str) -> pd.DataFrame:
6     """実験結果のファイルを読み込み、前処理済みのを返す。
7     CSVDataFrame
8
9     Parameters
10    -----
11    file_path : str ファイルのパス
12                CSV
13
14    Returns
15    -----
16    pd.DataFrame 前処理済みの
17                DataFrame
18
19    Raises
20    -----
21    FileNotFoundError ファイルが存在しない場合
22
23    ValueError の解析に失敗した場合
24                CSV
25    """
26    # ファイル存在確認
27    if not os.path.exists(file_path):
28        raise FileNotFoundError(
29            f"ファイルが見つかりません: {file_path}"
30        )
31
32    # 読み込みCSV
33    try:
34        df = pd.read_csv(file_path)
35    except Exception as e:
36        raise ValueError(
37            f"の解析に失敗しましたCSV ({file_path}): {e}"
38        )
```

```
39
40 # 1. 列名を英語の小文字に統一
41 df.columns = (
42     df.columns
43     .str.strip()
44     .str.lower()
45     .str.replace(" ", "_")
46 )
47
48 # 2. 数値列の欠損値を中央値で補完
49 numeric_cols = df.select_dtypes(include="number").
50     columns
51 for col in numeric_cols:
52     median_val = df[col].median()
53     df[col] = df[col].fillna(median_val)
54
55 # 3. 重複行を削除
56 df = df.drop_duplicates().reset_index(drop=True)
57
58 return df
```

6.3 主要ツールの活用法

6.3.1 Gemini Pro / ChatGPT / Claude でのコード生成

ブラウザベースの AI チャットツールは、最も手軽にコード生成を始められる方法である。

活用のコツ:

1. **新しいチャットで始める** — 過去の文脈が混ざらないよう、コード生成タスクは新しいセッションで行う
2. **コンテキストを最初に提供する** — 「私は大学院生で、心理学実験のデータを分析しています」のように背景を伝える

3. **段階的に依頼する** — 一度に全てを求めず、まず骨格を作り、次にエラー処理を追加するなど段階的に進める

プロンプト例 (Claude) :

プロンプト例

私は神経科学の大学院生です。脳波 (EEG) データの解析スクリプトを作成しています。

以下の関数を Python で作成してください。 - MNE-Python ライブラリを使用 - .fif 形式の EEG データを読み込む - バンドパスフィルタ (1-40Hz) を適用 - アーティファクト除去 (閾値: $\pm 100 \mu V$) を行う - エポック分割 (-0.2~0.8 秒、イベント ID 指定可能) を行う

各処理ステップにログ出力を含めてください。

6.3.2 GitHub Copilot の活用

GitHub Copilot は、VS Code 上でリアルタイムにコード補完を行うツールである。大学院生は **GitHub Education** を通じて無料で利用できる。

セットアップ手順:

1. GitHub Education (<https://education.github.com>) で学生認証を行う
2. VS Code の拡張機能から「GitHub Copilot」をインストール
3. GitHub アカウントでサインイン
4. .py ファイルを開いてコメントを書くと、自動的にコード候補が表示される

効果的な使い方:

```

1 # --- コメントで意図を書くと、が補完する Copilot ---
2
3 # ファイルを読み込み、欠損値の数を列ごとに表示する関数 CSV
4 def show_missing_values(file_path):
5     # ←ここでキーを押すと、がコードを提案する TabCopilot

```

Tips:

- コメントは英語でも日本語でも良いが、具体的に書くほど精度が上がる
- Alt +] で次の候補、Alt + [で前の候補を表示できる
- Copilot Chat（サイドパネル）を使えば、対話的にコードを改善できる

6.3.3 対話的コード開発のテクニック

AI との対話を通じてコードを段階的に改善する方法を、「対話的コード開発」と呼ぶ。以下にその流れを示す。

Step 1: 最初のプロンプト

プロンプト例

二群の平均値を比較する統計検定を行う Python 関数を書いてください。
scipy.stats を使用してください。

Step 2: 結果を見て追加指示

プロンプト例

ありがとうございます。以下の改善をお願いします：1. 正規性の検定（Shapiro-Wilk 検定）を先に行い、結果に応じて t 検定または Mann-Whitney U 検定を選択する 2. 効果量（Cohen's d）も計算する 3. 結果を辞書形式で返す

Step 3: さらに改善

プロンプト例

結果の辞書に以下も追加してください：- 検定名（使用した検定の名前） - 自由度（該当する場合） - 95%信頼区間

このように、段階的に要件を追加することで、自分の研究に最適なコードに近づけていく。

6.4 実践テクニック

6.4.1 段階的开发: 関数単位で生成・検証

AI にコード全体を一度に生成させるのではなく、**関数単位で生成し、一つずつ動作確認**する方法が最も効率的である。

手順:

1. まず、プログラム全体の構成を相談する
2. 各関数をつつ生成させる
3. 生成された関数を実行して動作確認する
4. 問題があれば修正を依頼する
5. 全関数が揃ったら統合テストを行う

プロンプト例:

プロンプト例

以下の研究データ分析パイプラインを設計したいです。まず、全体の構成（関数の一覧と役割）を提案してください。コードはまだ書かなくて良いです。

処理の流れ: 1. 複数の CSV ファイルを結合 2. データのクリーニング 3. 記述統計量の算出 4. 群間比較（統計検定） 5. 結果のグラフ作成 6. レポートの自動生成

6.4.2 テスト駆動: テストケースを先に書かせる

テスト駆動開発（TDD）の考え方を AI と組み合わせると、コードの品質向上が期待できる。

手順:

1. 関数の仕様を決める

2. AI にテストケースを先に生成させる
3. AI に関数本体を生成させる
4. テストを実行して合格を確認する

プロンプト例:

プロンプト例

以下の仕様の関数について、pytest のテストケースを先に書いてください。
関数本体はまだ書かないでください。

関数名: `calculate_effect_size` 入力: `group1 (list[float])`, `group2 (list[float])` 出力: `dict (cohen_d, interpretation を含む)` 仕様: - Cohen's `d` を計算する- 解釈: `—d— ; 0.2 → "negligible", ; 0.5 → "small", ; 0.8 → "medium", ; 0.8 → "large"` - 空リストが渡された場合は `ValueError` を発生

6.4.3 リファクタリング依頼

既存のコードを AI に改善させることも有用な活用法である。

プロンプト例:

プロンプト例

以下の Python コードをリファクタリングしてください。改善ポイント:

1. 可読性の向上 (変数名の改善、コメント追加)
2. 関数への分割 (1 関数 1 責任の原則)
3. エラー処理の追加
4. 型ヒントの追加

[既存のコードをここに貼り付ける]

6.4.4 デバッグ支援: エラーメッセージの活用

エラーに遭遇したとき、AI は頼りになるデバッグ支援ツールとなる。

プロンプト例:

プロンプト例

以下の Python コードを実行したところ、エラーが発生しました。原因と修正方法を教えてください。

```
## コード [コードを貼り付け]
## エラーメッセージ Traceback (most recent call
last): File "analysis.py", line 42, in module: result =
df.groupby("condition").apply(analyze_group) ValueError: unequal
length arrays
## 追加情報- Python 3.10, pandas 2.0, scipy 1.11 - データは条件ごと
に被験者数が異なります
```

Tips: エラーメッセージだけでなく、コード全体と実行環境の情報を含めることで、AI の回答精度が大幅に向上する。

6.5 テストコードの自動生成

6.5.1 pytest によるテストケース生成

テストコードの作成は、研究コードの信頼性を担保する上で極めて重要である。AI を活用すれば、包括的なテストケースを短時間で生成できる。

プロンプト例:**プロンプト例**

以下の Python 関数に対する pytest のテストケースを生成してください。
正常系、異常系、境界値テストを含めてください。
[関数のコードを貼り付け]

対象関数の例:

```
1 def normalize_scores(
2     scores: list[float],
3     method: str = "min-max"
```

```

4 ) -> list[float]:
5     """スコアのリストを正規化する。"""
6     if not scores:
7         raise ValueError("スコアリストが空です")
8     if method == "min-max":
9         min_val, max_val = min(scores), max(scores)
10        if min_val == max_val:
11            return [0.0] * len(scores)
12        return [(x - min_val) / (max_val - min_val)
13                for x in scores]
14    elif method == "z-score":
15        mean_val = sum(scores) / len(scores)
16        std_val = (sum((x - mean_val) ** 2
17                      for x in scores) / len(scores)) ** 0.5
18        if std_val == 0:
19            return [0.0] * len(scores)
20        return [(x - mean_val) / std_val for x in scores]
21    else:
22        raise ValueError(f"未対応の正規化方法: {method}")

```

AI が生成するテストコードの例:

```

1 import pytest
2 from normalize import normalize_scores
3
4
5 class TestNormalizeScoresMinMax:
6     """min-正規化のテストmax"""
7
8     def test_basic_normalization(self):
9         """基本的な正規化が正しく動作すること"""
10        result = normalize_scores([0, 50, 100])
11        assert result == [0.0, 0.5, 1.0]
12
13    def test_negative_values(self):

```

```
14         """負の値を含む場合"""
15         result = normalize_scores([-10, 0, 10])
16         assert result == [0.0, 0.5, 1.0]
17
18     def test_single_element(self):
19         """要素がつの場合 (1min == ) max"""
20         result = normalize_scores([42.0])
21         assert result == [0.0]
22
23     def test_all_same_values(self):
24         """全て同じ値の場合"""
25         result = normalize_scores([5.0, 5.0, 5.0])
26         assert result == [0.0, 0.0, 0.0]
27
28
29 class TestNormalizeScoresZScore:
30     """z-正規化のテスト score"""
31
32     def test_basic_z_normalization(self):
33         """基本的なz-正規化 score"""
34         result = normalize_scores(
35             [2, 4, 6], method="z-score"
36         )
37         assert pytest.approx(
38             result[1], abs=1e-6
39         ) == 0.0
40
41
42 class TestNormalizeScoresErrors:
43     """異常系テスト"""
44
45     def test_empty_list(self):
46         """空リストでValueError"""
47         with pytest.raises(
```

```

48         ValueError, match="スコアリストが空です"
49     ):
50         normalize_scores([])
51
52     def test_invalid_method(self):
53         """不正なメソッド名でValueError"""
54         with pytest.raises(
55             ValueError, match="未対応の正規化方法"
56         ):
57             normalize_scores([1, 2, 3], method="invalid")
58
59
60 class TestNormalizeScoresBoundary:
61     """境界値テスト"""
62
63     def test_very_large_values(self):
64         """非常に大きな値"""
65         result = normalize_scores([0, 1e10])
66         assert result == [0.0, 1.0]
67
68     def test_two_elements(self):
69         """要素がつの最小ケース2"""
70         result = normalize_scores([10, 20])
71         assert result == [0.0, 1.0]

```

6.5.2 テストカバレッジの考え方

テストカバレッジとは、テストがコードのどの程度をカバーしているかを示す指標である。

```

1 # カバレッジの測定
2 pip install pytest-cov
3 pytest --cov=my_module --cov-report=html tests/

```

研究コードでは、最低でも以下をカバーすることを目指す。

- **正常系:** 期待通りの入力に対する動作
- **異常系:** 不正な入力に対するエラー処理
- **境界値:** 最小値、最大値、空データ、要素 1 つなどの特殊なケース

6.5.3 AI 生成テストの検証ポイント

AI が生成したテストコードは、以下の点を必ず確認する。

1. **テストが実際に実行できるか** — インポートパスやファイル名が正しいか
2. **期待値が正しいか** — AI が計算した期待値が本当に合っているか手計算で確認する
3. **テストの独立性** — 各テストが他のテストに依存していないか
4. **意味のあるテストか** — `assert True` のような無意味なテストが含まれていないか

6.6 AI コードレビューとリファクタリング

6.6.1 AI にコードレビューを依頼するプロンプト

AI はコードレビューの補助ツールとしても活用できる。特に一人で研究を進める大学院生にとって、客観的なフィードバックを得る貴重な手段である。

プロンプト例:

プロンプト例

以下の Python コードをレビューしてください。以下の観点でチェックし、改善提案をお願いします。

1. ****可読性****: 変数名、関数名は適切か。コメントは十分か。2. ****効率性****: 不要なループや非効率な処理はないか。3. ****セキュリティ****: 入力検証は適切か。脆弱性はないか。4. ****エラー処理****: 例外処理は適切か。エッジケースへの対応は十分か。5. ****保守性****: コードの構造は適切か。将来の変更に対応しやすいか。

問題の深刻度を「重大」「中程度」「軽微」の3段階で分類してください。

[コードをここに貼り付け]

6.6.2 コードレビューチェックリスト

AI にレビューを依頼する際の観点をチェックリストとしてまとめる。

コードレビューチェックリスト

可読性

- 変数名・関数名が意図を表しているか
- 適切なコメント・docstring があるか
- コードの構造が論理的か

効率性

- 不要なループや計算がないか
- 適切なデータ構造を使っているか
- メモリ使用量は適切か

セキュリティ

- ユーザー入力を適切に検証しているか
- ファイルパスの検証があるか
- SQL インジェクション等の脆弱性がないか

エラー処理

- 想定されるエラーを適切にキャッチしているか
- エラーメッセージが分かりやすいか
- リソースの解放が確実に行われるか

テスト可能性

- 関数が適切な粒度に分割されているか
- 外部依存を注入可能か
- 副作用が最小限か

6.6.3 リファクタリングの種類と適用場面

6.6.4 ドキュメント自動生成

AI を使って、コードのドキュメントを自動生成できる。

docstring 生成プロンプト:

表 6.3 リファクタリングの種類と適用場面

種類	適用場面	プロンプト例
関数の抽出	長い関数を分割	「この関数を責務ごとに小さな関数に分割して」
変数名の改善	意味不明な変数名	「変数名をより意味が通るように改善して」
重複の排除	類似コードの繰り返し	「重複しているコードを共通関数にまとめて」
条件分岐の簡略化	複雑な if-else	「この条件分岐を簡潔に書き直して」
データ構造の変更	不適切な構造	「リストの代わりに辞書を使うように変更して」

プロンプト例

以下の Python 関数に NumPy スタイルの docstring を追加してください。
Parameters、Returns、Raises、Examples セクションを含めてください。
[関数のコードを貼り付け]

README 生成プロンプト:

プロンプト例

以下の Python プロジェクトの README.md を作成してください。
プロジェクト名: eeg-analysis-toolkit 概要: EEG データの前処理・分析
パイプライン主要ファイル: - preprocess.py: データ前処理- analyze.py: 統計分析- visualize.py: 結果の可視化- requirements.txt: 依存パッケージ
以下のセクションを含めてください: 1. プロジェクト概要 2. インストール方法 3. 使い方（コード例付き） 4. ファイル構成 5. ライセンス

6.7 AI 生成コードの品質と注意点

6.7.1 セキュリティリスク

AI 生成コードには、以下のようなセキュリティリスクが含まれる場合がある。

1. SQL インジェクション

```
1 # 危険: AI が生成しがちなコード
2 def get_user(name):
3     query = f"SELECT * FROM users WHERE name = '{name}'"
4     cursor.execute(query)
5
6 # 安全: パラメータ化クエリを使用
7 def get_user(name):
8     query = "SELECT * FROM users WHERE name = ?"
9     cursor.execute(query, (name,))
```

2. 不適切な入力検証

```
1 # 危険: ファイルパスを検証していない
2 def read_file(path):
3     with open(path) as f:
4         return f.read()
5
6 # 安全: パスの検証を含む
7 def read_file(path, allowed_dir="/data/experiments"):
8     real_path = os.path.realpath(path)
9     if not real_path.startswith(
10         os.path.realpath(allowed_dir)
11     ):
12         raise ValueError("許可されていないパスです")
13     with open(real_path) as f:
14         return f.read()
```

3. ハードコードされた秘密情報

```
1 # 危険: キーがコードに含まれているAPI
2 api_key = "sk-1234567890abcdef"
3
4 # 安全: 環境変数から読み込む
5 import os
```

```
6 api_key = os.environ.get("API_KEY")
7 if not api_key:
8     raise RuntimeError(
9         "API_KEY 環境変数が設定されていません"
10    )
```

6.7.2 ライセンス問題

AI が生成したコードには、学習データに含まれるオープンソースコードの断片が含まれる可能性がある。

- **研究コードの場合:** 論文の再現性のために公開する場合、ライセンスを明記する
- **推奨:** MIT ライセンスや Apache 2.0 ライセンスなど、許容的なライセンスを選択する
- **注意:** AI 生成コードの著作権は法的にグレーゾーンであることを認識しておく

6.7.3 「信頼するが検証する」の原則

AI 生成コードに対する基本姿勢は「Trust, but verify (信頼するが検証する)」である。

検証チェックリスト

- コードが意図通りに動作するか (手動テスト)
- テストケースが全て通過するか (自動テスト)
- エッジケースでも正しく動作するか
- エラー処理が適切か
- パフォーマンスが許容範囲内か
- セキュリティ上の問題がないか
- コードの論理が自分で理解できるか ← **最も重要**

最後の項目が最も重要である。**自分で理解できないコードは使わない**。AI に説明を求め、納得してから使用すること。

6.7.4 研究コードの再現性確保

研究で使用するコードは、再現性が欠かせない。以下の対策を講じる。

1. **依存関係の固定**: `requirements.txt` や `pyproject.toml` でバージョンを固定する
2. **乱数シードの設定**: 結果の再現性のために、乱数シードを明示的に設定する
3. **環境情報の記録**: Python バージョン、OS、主要ライブラリのバージョンを記録する
4. **バージョン管理**: Git でコードの履歴を管理する（第 8 章で詳述）

```
1 # 再現性確保のテンプレート
2 import random
3 import numpy as np
4
5 RANDOM_SEED = 42
6 random.seed(RANDOM_SEED)
7 np.random.seed(RANDOM_SEED)
8
9 # 実行環境の記録
10 import sys
11 import platform
12 print(f"Python: {sys.version}")
13 print(f"NumPy: {np.__version__}")
14 print(f"OS: {platform.platform()}")
```

演習問題

1. 演習 6-1: 基本的なコード生成

以下の仕様で、AI を使って Python 関数を生成しなさい。生成されたコードを実行し、正しく動作することを確認しなさい。

仕様:

- 関数名: `summarize_data`
- 入力: pandas の `DataFrame`
- 出力: 各数値列について、平均、中央値、標準偏差、最小値、最大値を含む辞書
- エラー処理: `DataFrame` が空の場合は `ValueError` を発生

使用したプロンプトと、生成されたコードを提出すること。

2. 演習 6-2: テスト駆動開発

演習 6-1 で作成した `summarize_data` 関数に対して、AI を使って `pytest` のテストケースを生成しなさい。以下のケースを最低限含めること。

- 正常な `DataFrame` に対するテスト
- 空の `DataFrame` に対するテスト
- 数値列のない `DataFrame` に対するテスト
- 欠損値を含む `DataFrame` に対するテスト

3. 演習 6-3: コードレビューとリファクタリング

以下のコードを AI にレビューさせ、改善版を作成しなさい。どのような問題が指摘され、どう改善されたかをレポートにまとめること。

```
1 def proc(d):
2     r = []
3     for i in range(len(d)):
4         if d[i] != None and d[i] > 0:
5             x = d[i] * 2.5 + 10
6             r.append(x)
7
8     s = 0
9     for j in r:
10         s = s + j
11
12     a = s / len(r)
```

```
11     return a
```

4. 演習 6-4: デバッグ支援

以下のコードにはバグが含まれている。AI を使ってバグを特定し、修正しなさい。修正の過程で使用したプロンプトも提出すること。

```
1  import pandas as pd
2
3  def calculate_growth_rate(df, value_col, time_col):
4      """時系列データの成長率を計算する"""
5      df = df.sort_values(time_col)
6      growth_rates = []
7      for i in range(len(df)):
8          current = df.iloc[i][value_col]
9          previous = df.iloc[i-1][value_col]
10         rate = (current - previous) / previous * 100
11         growth_rates.append(rate)
12     df["growth_rate"] = growth_rates
13     return df
14
15 # テストデータ
16 data = pd.DataFrame({
17     "year": [2020, 2021, 2022, 2023],
18     "revenue": [100, 120, 150, 180]
19 })
20
21 result = calculate_growth_rate(data, "revenue", "year
22     ")
23 print(result)
```

5. 演習 6-5: 研究データ分析スクリプトの作成

自分の研究テーマ（または以下のサンプルテーマ）に関連するデータ分析スクリプトを AI と対話的に作成しなさい。

サンプルテーマ: 「オンライン授業と対面授業の学習効果比較」

以下の機能を含むスクリプトを作成すること。

- (a) サンプルデータの生成（被験者 50 名、2 群、テストスコアあり）
- (b) 記述統計量の算出
- (c) 適切な統計検定の実施
- (d) 結果のグラフ作成

AI との対話履歴（プロンプトとレスポンスの要約）、最終的なコード、実行結果を提出すること。

本章のまとめ

AI は頼りになるコーディングパートナーであるが、最終的な品質責任は使用者にある。「AI に書かせて、人間が検証する」という姿勢を常に持ち、特に研究コードでは再現性とセキュリティに十分注意すること。プロンプトの書き方一つでコードの品質が大きく変わるため、明確で具体的な仕様記述のスキルを磨くことが重要である。

引用・参考文献

7

データ分析と可視化

【この章で学ぶこと】

- AIを活用したデータ分析ワークフロー（前処理→分析→可視化）を実践できる
- 論文品質のグラフを効率的に作成できる
- 統計的検定の基礎と適切な手法選択を理解する

7.1 研究におけるデータ分析

7.1.1 なぜデータ分析が重要か

大学院での研究において、データ分析は以下の場面で欠かせない。

1. **実験結果の解釈:** 収集したデータから意味のあるパターンを見出す
2. **仮説の検証:** 統計的手法により、仮説が支持されるかを客観的に判断する
3. **論文の説得力:** 適切な分析と可視化は、論文の査読者を説得する論文の説得力を大きく左右する
4. **新たな発見:** 探索的データ分析（EDA）により、予想外の知見が得られることがある

データ分析のスキルは、理系・文系を問わず、現代の大学院生に求められる基本能力である。心理学の実験データ、社会調査の回答データ、生物学の測定

データ、工学のシミュレーション結果——あらゆる研究分野でデータ分析が行われている。

7.1.2 AI がデータ分析をどう変えるか

従来、データ分析を行うには、プログラミング言語（Python, R 等）の習得に多くの時間を要した。AI の登場により、このハードルは大幅に下がっている。

AI が特に役立つ場面

- **分析コードの生成:** 「このデータで t 検定を行いたい」と伝えるだけで、コードが得られる
- **手法の選択支援:** データの特性を伝え、適切な統計手法を提案してくれる
- **グラフの作成:** 「論文に載せる棒グラフを作って」という指示で、出版品質のグラフコードが得られる
- **結果の解釈:** 統計結果の意味を平易な言葉で説明してもらえる

ただし、AI はあくまで道具であり、分析の設計と結果の解釈は研究者の責任である。本章では、AI の力を正しく活用しつつ、データ分析の基礎力を養うことを目指す。

7.2 データ分析ワークフロー

7.2.1 Step 1: データの理解と前処理

データ分析の第一歩は、手元のデータを正しく理解することである。

プロンプト例

以下の CSV ファイルを読み込み、データの概要を把握する Python スクリプトを作成してください。

ファイル: experiment_results.csv

使用ライブラリ: pandas

以下の情報を出力してください:

1. データの形状 (行数×列数)
2. 各列のデータ型
3. 最初の 5 行の表示
4. 基本統計量 (数値列: 平均・標準偏差・最小・最大、
カテゴリ列: ユニーク数・最頻値)
5. 欠損値の数と割合 (列ごと)
6. 各数値列のヒストグラム

AI が生成するコードの例:

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5
6 def explore_data(file_path: str) -> pd.DataFrame:
7     """ファイルを読み込み、データの概要を表示する CSV"""
8
9     # データ読み込み
10    df = pd.read_csv(file_path)
11
12    # 1. データの形状
13    print("=" * 50)
14    print(f"データの形状: {df.shape}行[0] × {df.shape}列[1]")
15    print("=" * 50)
```

```
16
17 # 2. 各列のデータ型
18 print("\ 【データ型】 n")
19 print(df.dtypes.to_string())
20
21 # 3. 最初の行5
22 print("\ 【最初の行】 n5")
23 print(df.head().to_string())
24
25 # 4. 基本統計量
26 print("\ 【基本統計量 (数値列)】 n")
27 print(df.describe().to_string())
28
29 # 5. 欠損値
30 print("\ 【欠損値】 n")
31 missing = df.isnull().sum()
32 missing_pct = (missing / len(df) * 100).round(2)
33 missing_df = pd.DataFrame({
34     "欠損数": missing,
35     "欠損率(%)" : missing_pct
36 })
37 print(missing_df[missing_df["欠損
38     数"] > 0].to_string())
39 if missing.sum() == 0:
40     print(" 欠損値なし ")
41
42 # 6. ヒストグラム
43 num_cols = df.select_dtypes(include=[np.number]).
44     columns
45 if len(num_cols) > 0:
46     fig, axes = plt.subplots(
47         1, len(num_cols),
48         figsize=(5 * len(num_cols), 4))
49     if len(num_cols) == 1:
```

```

48         axes = [axes]
49     for ax, col in zip(axes, num_cols):
50         df[col].hist(ax=ax, bins=20, edgecolor="black",
51                       ")
52         ax.set_title(col)
53         ax.set_xlabel("値")
54         ax.set_ylabel("頻度")
55     plt.tight_layout()
56     plt.savefig("data_overview_histograms.png", dpi
57               =150)
58     plt.show()

return df

```

欠損値処理の主な方法

方法	適用場面	Python コード
行の削除	欠損が少ない場合	<code>df.dropna()</code>
平均値補完	正規分布に近いデータ	<code>df.fillna(df.mean())</code>
中央値補完	外れ値がある場合	<code>df.fillna(df.median())</code>
前方補完	時系列データ	<code>df.ffill()</code>
多重代入法	欠損が多い場合	<code>IterativeImputer</code>

プロンプト例

以下の DataFrame の数値列について、外れ値を検出するコードを作成してください。

外れ値の定義: IQR 法 ($Q1 - 1.5 \times IQR$ 未満、

または $Q3 + 1.5 \times IQR$ 超過)

外れ値がある行を表示し、箱ひげ図で可視化してください。

7.2.2 Step 2: 探索的データ分析 (EDA)

探索的データ分析 (Exploratory Data Analysis, EDA) は、データの構造や

パターンを視覚的に把握するプロセスである。

プロンプト例

以下の DataFrame について、探索的データ分析（EDA）を行う Python コードを作成してください。

分析内容:

1. 数値列間の相関行列の計算とヒートマップの作成
2. 各数値列の分布（ヒストグラム + 正規分布フィット）
3. カテゴリ列ごとのグループ別統計量
4. 主要な変数間の散布図マトリクス

使用ライブラリ: pandas, matplotlib, seaborn

グラフは日本語フォントに対応させてください。

7.2.3 Step 3: 統計的検定

研究の仮説を検証するためには、適切な統計的検定を行う必要がある。以下に、よく使われる検定手法とその選び方を示す。

統計的検定の選び方フローチャート

データは正規分布に従うか？

└— はい → 2 群比較か？

| └— はい → 対応のあるデータか？

| | └— はい → 対応のある t 検定

| | └— いいえ → 独立 2 標本 t 検定

| └— いいえ → 3 群以上の比較

| └— 対応あり → 反復測定分散分析

| └— 対応なし → 一元配置分散分析

└— いいえ → 2 群比較か？

└— はい → 対応のあるデータか？

| └— はい → Wilcoxon 符号順位検定

| └— いいえ → Mann-Whitney U 検定

└— いいえ → Kruskal-Wallis 検定

カテゴリデータ同士 → カイ二乗検定

プロンプト例

以下のデータについて、2 群間の平均値の差を検定する

Python コードを作成してください。

データ構造:

- DataFrame に “group” 列 (“control” と “treatment”) と “score” 列がある
- 各群の被験者数は異なる可能性がある

処理手順:

1. 各群のデータの正規性を Shapiro-Wilk 検定で確認
2. 等分散性を Levene 検定で確認
3. 結果に応じて適切な検定を自動選択
4. 効果量 (Cohen’s d または r) を計算
5. 結果を分かりやすくレポート出力

p 値の正しい解釈

誤った解釈	正しい解釈
$p = 0.03$ は「差がある確率が 97%」	帰無仮説の下で、観測データ以上に極端な結果が得られる確率が 3%
$p < 0.05$ は「重要な差がある」	統計的に有意だが、実的な重要性は効果量で判断
$p = 0.06$ は「差がない」	有意水準を超えただけで、差がないとは限らない
p 値が小さいほど効果が大きい	p 値はサンプルサイズの影響を受ける。効果量を見るべき

7.3 論文品質のグラフ作成

7.3.1 matplotlib / seaborn の基本

Python でグラフを作成するための 2 大ライブラリが matplotlib と seaborn

である。

- **matplotlib**: 低レベルの描画ライブラリ。細かいカスタマイズが可能
- **seaborn**: matplotlib をベースにした高レベルの可視化ライブラリ。統計的な可視化に特化し、美しいデフォルト設定を持つ

7.3.2 効果的なグラフの選び方

データの性質	推奨グラフ	用途
カテゴリ別の量的比較	棒グラフ（エラーバー付き）	群間比較の結果表示
2 変数の関係	散布図	相関関係の可視化
分布の比較	箱ひげ図 / バイオリンプロット	群間の分布の違い
変数間の関連	ヒートマップ	相関行列の可視化
時系列データ	折れ線グラフ	時間変化の表示
割合・構成比	積み上げ棒グラフ	カテゴリの内訳表示

7.3.3 論文に必要なグラフの要素

学術論文や学会発表のグラフには、以下の要素が不可欠である。

必須要素

- 明確な軸ラベル（単位を含む）
- 適切なタイトル（Figure 1: ... の形式）
- 凡例（複数系列の場合）
- エラーバー（平均値を示す場合、標準誤差または 95%信頼区間）
- 適切なフォントサイズ（本文と同程度、最低 8pt）
- 高解像度（300dpi 以上）

7.3.4 白黒印刷対応のグラフ設計

多くの学術論文は白黒で印刷されることがあるため、色だけに頼らないデザインが重要である。

1. **ハッチパターン**: 棒グラフにストライプや格子模様を加える
2. **マーカーの形状**: 散布図で○、△、□など異なるマーカーを使う
3. **線のスタイル**: 折れ線グラフで実線、破線、点線を使い分ける
4. **コントラスト**: 色を使う場合も、白黒にしたときに区別できるコントラストを確保する

7.3.5 AIでグラフ作成コードを生成するプロンプト例

プロンプト例

以下の仕様で論文品質の棒グラフを作成する

Python コードを書いてください。

データ:

- 3 群 (Control, Treatment A, Treatment B)
- 各群の平均値と標準誤差がある

グラフの要件:

- matplotlib + seaborn を使用
- 白背景、枠線あり
- エラーバー (標準誤差) 付き
- 各棒に異なるハッチパターン (白黒印刷対応)
- 有意差を示すブラケットと * マーク
- フォントサイズ: タイトル 14pt、軸ラベル 12pt、目盛り 10pt
- 解像度 300dpi で PNG 保存
- 図のサイズ: 幅 8cm × 高さ 6cm (単カラム論文用)

7.4 インタラクティブ可視化

7.4.1 Plotly の紹介

Plotly は、インタラクティブなグラフを作成するライブラリである。マウスオーバーで値を表示したり、ズーム・パンができるため、データの探索段階で特に有用である。

```
1 import plotly.express as px
2
3 # インタラクティブ散布図
4 fig = px.scatter(
5     df, x="variable_a", y="variable_b",
6     color="group",
7     hover_data=["subject_id", "age", "score"],
8     title="変数と変数の関係AB",
9     labels={
10         "variable_a": "変数A",
11         "variable_b": "変数B"
12     }
13 )
14 fig.update_layout(
15     font=dict(size=14),
16     hoverlabel=dict(font_size=12)
17 )
18 fig.write_html("interactive_scatter.html")
19 fig.show()
```

7.4.2 データ探索での活用

インタラクティブ可視化は、以下の場面で特に有用である。

- **外れ値の特定**: マウスオーバーで外れ値の詳細情報を確認できる
- **パターンの発見**: ズーム機能で特定の領域を拡大して観察できる

- **プレゼンテーション:** ゼミ発表でリアルタイムにデータを探索できる
- **共同研究者との共有:** HTML ファイルを送るだけで、相手もインタラクティブに操作できる

ただし、論文に掲載するグラフは静的な画像（matplotlib 等）で作成すること。インタラクティブ可視化はあくまで探索・確認用である。

7.5 AI を使ったデータ分析の落とし穴

7.5.1 統計的な誤解

AI がデータ分析を手軽にする一方で、統計的な誤解に基づく分析が行われるリスクも増大している。

p-hacking (p 値ハッキング)

多数の検定を繰り返し、有意な結果が出たものだけを報告する行為。AI を使うと、様々な分析を手軽に試せるため、このリスクが高まる。

有意水準 5% で 20 回検定すれば、本当は差がなくても平均 1 回は「有意」になる（偽陽性）。

多重比較問題

複数の群を同時に比較する場合、検定の回数が増えるほど偽陽性の確率が高くなる。3 群を比較する場合は Bonferroni 補正（補正後の有意水準: $0.05/3 = 0.0167$ ）や Tukey HSD などの多重比較法を用いる。

7.5.2 過学習への注意

AI が提案する分析手法が必要以上に複雑になることがある。特に機械学習を使う場合、過学習（overfitting）のリスクに注意が必要である。

対策:

- 交差検証（Cross-validation）を必ず行う

- 単純なモデルから始め、必要な場合にのみ複雑化する
- サンプルサイズに見合った分析手法を選ぶ

7.5.3 AIが出した統計結果を鵜呑みにしない

AIが生成した分析コードは、一見正しく見えても論理的な誤りを含むことがある。

自分で確認すべきポイント

- 検定の前提条件は満たされているか（正規性、等分散性）
- 実験計画（対応あり/なし）に合った検定を使っているか
- 有意水準は事前に設定されているか
- 効果量も報告しているか
- 多重比較を行う場合、補正は適切か
- サンプルサイズは十分か
- 結果の解釈は論理的か

7.6 実例：AIを用いたアンケート調査データの統計分析

本節では、著者自身がAIを用いて統計分析と論文執筆を行った実例を紹介する。この事例は、本章で解説した手法（ノンパラメトリック検定、多重比較補正、効果量、論文品質のグラフ作成）を実際の研究で適用したものであり、AIとの協働の利点と落とし穴の両方を具体的に示す。

7.6.1 研究の概要

この研究は、日本の自動車産業における安全規格（ISO 26262、SOTIF、UL 4600）の採用状況を調査したものである。JASPAR（Japan Automotive Software Platform and Architecture）の機能安全ワーキンググループに参加する40社を対象にアンケート調査を実施し、30件の回答（回収率75%）を得た。

データは5段階リカート尺度で収集し、ノンパラメトリック検定で分析した。この研究は SafeComp 2026 に投稿された。

7.6.2 AI との協働の実際：コミット履歴から

この研究では、統計分析と論文執筆の全過程を Git で管理した。コミット履歴を振り返ると、AI との協働がどのように進んだかが明確に見える。

(1) Phase 1：データ投入と初期分析 (2026 年 1 月 14 日、8 コミット)

Excel データをリポジトリに追加し、AI に Python による統計分析を依頼した。AI は `scipy.stats` を使った Wilcoxon 検定や Mann-Whitney U 検定のコードを生成し、最初の分析レポートを約 2 時間で作成した。この段階では、記述統計、検定結果、考察までを含む統合レポートが生成された。

ただし、この初期レポートには複数の問題があった。

(2) Phase 2：統計手法の修正サイクル (1 月 15 日～21 日、15 コミット) 初期分析の結果を著者が精査したところ、AI が生成した統計処理にいくつかの誤りが見つかった。以下はコミット履歴から抽出した主な修正である。

表 7.1 AI の統計分析で見つかった誤りと修正 (コミット履歴より)

日付	AI の誤り	人間による修正
1/15	効果量 r の計算で全標本数 n を使用	Rosenthal ¹⁾ に基づき、 n_{nonzero} (ゼロ差分を除いた標本数) を使用するよう修正
1/20	小標本 ($n < 20$) で正規近似の p 値を使用	<code>exact</code> 検定に切り替え (<code>scipy.stats.wilcoxon</code> の <code>method='exact'</code>)
1/20	Satisfaction Gap をペア検定で確認したかのように記述	対応のないデータであることを明記し、記述的観察として再解釈
1/21	Mann-Whitney U 検定の効果量でタイ (同順位) を未考慮	タイ補正付き分散で Z 値と効果量 r を再計算
1/21	$n_{\text{nonzero}} < 10$ の場合も効果量 r を報告	標本が少なすぎる場合は r を報告しないルールを追加

この Phase 2 だけで 15 回のコミットが発生した。コミットメッセージの半数以上が「Fix」「Correct」「Revert」で始まっており、AI の出力を検証・修正するプロセスの密度がわかる。

教訓： AI は「もっともらしい統計分析」を生成するが、細部で間違える。特に (1) 効果量の計算方法、(2) 小標本での検定手法の選択、(3) データの対応関係の解釈で誤りが多かった。統計の教科書的知識がなければ、これらの誤りは見逃されていただろう。

(3) **Phase 3：論文執筆と推敲 (3月7日～14日、20コミット)** 分析結果を英語論文にまとめる段階では、AI に以下を依頼した。

- 分析レポート (日本語) から英語論文への変換
- 英語表現の改善 (非母語話者向けの読みやすさ向上)
- 非統計学者にも理解しやすい表現への書き換え
- 図の配色統一 (Fig. 1～3 で一貫したカラーパレット)

この段階でも AI の出力をそのまま採用することはなかった。たとえば、AI が生成した統計報告のうち、Holm 補正ファミリーのサイズが不正確な箇所 (RQ3 の p 値) を著者が発見し修正した。コミットメッセージ「Fix statistical reporting: restore RQ3 Holm family size, correct A-SPICE p -value claim」がその記録である。

7.6.3 AI が間違えた具体例

ここでは、特に教訓的な誤りを 2 つ取り上げる。

(a) **例 1：効果量の計算方法** AI は当初、Wilcoxon 符号順位検定の効果量 r を次のように計算した。

$$r = \frac{Z}{\sqrt{n}} \quad (\text{AI の計算})$$

ここで n は全標本数である。しかし、Wilcoxon 検定では差分がゼロのペアは除外されるため、正しくは非ゼロ差分の標本数 n_{nonzero} を用いるべきである。

$$r = \frac{Z}{\sqrt{n_{\text{nonzero}}}} \quad (\text{修正後})$$

この修正は小さいように見えるが、 n_{nonzero} が n より小さい場合（同順位が多いリカートデータでは頻繁に起こる）、効果量が過小評価される。

(b) **例2：小標本での検定手法** AIは`scipy.stats.wilcoxon`のデフォルト設定（正規近似）を使ったコードを生成した。しかし、 $n < 20$ 程度の小標本では正規近似は不正確であり、exact 検定を用いるべきである。著者が p 値の妥当性を手計算で確認したことでこの問題が発覚した。

7.6.4 この事例から学べること

1. **AI は下書き、人間が品質保証**：AI は 2 時間で統合分析レポートを生成したが、その後の検証・修正に 1 週間以上かかった。「速さ」と「正確さ」は別である。
2. **バージョン管理が「検証の証拠」になる**：Git のコミット履歴が、「AI の出力をいつ、どのように検証・修正したか」の記録となる。論文の再現性と透明性を担保する重要な手段である。
3. **統計の基礎知識は代替不可能**：AI が効果量の計算を間違えたとき、それを発見できたのは著者に統計の知識があったからである。AI は「偶有的複雑さ」（コードの記述）を効率化するが、「本質的複雑さ」（適切な手法の選択と結果の解釈）は人間の役割である。
4. **分析 $v3 \rightarrow v4 \rightarrow v5 \rightarrow v6 \rightarrow v7$ の反復**：最初の分析で完成することはない。AI との対話を通じて段階的に改善していくプロセスこそが、AI 活用の実態である。

ヒント：この事例は、著者の研究リポジトリのコミット履歴で全過程を追跡できる。読者も自分の研究で Git を使えば、AI との協働のプロセスを同様に記録・検証できる。第 8 章で解説する Git の活用は、AI 時代の研究では単なるコード管理を超えた意味を持つ。

演習問題

演習 7-1 データの前処理と概要把握

サンプルデータを生成し、AI を使ってデータの概要把握と前処理を行いなさい。データの概要把握（形状、型、分布、欠損値）、外れ値の検出と対処、欠損値の処理、処理前後の比較を実施し、レポートにまとめなさい。

演習 7-2 探索的データ分析

演習 7-1 のデータについて、グループごとの記述統計量、相関分析、各変数の分布の可視化を実施し、発見した特徴やパターンを 3 つ以上報告しなさい。

演習 7-3 適切な統計検定の実施

「`treatment` 群は `control` 群と比べて、`pre_score` から `post_score` への変化量が多い」という仮説を検定しなさい。正規性の検定、適切な統計検定の選択と実施、効果量の計算を行い、結果を論文の「結果」セクションの書き方で報告しなさい。

演習 7-4 論文品質のグラフ作成

演習 7-3 の結果を以下のグラフで可視化しなさい。全てのグラフは論文品質（300dpi、適切なフォントサイズ、軸ラベル）で作成すること。(1) 群ごとの `pre/post` スコアの棒グラフ（エラーバー付き）、(2) 群ごとの変化量の箱ひげ図（個別データ点付き）、(3) `pre_score` と `post_score` の散布図（群ごとに色分け、回帰直線付き）。

演習 7-5 総合分析レポート

自分の研究テーマについて、データ分析の全工程を実施し、レポートにまとめなさい。サンプルデータの生成、前処理と EDA、適切な統計分析、論文品質のグラフ 3 枚以上、結果のまとめ（発見した知見を 3 つ以上）を含めること。AI との対話履歴の要約も合わせて提出すること。

引用・参考文献

- 1) Robert Rosenthal. *Meta-Analytic Procedures for Social Research*. Sage Publications, revised edition, 1991.

8

GitHub による研究成果管理

【この章で学ぶこと】

- Git/GitHub の基本操作を習得する
- AI を活用した Web ページ作成と GitHub Pages での公開を実践できる
- 研究ポートフォリオの構築と活用方法を理解する

8.1 なぜ大学院生が GitHub を使うべきか

8.1.1 研究コードのバージョン管理

大学院での研究では、コードを何度も修正する。バージョン管理なしでは、以下のような問題が生じる。

よくある悲劇:

`analysis.py`

`analysis_v2.py`

`analysis_v2_final.py`

`analysis_v2_final_revised.py`

`analysis_v2_final_revised_REAL_FINAL.py`

このようなファイル管理では、「どの時点で何を変更したか」「なぜ変更したか」が全く分からなくなる。Git を使えば、コミットした変更の履歴を追跡でき、いつでも過去の状態に戻れる。

8.1.2 共同研究での活用

研究は一人で行うものではない。GitHub を使えば、次のようなことができる。

- **コードの共有:** URL を送るだけで共同研究者がコードにアクセスできる
- **同時作業:** 複数人が同じプロジェクトを同時に編集できる
- **変更の追跡:** 誰が、いつ、何を変更したかが一目で分かる
- **レビュー:** 共同研究者の変更をレビューしてからマージできる

8.1.3 就職活動でのポートフォリオ効果

技術系の就職活動において、GitHub のプロフィールは「第二の履歴書」になる。特に IT 企業やデータサイエンス関連のポジションでは、GitHub での活動が評価される。

GitHub プロフィールで評価されるポイント

- 定期的なコミット（継続的に開発・研究している証拠）
- README.md が整備されたりポジトリ（ドキュメンテーション能力）
- コードの品質（命名規則、コメント、テストの有無）
- オープンソースプロジェクトへの貢献

8.1.4 オープンサイエンスへの貢献

近年、研究のオープン化（Open Science）が世界的なトレンドとなっている。論文のデータや分析コードを GitHub で公開することで、以下のメリットがある。

- **研究の透明性向上:** 分析手法が第三者に検証可能になる

- **再現性の担保:** 他の研究者が同じ分析を再現できる
- **引用の増加:** 公開されたコードやデータは他の研究者に利用され、引用が増える傾向がある
- **学術コミュニティへの貢献:** 研究ツールやデータセットの共有

8.2 Git の 基 礎

8.2.1 バージョン管理とは何か

バージョン管理とは、ファイルの変更履歴を記録し、過去の任意の時点の状態を復元できるようにする仕組みである。

8.2.2 リポジトリ、コミット、ブランチの基本概念

Git を理解するための 3 つの核心概念を、ゲームのアナロジーで説明する。

Git の基本概念

リポジトリ (Repository) = セーブデータフォルダ

プロジェクトの全ファイルとその変更履歴を格納する場所である。

コミット (Commit) = セーブポイント

ある時点でのファイルの状態を保存する操作である。各コミットには「何を変更したか」を説明するメッセージを付ける。

ブランチ (Branch) = 並行世界

メインの開発ラインから分岐して、独立に作業できる仕組みである。

表 8.1 Git 用語とゲームのアナロジー

Git 用語	ゲームのアナロジー	説明
リポジトリ	セーブデータフォルダ	プロジェクト全体の格納場所
コミット	セーブポイント	ある時点の状態を記録
ブランチ	並行ルート	分岐して独立に作業
マージ	ルートの合流	分岐した作業を統合
プッシュ	クラウドに同期	ローカルの変更を GitHub に送る
プル	クラウドから同期	GitHub の変更をローカルに取得
クローン	セーブデータをコピー	GitHub からプロジェクトをダウンロード

8.3 GitHub の基本操作

8.3.1 GitHub アカウントの作成

1. <https://github.com> にアクセスする
2. 「Sign up」をクリックし、メールアドレス・ユーザー名・パスワードを入力する
3. メール認証を完了する

ユーザー名は本名またはそれに近い名前が望ましい（就職活動で使うため）。

GitHub Education (<https://education.github.com>) に大学のメールアドレスで学生認証を行うと、GitHub Copilot が無料で使えるなどの特典がある。

8.3.2 GitHub Desktop のセットアップ

コマンドラインが苦手な人には、GitHub Desktop (GUI アプリ) が推奨される。<https://desktop.github.com> からダウンロードし、GitHub アカウントでサインインする。

8.3.3 リポジトリの作成とクローン

GitHub 上でリポジトリを作成する手順:

1. GitHub にログイン
2. 右上の「+」→「New repository」をクリック
3. Repository name、Description を入力し、Public または Private を選択
4. 「Add a README file」にチェックし、「Create repository」をクリック

8.3.4 コミット→プッシュの基本フロー

日常的な作業の流れは以下の通りである。

1. ファイルを編集する (VS Code 等で)
2. 変更をステージングする (保存したい変更を選択)
3. コミットメッセージを書く
4. コミットする (ローカルにセーブ)
5. プッシュする (GitHub にアップロード)

良いコミットメッセージの書き方

悪い例	良い例
「更新」	「t 検定の結果を棒グラフで可視化」
「修正」	「欠損値処理のバグを修正: NaN が残る問題」
「追加」	「相関分析のコードとヒートマップを追加」

コミットメッセージは「何を、なぜ変更したか」が分かるように書く。

8.3.5 VS Code との連携

VS Code には Git の操作機能が組み込まれている。左サイドバーの「ソース管理」アイコンからファイルのステージング、コミット、プッシュが可能である。拡張機能として **GitLens** (コードの各行の変更履歴表示) や **Git Graph**

(コミット履歴のグラフ表示) の導入を推奨する。

8.4 AI で Web ページを作成する

8.4.1 AI で HTML/CSS コードを生成する

AI を使えば、プログラミングの知識がなくても自己紹介 Web ページを作成できる。

プロンプト例

大学院生の自己紹介 Web ページを作成する HTML/CSS コードを書いてください。

内容:

- 名前: 田中太郎
- 所属: ○○大学大学院 情報科学研究科 修士 2 年
- 研究テーマ: 「深層学習を用いた自然言語処理の研究」
- スキル: Python, PyTorch, 統計分析

デザイン要件:

- モダンでシンプルなデザイン
- レスポンシブ対応 (スマホでも見やすい)
- セクション: ヘッダー、自己紹介、研究概要、業績、スキル、連絡先
- CSS は同一ファイルに含める (1 ファイルで完結)

8.4.2 VS Code での編集とプレビュー

生成された HTML を VS Code で編集する手順は以下の通りである。

1. VS Code でプロジェクトフォルダを開く
2. `index.html` という名前でファイルを作成
3. AI が生成したコードを貼り付ける
4. 拡張機能「Live Server」をインストール

5. `index.html` を右クリック→「Open with Live Server」
6. ブラウザでリアルタイムにプレビューが表示される

8.4.3 AI でカスタマイズする

Web ページの修正やカスタマイズも AI に依頼できる。デザインの変更やセクションの追加など、自然言語で指示するだけで対応してもらえる。

8.5 GitHub Pages で公開する

8.5.1 GitHub Pages とは

GitHub Pages は、GitHub のリポジトリに置いた HTML ファイルを無料で Web サイトとして公開できるサービスである。

- 無料で利用可能
- `https://ユーザー名.github.io/リポジトリ名/` の URL で公開される
- リポジトリにプッシュするだけで自動的に更新される
- カスタムドメインも設定可能

8.5.2 GitHub Pages の設定手順

1. GitHub で新しいリポジトリを作成する (Public に設定)
2. リポジトリに `index.html` をアップロードする
3. リポジトリの「Settings」→「Pages」をクリック
4. 「Source」で Branch: `main`、Folder: `/ (root)` を設定し「Save」
5. 数分後に `https://ユーザー名.github.io/リポジトリ名/` で公開される

8.5.3 更新フロー

ローカルで `index.html` を編集し、Live Server でプレビュー確認した後、GitHub Desktop でコミット・プッシュすると、数分後に Web サイトに反映

される。

8.6 研究ポートフォリオの構築

8.6.1 ポートフォリオに含めるもの

セクション	内容	重要度
自己紹介	名前、所属、研究分野、顔写真	必須
研究概要	研究テーマの説明（一般向け）	必須
業績一覧	論文、学会発表、受賞歴	必須
スキル	プログラミング言語、ツール、資格	推奨
プロジェクト	研究プロジェクトの詳細と成果物	推奨
連絡先	メールアドレス、SNS	必須
CV/履歴書	PDF 形式でダウンロード可能	推奨

8.6.2 README.md の書き方

README.md は、リポジトリの「顔」である。GitHub でリポジトリを開いたとき最初に表示されるため、丁寧に書くことが重要である。プロジェクト概要、特徴、インストール方法、使い方、ファイル構成、依存ライブラリ、ライセンス、著者情報を含めるとよい。

8.6.3 フォルダ構成のベストプラクティス

my-research-project/

```

├── README.md
├── LICENSE
├── requirements.txt
├── .gitignore
├── src/                # ソースコード
├── data/
│   ├── raw/           # 生データ（変更不可）
│   └── processed/     # 処理済みデータ

```

— notebooks/	# Jupyter Notebook
— figures/	# 生成されたグラフ
— tests/	# テストコード
— docs/	# ドキュメント

.gitignore ファイル

.gitignore は、Git で追跡すべきでないファイルを指定するファイルである。大きなデータファイル、Python のキャッシュ (`__pycache__`/)、環境変数 (`.env`)、OS 固有ファイル (`.DS_Store`)などを指定する。

8.7 共同研究での GitHub 活用

8.7.1 Issue による課題管理

Issue は、プロジェクトの課題やタスクを管理する機能である。タイトルと説明を記入し、ラベル (bug, enhancement, question 等) を付与し、担当者を指定する。研究プロジェクトでは、データ前処理、統計分析、図表作成などのタスクを Issue として管理できる。

8.7.2 Pull Request によるコードレビュー

Pull Request (PR) は、ブランチの変更をメインブランチに統合する前に、レビューを行う仕組みである。

1. 新しいブランチを作成する
2. ブランチ上でコードを書いてコミットする
3. GitHub 上で Pull Request を作成する
4. 共同研究者がコードをレビューする
5. レビューが通ったらマージする

8.7.3 研究コードの共有とライセンス

研究コードを公開する場合、適切なライセンスを選択することが重要である。

ライセンス	特徴	推奨場面
MIT	最も自由。商用利用・改変・再配布可能	研究コードの一般公開
Apache 2.0	MIT + 特許条項	ツール・ライブラリの公開
GPL v3	派生物も同じライセンスで公開必須	オープンソースを望む場合
CC BY 4.0	クリエイティブ・コモンズ	データセットの公開

研究コードには **MIT ライセンス** が最も広く使われている。特別な事情がなければ MIT が無難な選択肢である。

演 習 問 題

演習 8-1 GitHub アカウントとリポジトリの作成

GitHub アカウントを作成し、GitHub Education に登録しなさい。my-first-repo という名前のリポジトリを作成し、GitHub Desktop でクローンした後、テキストファイルを 1 つ追加してコミット・プッシュしなさい。スクリーンショットを撮影して提出すること。

演習 8-2 AI で Web ページを作成する

AI チャットツールを使って、自己紹介 Web ページを作成しなさい。自分の情報を含むプロンプトを作成し、AI に HTML/CSS コードを生成させ、VS Code で編集して Live Server でプレビューすること。最低 2 回は AI にカスタマイズを依頼して改善しなさい。

演習 8-3 GitHub Pages で公開する

演習 8-2 で作成した Web ページを GitHub Pages で公開しなさい。公開 URL にアクセスして表示を確認し、公開 URL とスクリーンショットを提出すること。

演習 8-4 README.md の作成

自分の研究プロジェクト（または架空の研究プロジェクト）について、AIを活用して README.md を作成しなさい。プロジェクト概要、インストール方法、使い方（コード例付き）、ファイル構成、ライセンスを含めること。

演習 8-5 共同作業の実践

2人以上のグループで共有リポジトリを作成し、Issue を3つ以上作成して担当者を割り当て、各メンバーがブランチを作成して Pull Request とコードレビューを行いなさい。作業の流れと学んだことをレポートにまとめること。

引用・参考文献

9

就活文書の作成

【この章で学ぶこと】

- AI を活用してガクチカ・ES を効果的に作成するワークフローを習得する
- STAR 法による経験の構造化技法を身につける
- 研究内容を非専門家に分かりやすく伝える技法を理解する

9.1 大学院生の就職活動と AI

M1 の就活スケジュール

大学院修士課程の学生にとって、就職活動は研究と並行して進めなければならない重要なタスクである。

時期	活動内容
M1 4 月～6 月	自己分析、業界研究の開始
M1 6 月～8 月	サマーインターンシップ応募・参加
M1 9 月～11 月	秋冬インターンシップ、OB/OG 訪問
M1 12 月～2 月	本選考に向けた ES 準備、早期選考開始
M1 3 月～	本選考エントリー開始（広報解禁）
M2 4 月～6 月	面接・内定（選考解禁は 6 月 1 日が建前）

企業が大学院生に求めるもの

企業が大学院生を採用する際に注目するポイントは、大きく分けて2つある。

1. **専門性** — 専門知識そのものだけでなく、「深く学ぶ力」の証明。研究を通じて培った分析力、論理的思考力。技術的な課題に粘り強く取り組む姿勢。
2. **汎用スキル** — 課題発見力、仮説構築力、コミュニケーション力、プロジェクト管理力、主体性。

重要なのは、企業は「研究の専門知識をそのまま使いたい」のではなく、「研究を通じて培われた思考プロセスと行動特性」を評価しているということである。

AI を就活で活用する意義と注意点

AI 活用の意義と注意点

意義

- 自己分析の壁打ち相手として、客観的な視点を得られる
- 文章の構造化・推敲を効率的に行える
- 想定質問の生成により、面接準備の抜け漏れを減らせる
- 専門用語の平易化など、表現の調整に役立つ

注意点

- AI が生成した文章をそのまま提出すると、面接で自分の言葉で語れなくなる
- 複数の就活生が同じ AI を使うため、没個性的な文章になりやすい
- AI は「あなたの経験」を知らない。事実の入力は必ず自分で行う必要がある
- 企業によっては AI 使用を検出する仕組みを導入している場合がある

本章で提唱するアプローチは、「AI に書かせる」のではなく「AI と一緒に磨く」というスタンスである。

9.2 ガクチカとは何か

「学生時代に力を入れたこと」の本質

「ガクチカ」とは「学生時代に力を入れたこと」の略称で、ほぼすべての企業のESや面接で問われる定番質問である。ガクチカが問うているのは、「何をやったか」ではなく「どのように取り組んだか」である。

企業が見ているポイント

ポイント	説明	具体的に見ていること
課題発見力	状況の中から問題を見つける力	何を課題と認識したか
行動力	自ら考えて動く力	どのような工夫をしたか
成果	行動の結果として何が得られたか	定量的・定性的な成果
学び	経験から何を学んだか	自己成長、気づき
再現性	入社後も同様に活躍できるか	学んだことをどう活かすか

研究をガクチカにする場合の特有の課題

1. **専門用語の壁**: 研究内容をそのまま書くと、専門外の採用担当者には理解できない
2. **「やらされた」感の払拭**: 与えられたテーマの中で、自分がどのような課題意識を持ち、どう工夫したかを明確にする必要がある
3. **成果の表現**: 研究の社会的意義や、その成果が何に繋がるのかまで含めて説明する必要がある
4. **個人の貢献の明確化**: 研究室での研究はチームで行うことも多く、自分の独自の貢献を明確にしなければならない

9.3 AIを使ったガクチカ作成ワークフロー

ガクチカ作成を4つのステップに分け、各ステップでAIをどのように活用するかを解説する。

9.3.1 Step 1: 素材の棚卸し — AIに壁打ちしてもらう

最初のステップは、自分の経験を漏れなく洗い出すことである。AIを壁打ち相手にすることで、忘れていた経験や見落としとしていた側面を引き出せる。

プロンプト例

役割

あなたは大学院生の就職活動を支援するキャリアカウンセラーです。

依頼

私は修士課程の学生で、ガクチカの素材を探しています。

以下の情報をもとに、私の経験を深掘りする質問を

10個生成してください。

私の情報

- 専攻: [例: 機械工学]
- 研究テーマ: [例: 自動運転における安全性評価手法の研究]
- 研究期間: [例: 2024年4月～現在]
- その他の活動: [例: TAの経験、学会発表1回、研究室のゼミ運営]

質問の方向性: 研究の中での困難や工夫、チームワーク、

自発的な取り組み、失敗と学び、スキルアップの努力に

関する質問

このステップでは、AIに文章を書かせるのではなく、AIに「質問してもらう」ことが重要である。

9.3.2 Step 2: STAR 法での構造化

素材が見つかったら、STAR 法で構造化する。STAR 法は、経験を説得力のある形で整理するためのフレームワークである。

要素	内容	ガクチカにおける役割
Situation （状況）	どんな環境・背景だったか	読み手に場面をイメージさせる
Task （課題）	何が課題・目標だったか	取り組みの意義を示す
Action （行動）	具体的に何をしたか	自分の主体性・工夫を示す
Result （結果）	どんな成果が得られたか	行動の効果を証明する

9.3.3 Step 3: AI で文章化（400 字程度）

STAR 構造ができれば、これを読みやすい文章に変換する。

プロンプト例

役割

あなたは ES ライティングの専門家です。

依頼

以下の STAR 構造を、400 字以内のガクチカ文章にまとめてください。

STAR 構造

[Step 2 で作成した STAR 構造をここに貼る]

文章の条件

- 400 字以内（厳守）
- 最初の 1 文で「何に力を入れたか」が分かること
- 専門用語は最小限にし、非専門家でも理解できること
- 数字を使って具体性を出すこと
- 最後に「学び」や「今後への展望」を含めること

Before/After の例

Before（構造化前の素材メモ）：

自動運転の安全性の研究をしています。GSN というのをを使って安全性を評価する手法を提案しました。論文を 50 本くらい読みました。国際学会に通りました。

After（AI と磨いた完成稿）：

自動運転システムの安全性を論理的に証明する研究に力を入れました。修士研究として取り組んだのは、「なぜこのシステムは安全と言えるのか」を体系的に説明する枠組みの構築です。従来の安全性評価は、確認項目が断片的で全体像が見えにくいという課題がありました。私は 50 本以上の先行研究を精読して課題を整理した上で、安全性の論拠を木構造で可視化する手法と、システムの状態を継続的に記録する仕組みを組み合わせた新しいアプローチを考案しました。週 1 回の研究ミーティングでは、自ら論点を整理した資料を作成し、議論を主導しました。その結果、この成果をまとめた論文が安全性工学分野の国際学会に採択され、研究室初の海外口頭発表を実現しました。この経験を通じて、先行研究の徹底的な調査に基づく課題設定と、仮説を検証しながら新しい解決策を構築する力を身につけました。（350 字）

9.3.4 Step 4: AI で推敲・添削

文章化ができれば、採用担当者の視点で AI に添削してもらう。

プロンプト例

役割

あなたはメーカーの採用担当者（技術系）です。

年間 500 本以上の ES を読んでいます。

依頼

以下のガクチカ文章を、採用担当者の視点で添削してください。

添削の観点

1. 【第一印象】最初の 1 文で興味を持てるか
2. 【課題の明確さ】何が課題だったかが明確か
3. 【行動の具体性】何をしたかが具体的に書かれているか
4. 【主体性】「自分が」やったことが伝わるか
5. 【成果】成果が具体的か（定量情報があるか）
6. 【再現性】入社後も活かせそうな能力が見えるか
7. 【読みやすさ】文章は読みやすいか、冗長な部分はないか

出力形式

各観点について、○/△/×の評価と、具体的な改善提案

添削のサイクルは 2〜3 回繰り返すと効果的である。重要なのは、**AI の修正案をそのまま採用するのではなく、自分の言葉で書き直すこと**である。面接では必ずガクチカの深掘りがあり、自分の言葉で語れなければ見抜かれる。

9.4 研究を非専門家に伝える技法

エレベーターピッチ（30 秒で研究を説明）

エレベーターピッチとは、エレベーターに乗っている 30 秒程度の時間で、自分の研究を相手に伝える技法である。

30 秒エレベーターピッチの構造

1. **一言で言う**と（1 文）：研究を一言で表現
2. **なぜ重要**か（1～2 文）：社会的な文脈での意義
3. **何を**したか（1～2 文）：アプローチの概要
4. **何が**分かったか（1 文）：主要な成果

技術的な内容の平易化テクニック

専門的な表現	平易な表現（比喻）
Goal Structuring Notation 形式検証	安全性の根拠を木構造で整理する図 数学的に「絶対に間違いがない」と証明すること
機械学習モデルの過学習 API	丸暗記したけど応用問題が解けない状態 ソフトウェア同士が会話するための共通言語

- **テクニック 1: 日常の比喻を使う** — 専門概念を身近なものに例える
- **テクニック 2: 「何が嬉しいか」を先に言う** — How（何をしたか）より Why/What（なぜ重要か、何が嬉しいか）を先に伝える
- **テクニック 3: 数字で具体化する** — 「多くの先行研究」ではなく「50 本以上の論文を精読」

9.5 ES 全体での AI 活用

志望動機の作成

志望動機は、ガクチカ以上に「自分の言葉」が求められる設問である。ポイントは、AI に「文章」ではなく「論理構造」を出力させることである。構造ができたなら、自分の言葉で文章化する。

自己PRの作成

自己PRはガクチカと役割が異なる。

- **ガクチカ**: 具体的なエピソードを通じて能力を示す（帰納的）
- **自己PR**: 自分の強みを宣言し、エピソードで裏付ける（演繹的）

注意: AIに丸投げしない

ESは面接の「台本」である。ESに書いた内容は、面接で必ず深掘りされる。

AIを使って作成する場合でも、以下を心がけること。

1. **素材（事実・経験）は必ず自分で入力する** — AIは知らないことを捏造する
2. **AIの出力を鵜呑みにしない** — 自分の経験と合っているか確認する
3. **最終的な文章は自分の言葉に置き換える** — 音読して違和感がないか確認する
4. **面接で話す練習をする** — ESを見ずに、自分の言葉で3分間話せるか試す

9.6 面接準備でのAI活用

想定質問の生成

面接準備で最も効果的なAI活用は、想定質問の生成である。

プロンプト例

役割

あなたはメーカー（技術系）の面接官です。

10 年以上の面接経験があります。

依頼

以下の ES の内容を読み、面接で聞きそうな質問を

15 個生成してください。

質問のカテゴリ

- ガクチカの深掘り（5 問）
- 志望動機の深掘り（3 問）
- 研究内容に関する質問（3 問）
- 価値観・人柄に関する質問（2 問）
- 逆質問のヒント（2 問）

各質問に対して、質問文、この質問の意図、

回答のポイントを記載してください。

模擬面接（AI に面接官役をさせる）

ChatGPT や Claude などの対話型 AI を使って、模擬面接を行うことができる。1 回につき 1 つの質問をし、回答に対してフィードバックを述べてから次の質問に進む形式が効果的である。

面接後の振り返り

面接が終わった後の振り返りにも AI は活用できる。質問と自分の回答を記録し、各回答の論理構造、具体性、質問の意図への対応を AI に分析してもらう。

本章のキーポイント

1. 就活文書は「AI に書かせる」のではなく「AI と一緒に磨く」 — 素材は自分、構造化と推敲に AI を活用する
2. STAR 法は汎用的な構造化ツール — Situation → Task → Action → Result の順で整理すれば説得力が増す
3. 非専門家への説明は「Why/What」から始める — 「何をしたか」より「なぜ重要か」を先に伝える
4. ES は面接の台本 — AI の出力をそのまま使わず、自分の言葉で語れるようにする
5. AI を使った模擬面接は効果的だが、人間との練習も併用する

演習問題

演習 9-1 素材の棚卸し

本章の Step 1 のプロンプトを使い、AI に自分の経験を引き出す質問を生成してもらいなさい。生成された質問に対してすべて回答し、ガクチカの素材となりそうなエピソードを 3 つ選びなさい。

演習 9-2 STAR 法による構造化

演習 9-1 で選んだエピソードの 1 つを、STAR 法で構造化しなさい。AI を使ってもよいが、最終的な構造化は自分の言葉で記述すること。Situation, Task, Action, Result それぞれについて、3〜5 文で記述しなさい。

演習 9-3 ガクチカ文章の作成と推敲

演習 9-2 の STAR 構造をもとに、400 字以内のガクチカ文章を作成しなさい。AI に下書きを作成させた後、Step 4 の添削プロンプトで少なくとも 2 回の推敲サイクルを行い、Before（初稿）と After（最終稿）を比較して提出しなさい。

演習 9-4 エレベーターピッチの作成

自分の研究について、30 秒で説明できるエレベーターピッチを作成しなさい。以下の 2 つのバージョンを作成すること。

- 版 A: 同じ分野の研究者向け（専門用語使用可）
- 版 B: 文系の採用担当者向け（専門用語不使用）

実際に声に出して読み、30 秒以内に収まるか確認しなさい。

演習 9-5 模擬面接の実施

本章のプロンプトを使い、AI と模擬面接を実施しなさい。8 問以上のやり取りを行い、AI からのフィードバックをもとに、自分の回答の改善点を 3 つ挙げなさい。

引用・参考文献

【この章で学ぶこと】

- AI を使って自分の研究の新規性を客観的に分析・言語化できる
- これまでの章の成果物を統合した研究ポートフォリオを完成させる
- AI 活用の全体像を振り返り、今後の活用方針を立てられる

10.1 なぜ新規性の言語化が重要か

研究の価値を問われるとき、最も重視されるのは「何が新しいのか」という点である。しかし、多くの大学院生にとって、自分の研究の新規性を明確に言語化することは難しい。新規性の言語化が求められる場面は、研究生活の中で繰り返し登場する。

論文投稿（査読対応）

学術論文を投稿すると、査読者から必ず問われるのが「この研究の新規性は何か（What is the novelty of this work?）」である。Introduction で先行研究との差分が明確に記述されていなければ、どれほど実験結果が良くても、論文は採択されない。

学会発表 (Introduction での位置づけ)

学会発表のスライドでは、冒頭の Introduction で「既存研究の何が足りなくて、自分の研究が何を解決するのか」を明確に示す必要がある。

就職面接での研究説明

第9章で述べた通り、就職面接では「あなたの研究のオリジナリティは何ですか？」という質問が頻出である。

研究計画書の作成

博士課程への進学を志望する場合や、科研費などの研究費申請を行う場合、研究計画書の中で「提案する研究の新規性」を説得力を持って記述しなければならない。

これらすべての場面において、**自分の研究の新規性を客観的に分析し、適切な言葉で表現する能力**が求められる。

10.2 新規性分析の3つの観点

研究の新規性は、一つの軸だけで語れるものではない。ここでは、新規性を分析するための3つの観点を紹介する。

10.2.1 観点1: 方法論的新規性 — 新しい手法・アプローチの提案

最も典型的な新規性の形態である。既存の手法では解決できない問題に対して、新しいアルゴリズム、フレームワーク、モデル、評価手法などを提案するものである。

方法論的新規性を主張する際の表現パターン:

- “We propose a novel approach that combines X and Y to address Z”
- “Unlike existing methods that ..., our method ...”
- “To the best of our knowledge, this is the first work that ...”

10.2.2 観点2: 実践的新規性 — 実世界への適用・検証

必ずしも手法そのものが新しくなくても、既存手法を新しい領域・対象に適用し、その有効性を検証することは重要な新規性である。

実践的新規性を主張する際の表現パターン:

- “We apply X to the domain of Y for the first time”
- “We validate the effectiveness of X through a real-world case study of Y”
- “Our empirical evaluation reveals that ...”

10.2.3 観点3: 時間的観点 — 最新の研究動向の中での位置づけ

研究は常に時間軸の中に存在する。最新の技術動向、規格の改定、社会的ニーズの変化に対応することも、重要な新規性の一形態である。

時間的観点での新規性を主張する際の表現パターン:

- “With the recent advancement of X, there is an urgent need for Y”
- “In light of the latest revision of standard Z, we propose ...”
- “The emerging challenge of X necessitates a new approach to Y”

3つの観点の関係

多くの優れた研究は、これら3つの観点のうち複数を兼ね備えている。

観点	問いかけ	強い場合の特徴
方法論的新規性	手法のどこが新しいか？	新アルゴリズム、新フレームワーク
実践的新規性	どこに適用したのが新しいか？	新領域への適用、大規模実験
時間的観点	なぜ「今」この研究が必要か？	最新動向への対応、新規格対応

10.2.4 実例：自動運転安全保証研究の新規性分析

この例では、L4 自動運転フィールドトライアルにおける保証ケースを用いた合意形成の研究を題材に、3つの観点からの新規性分析を示す。

方法論的新規性：GSN ベースの保証ケースと、非専門家にも理解しやすい「安全状態レポート」を組み合わせた点が新しい。従来の GSN 単独利用では、安全性の専門家以外のステークホルダーへの伝達が困難であったが、安全状態レポートがこのギャップを埋める役割を果たす。

実践的新規性：提案手法を、実際の L4 自動運転ミニバスの実証実験（日本国内）に適用した点である。多くの先行研究が理論的なフレームワークの提案やシミュレーションに留まるのに対し、実環境で手法の有効性を検証した。

定量的評価の新規性：ステークホルダーの合意度を客観的に測定する「コンセンサススコア」を提案した。従来、合意形成の度合いを定量的に評価する標準的な手法は確立されておらず、この指標は新たな視点を提供する。

ヒント：この実例のように、新規性の主張は「既存手法 X と比べて、本研究のアプローチ Y は、Z という点で異なる」という比較構造で述べると説得力が増す。

10.3 AIを使った新規性分析ワークフロー

自分の研究の新規性を客観的に分析するために、3つのステップからなるワークフローを提案する。

10.3.1 Step 1: 研究の位置づけマッピング

プロンプト例

役割

あなたは [研究分野名] の専門家であり、
学術論文の査読経験が豊富な研究者です。

依頼

以下の研究の学術的な位置づけを分析してください。

研究の概要

- 分野: [例: 自動運転の安全性工学]
- テーマ: [例: GSN と安全状態レポートを統合した安全性保証手法の提案]
- 主要なキーワード: [例: Safety Case, GSN, Autonomous Driving, ODD]
- 使用した手法: [例: GSN による安全ケース構築、安全状態レポートとの統合]
- 主な結果: [例: ケーススタディで有効性を検証]

分析してほしい内容

1. この研究が属する学術分野・サブ分野の俯瞰図
2. 関連する研究コミュニティ（学会、ジャーナル）
3. この分野の主要な研究潮流（3～5 つ）
4. 各潮流における自分の研究の位置づけ
5. 隣接する研究分野との接点

注意: AI の回答はあくまで「仮説」として扱うこと。特に、具体的な論文名や著者名が挙げられた場合は、必ず実際にその論文が存在するか確認する必要がある（第 3 章で学んだハルシネーション対策）。

10.3.2 Step 2: 既存研究との差分分析

プロンプト例

役割

あなたは学術論文の査読者です。

依頼

以下の「自分の研究」と「先行研究」を比較し、差分を分析してください。

分析してほしい内容

1. 各先行研究と自分の研究の共通点
2. 各先行研究と自分の研究の相違点
3. 先行研究全体に共通する「限界」で、
自分の研究が解決しているもの
4. 比較表（行: 比較項目、列: 各研究）

このような比較表は、論文の Introduction や Related Work セクションで非常に効果的である。AI に下書きを作成させた後、自分で正確性を検証し、必要に応じて修正する。

10.3.3 Step 3: 新規性の言語化

プロンプト例

役割

あなたは学術英語ライティングの専門家です。

依頼

新規性分析の結果を、英語論文の Introduction の最後に置く

「本研究の貢献」パラグラフとしてまとめてください。

条件

- 3 文で構成（各文が 1 つの貢献に対応）
- 文 1: 方法論的新規性（何を提案したか）
- 文 2: 実践的新規性（何で検証したか）
- 文 3: 時間的観点（なぜ今重要か）
- “The main contributions of this paper are as follows:” で始めること

10.4 総合演習: 研究ポートフォリオの完成

本章の後半は、これまでの章で作成してきた成果物を統合し、研究ポートフォリオとして完成させる総合演習である。

これまでの成果物の振り返り

章	成果物	内容
第 3 章	文献調査レポート	先行研究のサーベイ
第 4 章	英語 Abstract	英文要旨（150～250 語）
第 5 章	発表スライド	学会発表やゼミ発表用
第 6 章	研究コード	実験・分析に使用したコード
第 7 章	データ分析・グラフ	データの可視化と分析結果
第 8 章	GitHub Web ページ	研究成果を公開する Web ページ
第 9 章	ガクチカ	就職活動用の ES 文章
第 10 章	新規性分析レポート	研究の新規性を整理した文書

GitHub リポジトリでの整理と公開

これらの成果物を、一つの GitHub リポジトリとして体系的に整理する。

```
my-research-portfolio/
├── README.md           ← リポジトリの概要
├── docs/
│   ├── literature-review/ ← 文献調査レポート
│   ├── abstract/         ← 英語 Abstract
│   ├── novelty-analysis/ ← 新規性分析レポート
│   └── job-search/       ← ガクチカ（公開範囲に注意）
├── slides/             ← 発表スライド
├── src/                 ← 研究コード
├── data/                ← データ分析・グラフ
├── .gitignore
└── LICENSE
```

10.5 AI 活用の振り返りと今後

AI が特に効果的だった場面

場面	効果	理由
文章の構造化	非常に高い	パターン認識と整理が得意
英語表現の改善	高い	大量の英語テキストで学習
コードの雛形生成	高い	定型的なパターンの再現が得意
アイデアの壁打ち	高い	多角的な視点を提供できる
想定質問の生成	高い	多様なパターンを網羅的に生成

AI の限界が明らかだった場面

場面	限界	対処法
事実の正確性	ハルシネーションのリスク	必ず原典で検証
最新情報の把握	学習データの時点で止まる	最新論文は自分で確認
研究の独創性	既存パターンの組み合わせが限界	本質的な着想は人間の役割
実験の設計	分野固有の暗黙知が不足	指導教員や先輩に相談
感情・動機の表現	表面的になりがち	自分の言葉で書く

今後の AI 進化への対応

ツールは変わっても、原理は変わらない。本書で学んだ以下の原理は、どのような AI ツールが登場しても適用できる。

AI 活用の 4 つの普遍的原理

1. **明確な指示の重要性** — ROCF (Role, Output, Context, Format) の考え方は、あらゆる AI ツールに応用できる。
2. **段階的な作業プロセス** — 複雑なタスクを段階に分けて進めることで、人間が各段階で検証・修正できる。
3. **批判的検証の習慣** — AI の出力に含まれる微妙な誤りを見つけられるのは、専門知識を持つ人間だけである。
4. **人間の判断が必要な領域の認識** — 研究の方向性を決める、仮説を立てる、実験結果を解釈する、倫理的な判断をする——これらは AI が「代替」できない。

「AI を使いこなす」研究者になるために

AI の普及により、「AI を使える人」と「使えない人」の間に生産性の格差が生まれつつあるという指摘がある。しかし、より本質的な格差は、「AI に使われる人」と「AI を使いこなす人」の間にある。

「AI に使われる人」の特徴:

- AI の出力をそのまま使う
- AI なしでは作業できなくなる
- AI の回答が正しいかどうか判断できない

「AI を使いこなす人」の特徴:

- AI の出力を批判的に検証し、取捨選択する
- AI を使って自分の思考を加速・拡張する
- AI の限界を理解し、適切な場面で活用する

AI は道具である。有用な道具であるが、あくまで道具である。あなたの研究は、あなたにしかできない知的営みである。AI はその営みを支える強力な助手であり、本書がその助手との付き合い方を学ぶ一助となれば幸いである。

本章のキーポイント

1. **新規性は3つの観点で分析する** — 方法論的新規性、実践的新規性、時間的観点
2. **新規性分析は段階的に行う** — 位置づけマッピング→差分分析→言語化の3ステップ
3. **成果物は統合して初めて価値が最大化する** — GitHub リポジトリとして体系的に整理する
4. **ツールは変わっても原理は変わらない** — 明確な指示、段階的プロセス、批判的検証、人間の判断
5. **「AIを使いこなす」研究者を目指す** — AIは助手であり、研究の本質は人間の知的営みにある

演習問題

演習 10-1 新規性の3観点分析

自分の研究（または取り組んでいるテーマ）について、方法論的新規性、実践的新規性、時間的観点の3つの観点から新規性を分析しなさい。各観点について、3～5文で記述すること。

演習 10-2 先行研究との比較表の作成

自分の研究テーマに関する先行研究を3本以上取り上げ、比較表の形式で自分の研究との差分を整理しなさい。比較項目は5つ以上設定すること。AIを使って下書きを作成してもよいが、各セルの内容の正確性は自分で検証すること。

演習 10-3 貢献記述の作成

演習 10-1 と 10-2 の結果をもとに、英語論文の Introduction で使える「本研究の貢献」パラグラフ（3文）を作成しなさい。日本語訳も併記すること。

演習 10-4 研究ポートフォリオの構築

本書の各章で作成した成果物を含む GitHub リポジトリを作成しなさい。
README.md を英語で記述し、リポジトリの構成、各成果物の概要、実行方法（コードがある場合）を記載すること。

演習 10-5 AI 活用方針書の作成

本書で学んだ内容を踏まえ、今後の研究活動における AI 活用方針書（A4 用紙 1 枚程度）を作成しなさい。以下の項目を含めること。

- 自分の研究活動で AI を活用する場面のリスト
- 各場面での AI 活用の具体的な方法
- AI に任せない（人間が判断する）領域の明記
- AI 使用の倫理的ガイドライン（自分なりの基準）
- 半年後に見直す予定日

引用・参考文献

A

主要 AI ツールリファレンス

注記: AI ツールの開発・更新は非常に速いペースで進んでいる。本付録の情報は 2026 年 3 月時点のものであり、最新のツール情報・料金体系については、本書の Web サポートページを参照されたい。

A.1 対話型 AI（汎用）

ツール名	開発元	料金	得意分野
ChatGPT	OpenAI	無料版あり/Plus \$20/月	汎用的な対話、文章・コード生成
Claude	Anthropic	無料版あり/Pro \$20/月	長文の分析・生成、論理的推論
Gemini	Google	無料版あり/Advanced \$20/月	Google 連携、マルチモーダル
Copilot	Microsoft	無料版あり/Pro \$20/月	Office 連携、Web 検索統合

選び方のポイント: 研究用途では、長文の分析能力と正確性が重要である。複数のツールを試し、自分の研究分野で最も精度の高い回答を返すツールを見つけることを推奨する。

A.2 検索特化型 AI

ツール名	開発元	料金	得意分野
Perplexity	Perplexity AI	無料版あり/Pro \$20/月	出典付き検索、最新情報の収集
Consensus	Consensus	無料版あり/Premium \$12/月	学術論文に特化した検索・要約
Elicit	Elicit	無料版あり/Plus \$12/月	学術論文の検索・分析・要約

選び方のポイント: 文献調査の初期段階では、学術論文に特化した Consensus や Elicit が有効である。一般的な情報収集には Perplexity が便利だが、学術的な正確性を求める場合は Google Scholar との併用を推奨する。

A.3 論文分析・執筆支援 AI

ツール名	開発元	料金	得意分野
Semantic Scholar	Allen AI	無料	論文検索、引用分析
Connected Papers	Connected Papers	無料版あり/月額\$3～	論文の関連性グラフ可視化
Writefull	Writefull	無料版あり/Premium 16 ユーロ/月	学術英語の文法・表現チェック
Grammarly	Grammarly	無料版あり/Premium \$12/月	英文の文法・スタイルチェック
DeepL Write	DeepL	無料版あり/Pro 8 ユーロ/月	英文の改善・書き換え提案

選び方のポイント: 論文執筆では、Writefull のように学術英語に特化したツールが有効である。Grammarly や DeepL Write は一般的な英文チェックに優れるが、学術特有の表現については専門ツールが勝る。

A.4 コード支援AI

ツール名	開発元	料金	得意分野
GitHub Copilot	GitHub/OpenAI	無料（学生）/\$10/月	コード補完・生成
Claude Code	Anthropic	Pro/Max プランに含まれる	コード生成、デバッグ
Cursor	Cursor	無料版あり/Pro \$20/月	AI 統合エディタ
Google Colab AI	Google	無料版あり/Pro \$12/月	Python コード支援

選び方のポイント: 研究でプログラミングを行う場合、GitHub Copilot は最も広く使われている。学生は無料で利用できるため、まずこれを試すことを推奨する。より高度なコード生成やプロジェクト全体の理解が必要な場合は、Claude Code や Cursor が有効である。

A.5 デザイン・プレゼンテーション AI

ツール名	開発元	料金	得意分野
Canva (AI 機能)	Canva	無料版あり/Pro \$13/月	スライド・ポスターデザイン
Gamma	Gamma	無料版あり/Plus \$8/月	AI 自動スライド生成
Beautiful.ai	Beautiful.ai	\$12/月～	デザイン自動調整スライド
Mermaid	オープンソース	無料	テキストから図表生成

選び方のポイント: 学会発表用のスライドは、 $\text{\LaTeX}$ Beamer や PowerPoint で作成するのが一般的だが、Gamma を使って初期構成を自動生成し、その後手動で調整するワークフローも効果的である。図表の生成には Mermaid が便利で、Markdown 文書に直接埋め込める。

B

プロンプトテンプレート集

本付録では、本書で紹介したプロンプトのエッセンスをテンプレートとして整理する。各テンプレートには、ROCF (Role, Output, Context, Format) の各要素がどこに対応するかを注釈として示している。

ROCF 要素の凡例

- **【R】** Role (役割) : AI に与える役割設定
- **【O】** Output (出力) : 求める出力の内容
- **【C】** Context (文脈) : 背景情報や条件
- **【F】** Format (形式) : 出力の形式指定

B.1 文献調査用プロンプト

プロンプト例

テンプレート 1: サーベイの起点を見つける

【R】あなたは[分野名]の研究者で、文献調査の専門家です。

【O】以下のテーマに関するサーベイ論文・レビュー論文を
探すための検索戦略を提案してください。

【C】テーマ: [テーマ] / 知りたいこと: [具体的な問い] /
すでに知っていること: [既知の情報]

【F】推奨する検索キーワード（英語）5セット /
推奨する学術データベース / 注意すべき関連分野

プロンプト例

テンプレート 2: 論文の要約を依頼する

【R】あなたは学術論文の読解に精通した研究アシスタントです。

【O】以下の論文情報を、構造化された要約にまとめて
ください。

【C】タイトル: [タイトル] / 著者: [著者] /
概要: [Abstract または要点]

【F】研究の目的（2文以内） / 手法の概要（3文以内） /
主な結果（箇条書き3点） / 限界・今後の課題（2点） /
自分の研究との関連（※ここは自分で記入すること）

プロンプト例

テンプレート 3: 研究動向の整理

【R】あなたは[分野名]の動向に詳しい研究者です。

【O】以下のキーワードに関する研究動向を整理してください。

【C】キーワード: [キーワード] / 期間: 過去 [5] 年間 /

対象: [学会名/ジャーナル名]

【F】主要な研究潮流（3～5つ） / 最近のトレンド・転換点 /
未解決の課題・今後の方向性

※ 挙げられた論文は必ず実在を確認すること

プロンプト例

テンプレート 4: 論文間の関係性の整理

【R】あなたは文献レビューの専門家です。

【O】以下の複数の論文の関係性を整理してください。

【C】論文 1: [タイトル, 著者, 年] — [要点]

論文 2: [タイトル, 著者, 年] — [要点]

論文 3: [タイトル, 著者, 年] — [要点]

【F】各論文の共通点と相違点を表形式で /
研究の発展の流れ（時系列） / 未解決の問題

プロンプト例

テンプレート 5: 批判的読解の支援

【R】あなたは厳格な査読者です。

【O】以下の論文の主張について、批判的な視点からの質問を生成してください。

【C】論文の主張: [主張] / 使用された手法: [手法] /
実験の規模: [データ数等]

【F】以下の観点から各 3 問ずつ: 方法論的な妥当性 /
結果の解釈 / 一般化可能性

B.2 英語論文用プロンプト

プロンプト例

テンプレート 1: Abstract の英訳・改善

【R】あなたは学術英語ライティングの専門家です。

[分野名] の論文に精通しています。

【O】以下の日本語要旨を、学術英語の Abstract として書き直してください。

【C】[日本語の要旨] / 対象ジャーナル/学会: [名称] /
字数制限: [語数] 語以内

【F】構成: Background → Purpose → Methods →
Results → Conclusion / 学術英語の慣習に従った表現 /
出力後、使用した主要な学術表現のリストを付記

プロンプト例

テンプレート 2: Introduction の段落構成

【R】あなたは学術論文のライティングコーチです。

【O】以下の情報をもとに、Introduction の段落構成案を作成してください。

【C】研究分野: [分野] / 研究の背景: [2-3 文] /

既存研究の課題: [箇条書き] / 本研究の提案: [1-2 文]

【F】各段落の役割 / 使うべき英語表現の候補 (2-3 個) / 段落間の接続の仕方

プロンプト例

テンプレート 3: 英文の校正・改善

【R】あなたはネイティブの学術英語校正者です。

【O】以下の英文を校正し、より学術的に洗練された表現に改善してください。

【C】[校正対象の英文] / 論文のセクション:

[Introduction / Methods / Results / Discussion] /

分野: [分野名]

【F】修正箇所を明示 (元の表現→修正後の表現) /

各修正の理由を簡潔に説明 / 全体的な改善提案

プロンプト例

テンプレート 4: 査読コメントへの回答文作成

【R】あなたは学術論文の査読対応に精通した研究者です。

【O】以下の査読コメントに対する回答文の下書きを作成してください。

【C】査読コメント: [コメント] /

対応方針: [どのように対応するか]

【F】英語で記述 / “We thank the reviewer for...” で始める丁寧な書き出し / 論文の修正箇所を明示

プロンプト例

テンプレート 5: 関連研究セクションの構成

【R】あなたは学術論文の Related Work セクションの構成アドバイザーです。

【O】以下の先行研究リストを、Related Work セクションとして効果的に構成する方法を提案してください。

【C】[先行研究のリスト] / 本研究のテーマ: [テーマ]

【F】サブセクションの分け方 /

各サブセクションで述べるべき内容 /

先行研究から本研究への橋渡しの書き方

B.3 コード生成用プロンプト

プロンプト例

テンプレート 1: 分析コードの雛形生成

【R】あなたは Python のデータ分析に精通したプログラマーです。

【O】以下の分析タスクを実行する Python コードを生成してください。

【C】目的: [例: CSV ファイルからデータを読み込み、基本統計量を算出] / データの形式: [例: CSV、列名は A, B, C] / 使用ライブラリ: [例: pandas, matplotlib]

【F】コメント付きの読みやすいコード / 各ステップの説明 / エラーハンドリングを含む

プロンプト例

テンプレート 2: 既存コードのリファクタリング

【R】あなたはコードレビューの専門家です。

【O】以下のコードをリファクタリングしてください。

【C】[リファクタリング対象のコード] / 問題点: [例: 可読性が低い、重複が多い]

【F】リファクタリング後のコード / 変更点のリスト（何を、なぜ変更したか）

プロンプト例

テンプレート 3: テストコードの生成

【R】あなたはソフトウェアテストの専門家です。

【O】以下の関数に対するユニットテストを生成してください。

【C】[テスト対象の関数] / テストフレームワーク: [例: pytest]

【F】正常系テスト (3 ケース以上) /

異常系テスト (2 ケース以上) /

エッジケーステスト (2 ケース以上)

プロンプト例

テンプレート 4: エラーのデバッグ支援

【R】あなたは Python のデバッグに精通したエンジニアです。

【O】以下のエラーの原因と修正方法を教えてください。

【C】コード: [エラーが発生するコード] /

エラーメッセージ: [エラーメッセージ] /

実行環境: [例: Python 3.11, macOS]

【F】エラーの原因の説明 (初学者にも分かるように) /

修正後のコード / 同様のエラーを防ぐためのヒント

プロンプト例

テンプレート 5: README.md の生成

【R】あなたはオープンソースプロジェクトのドキュメント専門家です。

【O】以下のプロジェクト情報から、GitHub 用の README.md を生成してください。

【C】プロジェクト名: [名前] / 概要: [1-2 文] /

使用言語: [例: Python] / 主な機能: [箇条書き]

【F】英語で記述 / 構成: Title → Badge → Overview →

Installation → Usage → Project Structure → License

B.4 データ分析用プロンプト

プロンプト例

テンプレート 1: 分析計画の立案

【R】あなたはデータサイエンティストです。

【O】以下のデータと研究目的に対する分析計画を
提案してください。

【C】研究目的: [目的] /

データの概要: [データの種類、サイズ、変数] /

仮説: [検証したい仮説]

【F】推奨する分析手法（3つ以上）と理由 /

分析の手順（ステップバイステップ）/ 必要な前処理 /

結果の解釈の仕方

プロンプト例

テンプレート 2: グラフの改善提案

【R】あなたはデータ可視化の専門家です。

【O】以下のグラフの改善提案をしてください。

【C】グラフの種類: [例: 棒グラフ] /

何を示したいか: [意図] / 現在の問題: [例: 見づらい] /

使用ツール: [例: matplotlib]

【F】改善ポイント（5つ以上）/ 改善後のコード例 /

なぜその改善が効果的かの説明

プロンプト例

テンプレート 3: 統計的検定の選択

【R】あなたは統計学の専門家です。

【O】以下の研究状況に適した統計的検定手法を
推薦してください。

【C】比較したいもの: [例: 2 群の平均値の差] /
データの性質: [例: 正規分布かどうか、対応の有無] /
有意水準: [例: 0.05]

【F】推薦する検定手法とその理由 / 前提条件の確認方法 /
Python での実装コード / 結果の解釈の仕方 /
論文での記載例 (英語)

B.5 スライド作成用プロンプト

プロンプト例

テンプレート 1: スライド構成の立案

【R】あなたは学会発表のプレゼンテーションコーチです。

【O】以下の研究内容を、[発表時間] 分の学会発表
スライドの構成案にしてください。

【C】研究タイトル: [タイトル] / 対象学会: [学会名] /
聴衆: [同分野の研究者/広い分野/実務者] /
研究の概要: [3-5 文]

【F】各スライドについて: スライドタイトル /
含めるべき内容 / 推定所要時間 / 注意点

プロンプト例

テンプレート 2: スライドの文字量削減

【R】あなたはプレゼンテーションデザインの専門家です。

【O】以下のスライドの内容を、視覚的に見やすくなるよう文字量を削減してください。

【C】現在のスライド内容: [テキスト] / 最も伝えたいこと: [1 文]

【F】削減後のテキスト（元の 50%以下の文字数） /
キーワードの強調提案 / 図表化できる部分の提案

プロンプト例

テンプレート 3: Q&A 想定質問の準備

【R】あなたは [分野名] の学会に参加する研究者です。

【O】以下の発表内容に対して、Q&A セッションで出そうな質問を生成してください。

【C】発表タイトル: [タイトル] / 発表の概要: [3-5 文] /
使用した手法: [手法] / 主な結果: [結果]

【F】以下のカテゴリに分けて各 3 問:
手法に関する質問 / 結果の解釈に関する質問 /
今後の展望に関する質問 / 想定される厳しい質問
各質問に回答のポイントを付記

B.6 ガクチカ・ES 用プロンプト

プロンプト例

テンプレート 1: 経験の棚卸し

【R】あなたは大学院生専門のキャリアカウンセラーです。

【O】以下の情報をもとに、ガクチカの素材を引き出すための深掘り質問を 10 個生成してください。

【C】専攻: [専攻] / 研究テーマ: [テーマ] /

研究以外の活動: [箇条書き] /

大学院生活で苦労したこと: [箇条書き]

【F】各質問に対して: 質問文 /

この質問が引き出そうとしているポイント / 回答のヒント

プロンプト例

テンプレート 2: STAR 法での構造化

【R】あなたは ES ライティングの専門コーチです。

【O】以下の経験メモを、STAR 法で構造化してください。

【C】[経験の箇条書きメモ] / 志望業界: [業界]

【F】S (状況) : [3 文以内] / T (課題) : [2 文以内] /

A (行動) : [3-4 文] /

R (結果) : [2 文以内、定量的な成果を含む]

+ 各要素の改善ポイント

プロンプト例

テンプレート 3: 採用担当者視点での添削

【R】あなたは [業界名] の採用担当者で、
年間 500 本以上の ES を読んでいます。

【O】以下のガクチカを、7つの観点で評価・添削して
ください。

【C】[ガクチカの文章] / 志望職種: [職種]

【F】以下の各観点について ○/△/× と改善提案:

- (1) 第一印象 (2) 課題の明確さ (3) 行動の具体性
 - (4) 主体性の伝わり方 (5) 成果の定量性
 - (6) 入社後の再現性 (7) 全体の読みやすさ
- +修正案（修正箇所を明示）

C | 学術英語表現集

論文の各セクションで使える英語表現を整理する。これらの表現は、AI に英文生成を依頼する際の参考にもなるし、AI の出力をチェックする際の基準にもなる。

C.1 Introduction で使える表現

研究背景の記述

日本語の意図	英語表現
近年、～が注目されている	In recent years, ... has attracted considerable attention.
～は重要な課題である	... remains a critical challenge in the field of ...
～の需要が高まっている	There is a growing demand for ...
～は広く研究されてきた	... has been extensively studied in the literature.
～の進展に伴い	With the rapid advancement of ...
～は～において重要な役割を果たす	... plays a crucial role in ...
～はますます重要になっている	... has become increasingly important.
～の分野では	In the domain of ...
～が実用化されつつある	... is being deployed in real-world applications.
～に関する研究が活発化している	A growing body of research has focused on ...

研究目的の記述

日本語の意図	英語表現
本研究では～を提案する	In this paper, we propose ...
本研究の目的は～である	The aim of this study is to ...
本研究は～に取り組む	This work addresses the problem of ...
～を明らかにすることを目指す	We aim to elucidate ...
本研究の主な貢献は以下の通りである	The main contributions of this paper are as follows:
我々の知る限り、初めて	To the best of our knowledge, this is the first study to ...

論文の構成の記述

日本語の意図	英語表現
本論文は以下のように構成される	The remainder of this paper is organized as follows.
第2節では～について述べる	Section 2 describes ...
第3節では～を紹介する	Section 3 presents ...
最後に、第6節で結論を述べる	Finally, Section 6 concludes the paper.

C.2 Methods で使える表現

手 法 の 説 明

日本語の意図	英語表現
提案手法の概要を述べる	We provide an overview of the proposed method.
～のアルゴリズムを以下に示す	The algorithm for ... is presented below.
本研究では～を採用した	In this study, we adopted ...
～を～に適用した	We applied ... to ...
～は以下の 3 ステップから成る	... consists of the following three steps.
～に基づいて	Based on ... / Building upon ...
～を拡張して	By extending ... / We extend ... to ...
図 X は～の概要を示す	Figure X illustrates the overview of ...
～と定義する	We define ... as ...
～を仮定する	We assume that ...

実験条件の記述

日本語の意図	英語表現
実験環境は以下の通りである	The experimental setup is as follows.
～を用いて評価を行った	We conducted the evaluation using ...
パラメータは～に設定した	The parameter was set to ...
～を比較手法として用いた	We used ... as a baseline for comparison.
実験は～回繰り返した	The experiment was repeated ... times.
～の条件下で	Under the condition of ...

C.3 Results で使える表現

結果の記述

日本語の意図	英語表現
表 X は～の結果を示す	Table X shows the results of ...
図 X に～を示す	Figure X presents ...
結果から～が分かる	The results indicate that ...
～が観察された	... was observed.
～は～を上回った	... outperformed ...
～において有意な差が見られた	A significant difference was found in ...
～は～と一致する	... is consistent with ...
予想通り、～であった	As expected, ...
興味深いことに、～であった	Interestingly, ...
～は～%の改善を示した	... showed an improvement of ...% over ...

比較の記述

日本語の意図	英語表現
～と比較して	Compared with ... / In comparison with ...
～に対して	In contrast to ... / As opposed to ...
同様に	Similarly, ... / Likewise, ...
一方で	On the other hand, ... / Meanwhile, ...
特に	In particular, ... / Notably, ...

C.4 Discussion/Conclusion で使える表現

考 察

日本語の意図	英語表現
この結果は～を示唆している	These results suggest that ...
～の理由として考えられるのは	A possible explanation for ... is that ...
この知見は～と整合する	This finding is in line with ...
～の観点から考えると	From the perspective of ...
～は～によって説明できる	... can be attributed to ...
さらなる調査が必要である	Further investigation is warranted.

限 界 の 記 述

日本語の意図	英語表現
本研究にはいくつかの限界がある	This study has several limitations.
本研究の限界として	A limitation of this study is that ...
～は本研究の範囲外である	... is beyond the scope of this study.
データセットの規模が限られている	The dataset size is limited.
一般化には注意が必要である	Caution is needed when generalizing ...

展 望・結 論

日本語の意図	英語表現
今後の研究では～を検討する	In future work, we plan to investigate ...
～への拡張が期待される	Extension to ... is a promising direction.
本研究は～に貢献する	This study contributes to ...
結論として	In conclusion, ... / To summarize, ...
本研究で提案した手法は～に有用である	The proposed method is useful for ...

C.5 接続表現・論理展開表現

追加・列挙

機能	表現
さらに	Furthermore, / Moreover, / In addition, / Additionally,
第一に…第二に	First, ... Second, ... / Firstly, ... Secondly, ...
同様に	Similarly, / Likewise, / In the same vein,

対比・逆接

機能	表現
しかし	However, / Nevertheless, / Nonetheless,
一方で	On the other hand, / Conversely, / In contrast,
～にもかかわらず	Despite ... / In spite of ... / Notwithstanding ... [†]
～であるが	Although ... / While ... / Whereas ...

因果・理由

機能	表現
したがって	Therefore, / Thus, / Hence, / Consequently,
～のために	Due to ... / Owing to ... / Because of ...
その結果	As a result, / As a consequence,
～を考慮すると	Considering ... / Given that ...

例示・具体化

機能	表現
例えば	For example, / For instance, / e.g.,
具体的には	Specifically, / More specifically, / In particular,
すなわち	That is, / Namely, / i.e.,

要 約・結 論

機能	表現
要するに	In summary, / To summarize, / In brief,
結論として	In conclusion, / To conclude,
全体として	Overall, / On the whole, / Taken together,

D

15回授業計画

本書を教科書として半期 15 回の授業で使用する場合の授業計画を示す。各回 90 分の授業を想定し、講義と演習のバランスを考慮している。

D.1 授業計画表

回	対応章	テーマ	授業の進め方	課題
1	第 1 章	研究活動と生成 AI 入門	【講義 60 分】AI 活用の意義と倫理、批判的 AI 活用の考え方を解説。【演習 30 分】各自が普段使っている AI ツールとその使い方を共有するワークショップ。	自分の研究活動における AI 活用の現状と課題を 400 字でまとめる
2	第 2 章前半	プロンプトエンジニアリング基礎	【講義 40 分】ROCF (Role, Output, Context, Format) の解説と実例紹介。【演習 50 分】同じタスクに対して異なるプロンプトを試し、出力の違いを比較する実践演習。	自分の研究テーマに関するプロンプトを ROCF で設計し、3 パターン作成
3	第 2 章後半	プロンプトエンジニアリング実践	【講義 30 分】Few-shot, Chain-of-Thought, 段階的プロンプトの解説。【演習 60 分】研究テーマに関する実践的なプロンプト作成と改善のサイクルを体験。	最も効果的だったプロンプトとその改善過程をレポートにまとめる

(次ページに続く)

回	対応章	テーマ	授業の進め方	課題
4	第 3 章	AI を活用した 文献調査	【講義 30 分】AI 時代の文献 調査戦略、ハルシネーショ ン対策。【演習 60 分】自分 の研究テーマについて、AI を使って文献サーベイの初 期段階を実施。	文 献 調 査 レ ポート（先行 研究 5 本以上 の要約と比較 表）
5	第 4 章	英語論文の読 解と執筆支援	【講義 40 分】学術英語の構 造、AI を使った英文作成・ 校正の方法。【演習 50 分】 自分の研究の Abstract を 日本語→英語で作成する実 践。	自分の研究の 英語 Abstract (150-250 語) を作成
6	第 5 章	プレゼンテー ション準備	【講義 30 分】効果的な学会 発表の構成、AI を使ったス ライド作成。【演習 60 分】 5 分間の研究紹介スライド (5-7 枚) の構成を AI と共 に設計。	研究紹介スラ イドの完成版 を提出
7	第 6 章前半	プログラミング と AI (基礎)	【講義 30 分】AI 支援コード ディングの原則、GitHub Copilot 等の使い方。【演習 60 分】自分の研究に関連す るデータ処理コードを AI と共に作成。	作成したコード と AI とのやり 取りの記録 を提出
8	第 6 章後半	プログラミング と AI (応用)	【講義 20 分】コードレビ ュー、テスト、デバッグへの AI 活用。【演習 40 分】前回 のコードのリファクタリン グとテスト作成。【発表 30 分】中間発表 (3 分×12 名 程度)。	コードを改善 し、README 付きで提出
9	第 7 章	データ分析・可 視化と AI	【講義 30 分】AI を使った データ分析の計画立案、グ ラフ作成の原則。【演習 60 分】自分の研究データ（また はサンプルデータ）を使っ た分析と可視化の実践。	データ分析結 果のグラフ 3 枚と各グラフ の解釈を記述 したレポート
10	第 8 章	GitHub Web ページの作成	【講義 20 分】GitHub Pages の仕組み、研究成果の Web 公開の意義。【演習 70 分】 自分の研究紹介 Web ペー ジの作成 (GitHub Pages)。	GitHub Pages による Web ページを 公開し、URL を提出

(次ページに続く)

回	対応章	テーマ	授業の進め方	課題
11	第 9 章前半	ガクチカ・ES 作成（基礎）	【講義 30 分】就活スケジュール、企業の評価ポイント、STAR 法の解説。【演習 60 分】AI を使った経験の棚卸しと STAR 法での構造化。	STAR 法で構造化した経験メモ（2 テーマ分）
12	第 9 章後半	ガクチカ・ES 作成（実践）	【講義 20 分】文章化のテクニック、面接との連動。【演習 70 分】ガクチカの文章化→ AI 添削→修正のサイクルを 2 回以上実施。ペアで相互レビュー。	ガクチカ 完 成版（400 字 以 内）、Be- fore/After 比 較付き
13	第 10 章前半	研究の新規性 分析	【講義 30 分】新規性の 3 つの観点、言語化の方法。【演習 60 分】自分の研究について新規性分析ワークフローを実施。	新規性分析レ ポート（テンプ レートに沿っ て作成）
14	第 10 章後半	研究ポートフ ォリオの統合	【講義 20 分】ポートフォリオの構成、GitHub リポジトリの整理方法。【演習 70 分】これまでの成果物を GitHub リポジトリとして統合・整理。	研 究 ポ ー ト フ ォ リ オ （GitHub リポ ジトリ）の完 成版を提出
15	総合	最終発表と振 り返り	【発表 60 分】研究ポートフォリオのプレゼンテーション（5 分× 12 名程度）。【振り返り 30 分】AI 活用の全体振り返り、今後の活用方針の策定。	AI 活用方針書 （A4 用紙 1 枚） を最終課題と して提出

D.2 授業運営のヒント

事前準備

- 受講者全員が ChatGPT または Claude のアカウントを持っていることを確認する（第 1 回の前に案内）
- GitHub アカウントの作成も第 1 回の前に案内する
- GitHub Copilot の学生無料枠の申請は手続きに時間がかかるため、早めに案内する

演習の進め方

- 演習時間は、受講者が実際に AI と対話しながら作業する時間である。教員は巡回してサポートする
- 可能であれば、受講者同士のペアワーク・グループワークを積極的に取り入れる。他の人のプロンプトや使い方を見ることが大きな学びとなる
- 「うまくいかなかった例」も学びの素材として共有する

評価方法の案

評価項目	配分	評価基準
各回の課題	40%	期限内の提出、指示への対応度
中間発表	10%	研究内容の伝達力、スライドの質
研究ポートフォリオ	30%	成果物の質と統合度、GitHub の整理状況
最終発表	10%	プレゼンテーション力、質疑応答
AI 活用方針書	10%	振り返りの深さ、今後への具体的な方針

ポートフォリオの評価ルーブリック

観点	A（優秀）	B（良好）	C（合格）	D（要改善）
成果物の網羅性	全章の成果物が揃っている	大半の成果物が揃っている	必須の成果物は揃っている	成果物が不足している
成果物の質	高品質で研究内容が正確に反映	一定の質を持つ	最低限の要件を満たしている	質に問題がある
AI 活用の適切さ	効果的に活用しつつ批判的検証ができている	概ね適切に検証している	AI を使っているが検証が不十分	AI の出力を無批判に使用
GitHub の整理	README 充実、構造明確、再現性確保	構造が概ね整理されている	最低限の構造がある	整理されていない
オリジナリティ	独自の工夫や視点が随所に見られる	一部に独自の工夫がある	テンプレート通り	独自性がない

D.3 授業の変形パターン

本書は 15 回の授業を想定しているが、以下のような変形も可能である。

8 回集中講座の場合

回	内容
1	第 1 章+第 2 章: AI 基礎とプロンプトエンジニアリング
2	第 3 章+第 4 章: 文献調査と英語論文
3	第 5 章: プレゼンテーション
4	第 6 章: プログラミング
5	第 7 章: データ分析
6	第 8 章+第 9 章: Web 公開と就活文書
7	第 10 章: 新規性分析とポートフォリオ
8	最終発表

自主ゼミ（週 1 回、2 時間）の場合

各回の演習部分を充実させ、参加者同士のディスカッションを中心に進める。講義部分は事前に本書の該当章を読んできてもらい、演習とディスカッションに時間を割く反転授業形式が効果的である。